

Chapter 1: Introduction to Data Structures and Algorithms

1. What are Data Structures and Algorithms?

A **data structure** is a specialized format for organizing, processing, retrieving, and storing data. It defines the relationship between data elements and the operations that can be performed on them. Examples include arrays, linked lists, stacks, queues, trees, and hash tables.

Real-life analogy: Think of a data structure as a way of organising a library. Books can be arranged alphabetically (like an array), grouped by genre (like a hash table), or stacked in a pile (like a stack). The organisation method determines how quickly you can find, add, or remove a book.

An **algorithm** is a finite sequence of well-defined, step-by-step instructions for solving a specific problem or accomplishing a task. Algorithms transform input data into desired output through a series of computational steps.

Real-life analogy: A recipe for baking a cake is an algorithm—it takes ingredients (input), follows a fixed set of instructions (process), and produces a cake (output). Similarly, a GPS navigation system runs an algorithm to find the shortest route from your location to a destination.

The synergy between data structures and algorithms is fundamental: the choice of data structure directly impacts the efficiency of an algorithm, and conversely, algorithms are often designed to leverage the properties of specific data structures.

2. Why DSA Matters in Software Development and Interviews

In Software Development

- **Performance optimization:** Efficient DSA choices reduce computational cost, memory usage, and response times in production systems.
- **Scalability:** Systems handling millions of users or terabytes of data require algorithms that scale sub-linearly or log-linearly.

- **Resource management:** Embedded systems, real-time applications, and high-frequency trading platforms demand predictable time and space bounds.
- **Problem solving:** Many real-world problems (e.g., routing, scheduling, searching, sorting) have known DSA solutions that avoid reinventing the wheel.

In Technical Interviews

- **Problem-solving signal:** DSA questions test analytical thinking, pattern recognition, and coding fluency under constraints.
- **Common framework:** Most major tech companies (FAANG, Microsoft, etc.) use DSA problems to standardize candidate evaluation.
- **Transferable skill:** Mastery of DSA enables candidates to tackle unfamiliar problems by reducing them to known patterns (e.g., sliding window, dynamic programming, graph traversal).

3. Algorithm Efficiency: Time and Space Complexity

Complexity analysis quantifies the resources an algorithm consumes as a function of its input size n .

- **Time complexity:** The amount of computational time (number of elementary operations) an algorithm takes to run.
- **Space complexity:** The amount of additional memory (excluding input storage) an algorithm allocates during execution.

Both are expressed using asymptotic notation, which describes the limiting behaviour as n grows arbitrarily large.

Real-life analogy: Compare two ways to find a word in a dictionary. Skimming every page (linear search) takes time proportional to the dictionary's thickness. Opening the dictionary at the middle and repeatedly halving the search zone (binary search) is dramatically faster—this difference is captured by time complexity.

Why analyse complexity?

- Compare algorithms independently of hardware or programming language.
- Identify bottlenecks and scalability limits.
- Predict performance for large inputs before implementation.

4. Asymptotic Notations

Asymptotic notations describe the growth rate of functions. They ignore constant factors and lower-order terms.

Big-O (O) – Upper Bound

Defines an asymptotic upper bound: the worst-case scenario (or an upper limit on growth).

$f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Big-Omega (Ω) – Lower Bound

Defines an asymptotic lower bound: the best-case scenario or a lower limit on growth.

$f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.

Big-Theta (Θ) – Tight Bound

Defines both upper and lower bounds, meaning the function grows at the same rate as $g(n)$.

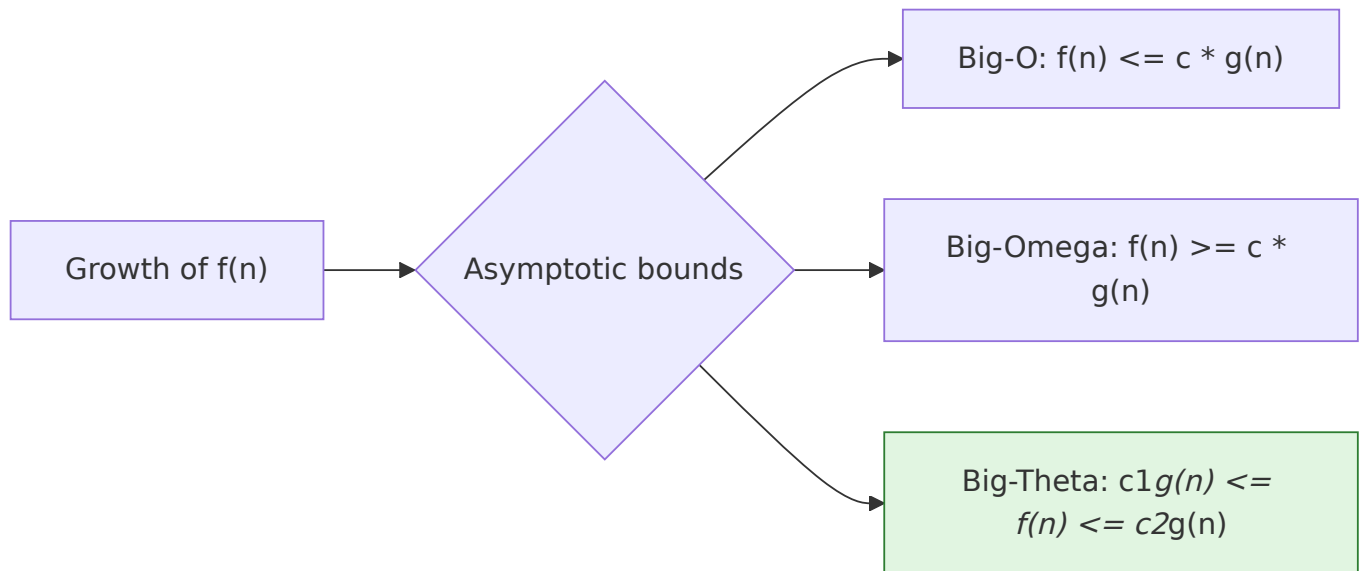
$f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2, n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

Relationships

- If $f(n) = \Theta(g(n))$, then $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
- O is used most often in practice to describe worst-case complexity.

Real-life analogy: Suppose you drive to work. In the worst case (traffic jam) it takes 60 minutes (Big-O). In the best case (no traffic) it takes 30 minutes (Big-Omega). On a typical day, it takes 40–50 minutes (Big-Theta). The notation captures these bounds.

The diagram below illustrates the relationship between these notations for a hypothetical time complexity function $f(n)$.



5. Common Complexity Classes

The table below summarises the most frequently encountered complexity classes, typical algorithms that exhibit them, and practical implications.

Class	Name	Example Algorithm	Scales to n
$O(1)$	Constant	Array access by index, push/pop on stack	Huge
$O(\log n)$	Logarithmic	Binary search, balanced BST search	Very large
$O(n)$	Linear	Linear search, array sum	Millions
$O(n \log n)$	Linearithmic	Merge sort, heap sort	Millions
$O(n^2)$	Quadratic	Bubble sort, naive matrix multiplication	Thousands
$O(2^n)$	Exponential	Recursive Fibonacci (naive), subset generation	20–30
$O(n!)$	Factorial	Naive travelling salesman (permutations)	10–12

Real-life analogies for complexity classes:

- $O(1)$ – Turning on a light switch: always one action, regardless of room size.
- $O(\log n)$ – Finding a name in a phone book by repeatedly splitting in half.
- $O(n)$ – Checking each item in a shopping list.
- $O(n \log n)$ – Sorting a deck of cards using a divide-and-conquer method like merge sort.
- $O(n^2)$ – Organising a meeting where every person shakes hands with every other person.

- $O(2^n)$ – Deciding which subset of items to take on a trip (each item either taken or not).
- $O(n!)$ – Arranging all books on a shelf in every possible order.

Detailed Descriptions with C++ Examples

$O(1)$ – Constant Time

The algorithm executes a fixed number of operations regardless of input size.

Example: Access an element in an array by index.

```
int getFirstElement(int arr[], int n) {  
    // Input n is ignored; single indexing operation  
    return arr[0];    //  $O(1)$   
}
```

$O(\log n)$ – Logarithmic Time

The input size is halved (or reduced by a constant factor) at each step. Very efficient.

Example: Binary search on a sorted array.

```
int binarySearch(int arr[], int left, int right, int target) {  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
        if (arr[mid] == target)  
            return mid;  
        else if (arr[mid] < target)  
            left = mid + 1;  
        else  
            right = mid - 1;  
    }  
    return -1;  
}
```

$O(n)$ – Linear Time

The algorithm performs a single pass over the input.

Example: Sum all elements of an array.

```
int arraySum(int arr[], int n) {  
    int total = 0;  
    for (int i = 0; i < n; ++i) {  
        total += arr[i];  
    }  
    return total;  
}
```

$O(n \log n)$ – **Linearithmic Time**

Often achieved by divide-and-conquer algorithms that split the problem into smaller parts and then combine them.

Example: Merge sort.

```

void merge(List<Integer> arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    List<Integer> L = new ArrayList<>(n1);
    List<Integer> R = new ArrayList<>(n2);
    for (int i = 0; i < n1; ++i) L.add(arr.get(left + i));
    for (int j = 0; j < n2; ++j) R.add(arr.get(mid + 1 + j));

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L.get(i) <= R.get(j)) {
            Collections.swap(arr, k, L.get(i));
            i++;
        } else {
            Collections.swap(arr, k, R.get(j));
            j++;
        }
        k++;
    }
    while (i < n1) {
        Collections.swap(arr, k, L.get(i));
        i++;
        k++;
    }
    while (j < n2) {
        Collections.swap(arr, k, R.get(j));
        j++;
        k++;
    }
}

```

```

void mergeSort(List<Integer> arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
}

```

```
merge(arr, left, mid, right);  
}
```

$O(n^2)$ – Quadratic Time

Double nested iterations over the input. Becomes slow quickly.

Example: Bubble sort.

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; ++i) {  
        for (int j = 0; j < n - i - 1; ++j) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

$O(2^n)$ – Exponential Time

Often arises in recursive algorithms that solve overlapping sub-problems without memoization.

Example: Naive recursive Fibonacci.

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

$O(n!)$ – Factorial Time

Algorithms that generate all permutations of a set. Only feasible for very small n .

Example: Naive generation of all permutations (printing each).


```

void printPermutations(List<Integer> arr) {
    Collections.sort(arr);
    do {
        // Process current permutation (e.g., print)
        for (int x : arr) System.out.print(x + " ");
        System.out.println();
    } while (nextPermutation(arr));
}

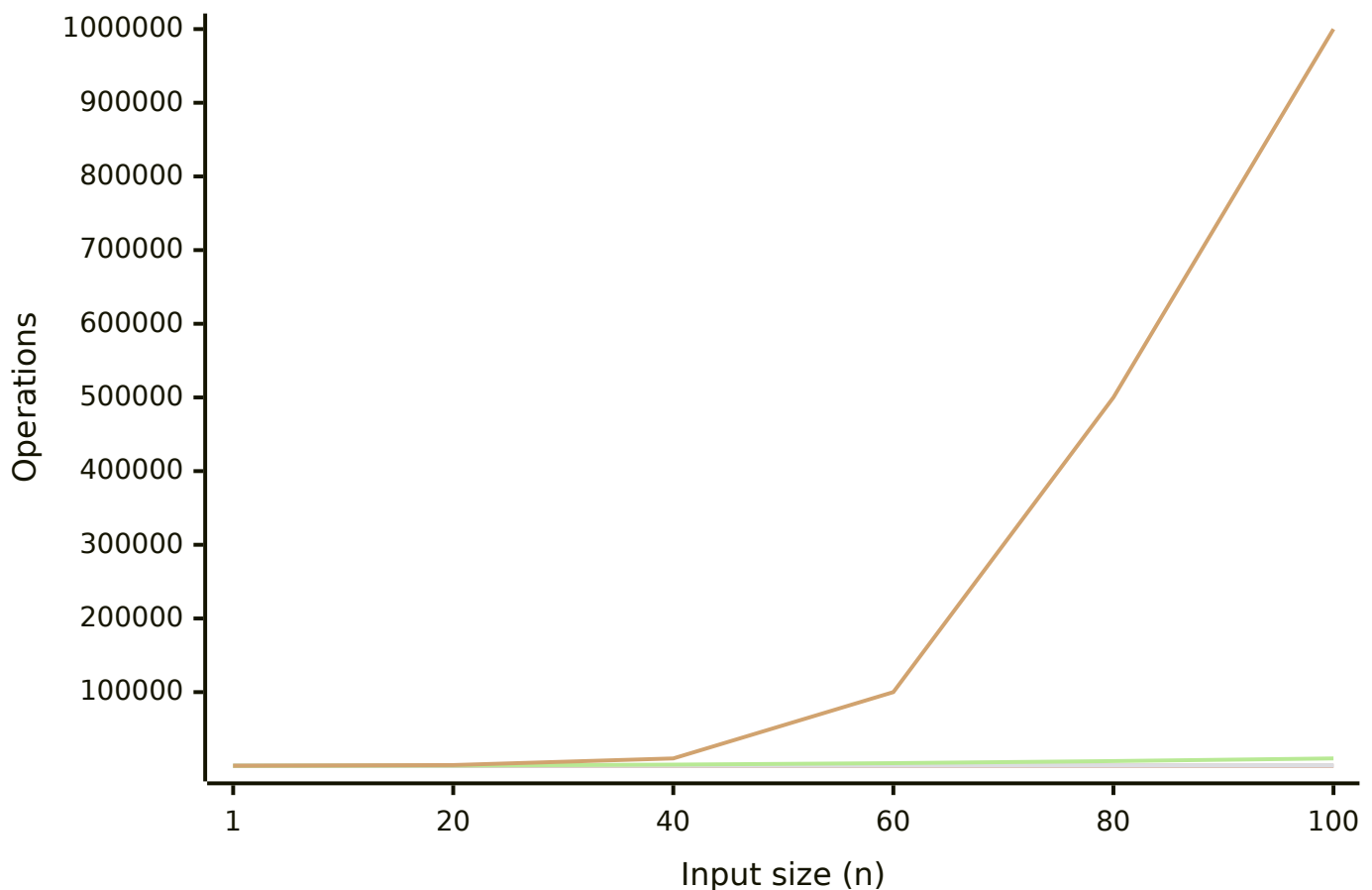
// Helper method to generate next permutation (similar to
std::next_permutation)
private static boolean nextPermutation(List<Integer> arr) {
    int i = arr.size() - 2;
    while (i >= 0 && arr.get(i) >= arr.get(i + 1)) {
        i--;
    }
    if (i < 0) {
        return false;
    }
    int j = arr.size() - 1;
    while (arr.get(j) <= arr.get(i)) {
        j--;
    }
    Collections.swap(arr, i, j);
    // Reverse the suffix
    int left = i + 1;
    int right = arr.size() - 1;
    while (left < right) {
        Collections.swap(arr, left, right);
        left++;
        right--;
    }
    return true;
}

```

Visual Comparison of Growth Rates

The following Mermaid diagram plots the number of operations against input size for major complexity classes. Note the logarithmic scale on the y-axis to accommodate large ranges.

Asymptotic Growth Rates (n up to 100, operations capped for $O(2^n)$)



Note: For readability, the diagram uses a logarithmic conversion on the vertical axis. Actual $O(2^n)$ at $n = 100$ is astronomically large.

6. Practical Guidelines for Complexity Analysis

1. **Always consider worst-case** unless the problem explicitly asks for average or best-case analysis.
2. **Drop constants and lower-order terms:** $3n^2 + 5n + 2$ simplifies to $O(n^2)$.
3. **Analyse loops carefully:** A loop that runs n times contributes a factor of n ; nested loops multiply.
4. **Recursive algorithms** often follow recurrence relations (e.g., $T(n) = 2T(n/2) + n$ yields $O(n \log n)$).
5. **Space complexity** includes call stack memory for recursion (e.g., naive recursion for Fibonacci uses $O(n)$ stack space).

7. Summary

- Data structures organise data; algorithms manipulate it. Together they form the foundation of efficient software.
- DSA knowledge is critical for building scalable systems and passing technical interviews.
- Complexity analysis uses asymptotic notations (Big-O, Big-Omega, Big-Theta) to describe resource usage.
- Recognising common complexity classes ($O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$) helps choose the right algorithm for the input size constraints.

The next chapter will cover fundamental data structures: arrays, linked lists, stacks, and queues, along with their time complexity trade-offs.

Chapter 2: Mathematics & Prerequisites

This chapter covers essential foundations for data structures and algorithms: recursion, backtracking, discrete mathematics, and bit manipulation. Each section explains **what** and **when to use** to help you apply these concepts in problem-solving.

1. Recursion Basics

What: A function that calls itself to solve a smaller instance of the same problem.

When to use:

- Problems that can be broken into identical sub-problems (divide & conquer)
- Tree and graph traversals (DFS, binary tree operations)
- Backtracking (exploring decision trees)
- Mathematical sequences defined recursively (factorial, Fibonacci)
- Parsing recursive structures (JSON, XML, nested lists)

1.1 Components (to the point)

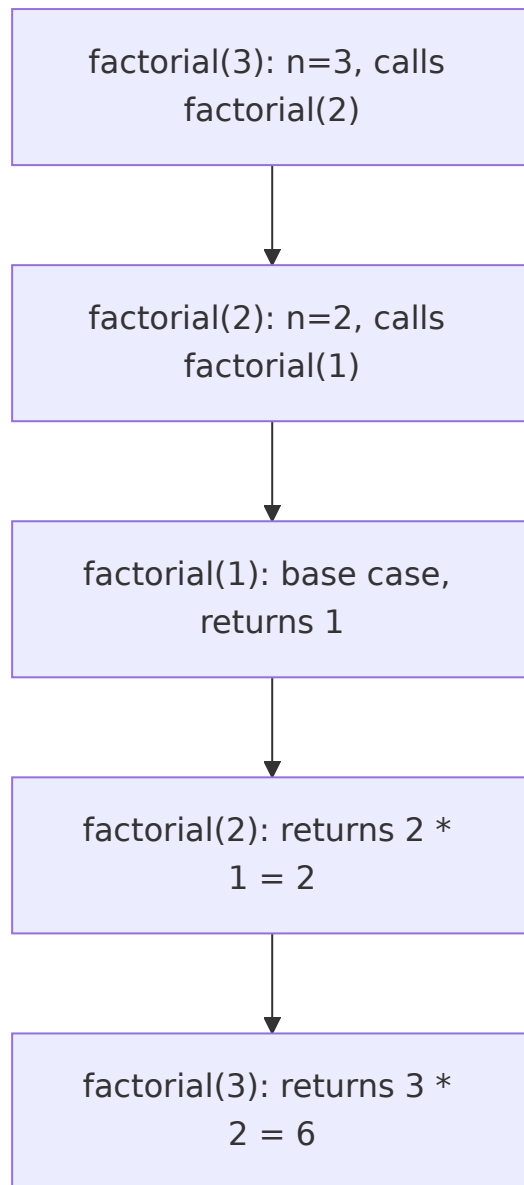
- **Base case:** Terminates recursion; the smallest problem instance with a direct answer.
- **Recursive case:** Calls itself with reduced/modified input moving toward the base case.
- **Call stack:** Automatically managed stack storing local variables, parameters, and return addresses for each active call.

Real-life analogy: Russian nesting dolls – open outer to reach inner (recursive descent), then close on way back (returns).

1.2 Example: Factorial

```
int factorial(int n) {  
    if (n <= 1) return 1;           // base case  
    return n * factorial(n - 1);    // recursive case  
}
```

1.3 Call Stack Visualisation (factorial(3))



1.4 When to Avoid Recursion

- **Deep recursion** (e.g., $n > 10^5$) risks stack overflow → use iterative version.
- **Expensive recomputation** (naive Fibonacci) → use memoisation or iteration.
- **Performance-critical real-time systems** → iteration avoids function call overhead.

2. Backtracking Principles

What: Systematic trial-and-error search that incrementally builds candidates and aborts (backtracks) those that violate constraints.

When to use:

- Combinatorial search problems: subsets, permutations, combinations
- Constraint satisfaction: N-Queens, Sudoku, crossword puzzles

- Path finding in mazes or graphs (with pruning)
- Generating all valid configurations (e.g., parentheses, letter combinations)

2.1 Backtracking Template (to the point)

```
void backtrack(state, choices):
    if is_goal(state): record(state); return
    for each choice in choices:
        if is_valid(state + choice):
            make_choice(choice)
            backtrack(state + choice, new_choices)
            undo_choice(choice)    // backtrack
```

Real-life analogy: Solving a maze – try a path; if dead end, return to last junction and try next direction.

2.2 Example: Generate All Subsets

```
void generateSubsets(List<Integer> nums, int index, List<Integer>
current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current));           // include
current subset
    for (int i = index; i < nums.size(); ++i) {
        current.add(nums.get(i));                  // choose
        generateSubsets(nums, i + 1, current, result);
        current.remove(current.size() - 1);         //
backtrack
    }
}
```

Time: $O(2^n)$, **Space:** $O(n)$ (stack depth).

2.3 When Backtracking is Inefficient

- Large search space without effective pruning (e.g., brute-force all permutations of 50 items)
- Problems with overlapping subproblems – dynamic programming is better (e.g., shortest path in DAG)

3. Mathematical Foundations

3.1 Logarithms and Exponents

Definition: $\log_b a = c \iff b^c = a$. In CS, base 2 ($\log n$).

When to use:

- **Complexity analysis:** $O(\log n)$ appears in binary search, balanced BSTs, priority queues (heaps).
- **Divide & conquer:** Algorithms that halve input (merge sort, quick sort average case).
- **Repeated squaring:** Fast modular exponentiation.

Key property: $\log_b(xy) = \log_b x + \log_b y$.

Real-life analogy: Folding a paper in half repeatedly – number of folds to reach single thickness is $\log_2(\text{length})$.

3.2 Summations and Series

Definition: Compact representation of loop/recursive costs.

Series	Formula	Typical Use
Arithmetic	$\sum_{i=1}^n i = n(n+1)/2$	Nested loops, analysing $O(n^2)$
Geometric	$\sum_{i=0}^n 2^i = 2^{n+1} - 1$	Exponential algorithms, binary recursion
Harmonic	$\sum_{i=1}^n 1/i \approx \ln n$	Average-case of quicksort, load balancing

When to use: To derive exact complexity from loop structures or recurrence relations.

3.3 Permutations and Combinations

Definition:

- Permutation (ordered): $P(n, k) = n!/(n-k)!$
- Combination (unordered): $\binom{n}{k} = n!/(k!(n-k)!)$

When to use:

- Brute-force upper bounds: TSP $O(n!)$, subset generation $O(2^n)$.
- Probability calculations in randomised algorithms (e.g., random permutations).
- Counting problems in dynamic programming state spaces.

3.4 Modular Arithmetic

Definition: Arithmetic where numbers wrap around after reaching a modulus m .

When to use:

- **Hashing:** Hash tables use modulo to map keys to bucket indices.
- **Cryptography:** RSA, Diffie-Hellman (modular exponentiation).
- **Preventing overflow:** Multiply large numbers by taking mod at each step.
- **Competitive programming:** Many results required modulo $10^9 + 7$.

Fast modular exponentiation (binary exponentiation):

```
long long modPow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
    while (exp) {
        if (exp & 1) result = (result * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return result;
}
```

3.5 Prime Numbers and GCD

Definition:

- **Prime:** integer > 1 with no divisors other than 1 and itself.
- **GCD:** greatest common divisor of two integers.

When to use:

- **Prime numbers:** Hash table sizes (reduce collisions), RSA key generation, primality tests in contest problems.
- **GCD:** Simplifying fractions, computing LCM ($\text{lcm}(a, b) = a \cdot b / \text{gcd}(a, b)$), cryptography, periodicity calculations.

Euclidean algorithm for GCD – $O(\log \min(a, b))$:


```
int gcd(int a, int b) {  
    while (b) { int t = b; b = a % b; a = t; }  
    return a;  
}
```

4. Bit Manipulation

What: Direct operations on binary representation using bitwise operators.

When to use:

- **Performance:** Extremely fast (single CPU instruction).
- **State compression:** Store multiple boolean flags in one integer (e.g., visited sets in DP).
- **Subset enumeration:** Iterate over all subsets of a set using bitmask $0 \dots 2^n - 1$.
- **Low-level programming:** Device drivers, embedded systems, network protocols.
- **Competitive programming:** Solve problems with small constraints ($n \leq 20$ for exponential bitmask DP).

4.1 Bitwise Operators (quick reference)

Operator	Symbol	Effect
AND	&	Set bit if both are 1
OR		Set bit if either is 1
XOR	^	Set bit if bits differ
NOT	~	Flip all bits
Left shift	<<	Shift left, zero fill
Right shift	>>	Shift right (logical for unsigned)

4.2 Common Tricks (when to apply)

Trick	Code	Use Case
Power of two	<code>n && !(n & (n-1))</code>	Check alignment, hash table sizing

Set k-th bit	<code>n = (1 << k)</code>	Mark present in bitmask
Clear k-th bit	<code>n &= ~(1 << k)</code>	Remove flag
Toggle k-th bit	<code>n ^= (1 << k)</code>	Flip flag
Check k-th bit	<code>(n >> k) & 1</code>	Test membership
Count set bits (Kernighan)	<code>while(n) { n &= n-1; count++; }</code>	Hamming weight, parity
Isolate lowest set bit	<code>n & -n</code>	Tree-like structures (Fenwick tree)
Swap without temp	<code>a ^= b; b ^= a; a ^= b;</code>	Limited use (readability suffers)

Example: Subset enumeration using bitmask

```
int n = 3; // elements 0,1,2
for (int mask = 0; mask < (1 << n); ++mask) {
    // mask's bits indicate which elements are in subset
    for (int i = 0; i < n; ++i) {
        if (mask & (1 << i))
            System.out.print(i + " ");
    }
    System.out.println();
}
```

5. Summary

Topic	What (to the point)	When to Use
Recursion	Function calls itself	Divide & conquer, tree/graph traversal, backtracking
Backtracking	Trial & error with pruning	Combinatorial search, constraint satisfaction

Logarithms	Inverse of exponentiation	Complexity analysis ($O(\log n)$), divide & conquer
Summations	Compact loop cost	Analysing nested loops, recurrence relations
Permutations/combinations	Ordered/unordered selections	Brute-force bounds, probability, counting
Modular arithmetic	Wrap-around numbers	Hashing, crypto, overflow prevention
Primes & GCD	Divisibility basics	Hash sizes, crypto, fraction reduction
Bit manipulation	Direct bitwise ops	Flags, state compression, subset enumeration, performance

The next chapter will analyse recursive algorithms using recurrence relations and the Master Theorem, and introduce advanced mathematical tools for DSA.

Chapter 3: Arrays and Strings

This chapter covers fundamental linear data structures: arrays and strings. It introduces essential techniques such as sliding window, two pointers, prefix sums, difference arrays, and pattern matching algorithms, followed by classic problems that demonstrate these concepts.

1. Arrays

An array is a contiguous block of memory storing elements of the same type, accessible by index in $O(1)$ time.

1.1 One-Dimensional and Multi-Dimensional Arrays

What: A linear collection of elements (1D) or a grid of elements (2D, 3D, etc.).

When to use:

- **1D arrays:** Storing sequential data (scores, temperatures, etc.)
- **2D arrays:** Matrices, game boards, pixel grids, DP tables.

Static vs Dynamic:

- **Static array:** Fixed size known at compile time (C++ `int arr[100];`).
- **Dynamic array:** Grows automatically (`std::vector<T>`).

Java example for 2D array using ArrayList:

```
import java.util.ArrayList;
ArrayList<ArrayList<Integer>> matrix = new ArrayList<>(rows);
for (int i = 0; i < rows; i++) {
    matrix.add(new ArrayList<>(cols));
}
```

1.2 Dynamic Arrays (std::vector)

What: A resizable array container that manages memory automatically.

When to use: When the number of elements is unknown at compile time or changes frequently.

Key properties:

- Amortized $O(1)$ for `push_back` (occasional reallocation)
- $O(1)$ random access
- $O(n)$ insertion/deletion at arbitrary positions

```
ArrayList<Integer> v = new ArrayList<>();  
v.add(10); // O(1) amortized  
v.remove(v.size() - 1); // O(1)
```

1.3 Common Operations

Operation	C++ Code	Time Complexity
Traversal	<code>for (int x : arr) {}</code>	$O(n)$
Access by index	<code>arr[i]</code>	$O(1)$
Insert at end	<code>arr.push_back(x)</code>	Amortized $O(1)$
Insert at position	<code>arr.insert(arr.begin()+i, val)</code>	$O(n)$
Delete at position	<code>arr.erase(arr.begin()+i)</code>	$O(n)$
Linear search	<code>find(arr.begin(), arr.end(), val)</code>	$O(n)$
Binary search (sorted)	<code>binary_search(arr.begin(), arr.end(), val)</code>	$O(\log n)$

1.4 Prefix Sum Array

What: An auxiliary array where `prefix[i] = sum of arr[0..i-1]` (or up to i).

When to use: Answering multiple range sum queries (`sum(l..r)`) efficiently.

Construction:

```
int[] prefix = new int[n+1];
for (int i = 0; i < n; i++)
    prefix[i+1] = prefix[i] + arr[i];
// Sum from l to r inclusive: prefix[r+1] - prefix[l]
```

Real-life analogy: A running total on a cash register. When you need the sum of sales between two hours, you subtract the total before the first hour from total after the second hour.

1.5 Difference Array

What: An array `diff` where `diff[i] = arr[i] - arr[i-1]` (with `diff[0] = arr[0]`). It allows $O(1)$ range updates.

When to use: Many range increment/decrement operations followed by a single read of the final array.

Operation: To add `k` to `arr[l..r]`, do `diff[l] += k; diff[r+1] -= k;` (if `r+1 < n`). After all updates, reconstruct `arr` via prefix sum over `diff`.

```
int[] diff = new int[n];
// range update: add k from l to r
diff[l] += k;
if (r+1 < n) diff[r+1] -= k;

// final array
int[] result = new int[n];
result[0] = diff[0];
for (int i = 1; i < n; i++)
    result[i] = result[i-1] + diff[i];
```

2. Sliding Window Technique

What: Maintaining a subset (window) of array elements that expands or contracts while processing the array linearly. Each element enters and leaves the window at most once.

When to use: Subarray or substring problems where you need to compute something for all contiguous subarrays of fixed or variable length.

Why a while loop is natural: For variable-size windows, the left pointer may shrink multiple steps while a condition holds. A `while` loop makes this shrinking explicit and easier to read.

2.1 Fixed-Size Window

Problem: Find the maximum sum of any subarray of size `k`.

```
int maxSumFixed(List<Integer> arr, int k) {
    int sum = 0;
    for (int i = 0; i < k; ++i) sum += arr.get(i);
    int maxSum = sum;
    for (int i = k; i < arr.size(); ++i) {
        sum += arr.get(i) - arr.get(i - k);
        maxSum = Math.max(maxSum, sum);
    }
    return maxSum;
}
```

Time: $O(n)$. **Space:** $O(1)$.

2.2 Variable-Size Window (with while loop)

Problem: Longest substring without repeating characters.

Idea: Expand right pointer and use a hash set or array to track seen characters. When a duplicate is found, shrink from the left until the duplicate is removed.

```

int longestUniqueSubstr(string s) {
    int[] lastIndex = new int[256];
    Arrays.fill(lastIndex, -1);    // last occurrence of each char
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.size(); ++right) {
        // If current char already inside window, move left past its
        // last occurrence
        if (lastIndex[s[right]] >= left) {
            left = lastIndex[s[right]] + 1;
        }
        lastIndex[s[right]] = right;
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}

```

Alternative while-loop version (explicit shrinking):

```

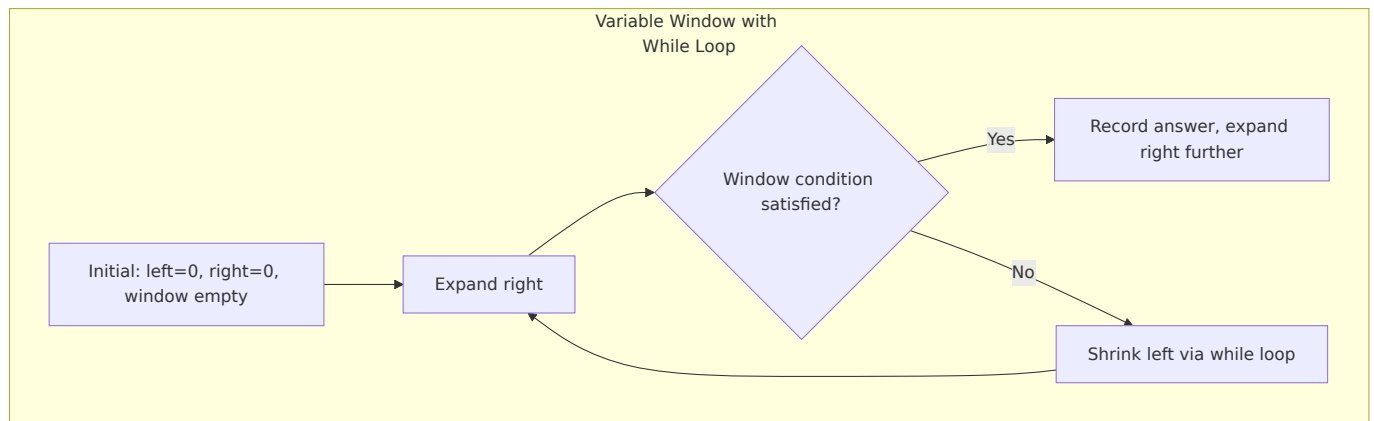
int longestUniqueSubstr(string s) {
    int[] freq = new int[256];
    Arrays.fill(freq, 0);    // frequency inside window
    int left = 0, right = 0, maxLen = 0;
    while (right < s.size()) {
        char c = s[right];
        freq[c]++;
        // shrink window until c is unique
        while (freq[c] > 1) {
            freq[s[left]]--;
            left++;
        }
        maxLen = max(maxLen, right - left + 1);
        right++;
    }
    return maxLen;
}

```

Time: $O(n)$ because each character enters and leaves the window at most once.

Real-life analogy: A conveyor belt inspection window. You slide a fixed-size frame (fixed window) or adjust the frame size to keep all items inside without duplicates (variable window).

Visualisation of sliding window:



3. Two Pointers Technique

What: Using two indices (pointers) that traverse the array, either moving in opposite directions or in the same direction at different speeds.

When to use:

- Opposite direction: Sorted two-sum, palindrome checking, trapping rainwater.
- Same direction: Merging sorted arrays, removing duplicates, partition.

3.1 Opposite Direction

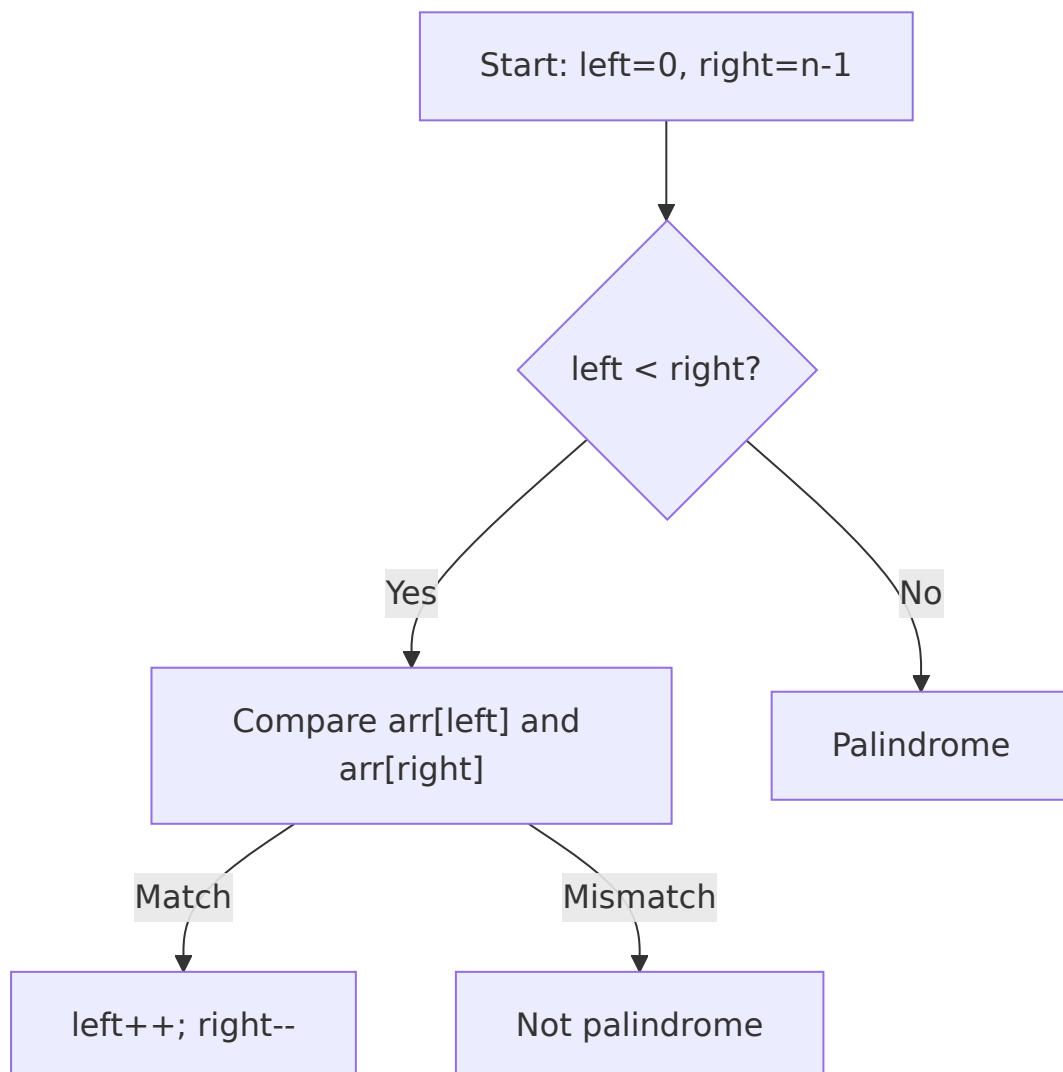
Example: Check if a string is a palindrome.

```
bool isPalindrome(string s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        if (s[left] != s[right]) return false;
        left++; right--;
    }
    return true;
}
```

3.2 Same Direction (Fast & Slow)

Example: Remove duplicates from sorted array in-place.

```
int removeDuplicates(vector<int>& nums) {  
    if (nums.empty()) return 0;  
    int slow = 0;  
    for (int fast = 1; fast < nums.size(); ++fast) {  
        if (nums[fast] != nums[slow])  
            nums[++slow] = nums[fast];  
    }  
    return slow + 1;  
}
```



4. Strings

A string is a sequence of characters. In C++, `std::string` is mutable, but operations like concatenation may allocate new memory.

4.1 String Immutability and Builders

What: In some languages strings are immutable, but C++ strings are mutable. However, repeated concatenation using `+` or `+=` can still cause reallocations.

When to use: Use `std::string` for most operations; if many concatenations are needed, use `reserve()` to pre-allocate or `std::stringstream`.

```
string buildString(int n) {
    string result;
    result.reserve(n * 2);    // avoid reallocations
    for (int i = 0; i < n; ++i)
        result += "ab";
    return result;
}
```

4.2 Pattern Matching Algorithms

Naive Search

What: Slide the pattern over the text and compare character by character.

Time: $O(n * m)$ in worst case.

```
int naiveSearch(string text, string pattern) {
    int n = text.size(), m = pattern.size();
    for (int i = 0; i <= n - m; ++i) {
        int j;
        for (j = 0; j < m; ++j)
            if (text[i+j] != pattern[j]) break;
        if (j == m) return i;
    }
    return -1;
}
```

KMP (Knuth-Morris-Pratt)

What: Preprocesses the pattern to build a longest proper prefix-suffix (LPS) array. Uses it to avoid re-scanning matched characters.

Time: $O(n + m)$. **Space:** $O(m)$.

When to use: Searching in large texts, especially when the pattern has repetitive structure.

LPS construction:

```
vector<int> buildLPS(string pat) {
    int m = pat.size();
    vector<int> lps(m, 0);
    int len = 0, i = 1;
    while (i < m) {
        if (pat[i] == pat[len]) lps[i++] = ++len;
        else if (len) len = lps[len-1];
        else lps[i++] = 0;
    }
    return lps;
}
```

Real-life analogy: When you look for a word in a book and you already matched a prefix, you don't start from the beginning of the word after a mismatch – you use the knowledge of the matched part.

Rabin-Karp (Rolling Hash)

What: Uses a hash function (e.g., polynomial rolling hash) to compare pattern and text substrings in $O(1)$ on average. When hash matches, verify character-by-character.

Time: Average $O(n + m)$, worst $O(n*m)$ (hash collisions). **Space:** $O(1)$.

When to use: Multiple pattern matching, plagiarism detection, avoiding high constant factor of KMP.

```

#define MOD 1000000007
#define BASE 256

int rabinKarp(string text, string pattern) {
    int n = text.size(), m = pattern.size();
    if (m > n) return -1;
    long long pHash = 0, tHash = 0, pow = 1;
    for (int i = 0; i < m; ++i) {
        pHash = (pHash * BASE + pattern[i]) % MOD;
        tHash = (tHash * BASE + text[i]) % MOD;
        pow = (pow * BASE) % MOD;
    }
    for (int i = 0; i <= n - m; ++i) {
        if (pHash == tHash) {
            bool match = true;
            for (int j = 0; j < m; ++j)
                if (text[i+j] != pattern[j]) { match = false; break; }
            if (match) return i;
        }
        if (i < n - m) {
            tHash = (tHash * BASE - text[i] * pow + text[i+m]) % MOD;
            if (tHash < 0) tHash += MOD;
        }
    }
    return -1;
}

```

4.3 Anagram and Palindrome Problems

Anagram: Two strings with same character frequencies.

Approach 1: Sort both strings – $O(n \log n)$.

Approach 2: Frequency array of size 256 – $O(n)$.

```
bool isAnagram(string s1, string s2) {
    if (s1.length() != s2.length()) return false;
    vector<int> count(256, 0);
    for (char c : s1) count[c]++;
    for (char c : s2) if (--count[c] < 0) return false;
    return true;
}
```

Palindrome: A string that reads the same forwards and backwards.

Check using two pointers (see section 3.1). For finding longest palindromic substring, use expand around center ($O(n^2)$).

5. Important Problems

5.1 Kadane's Algorithm (Maximum Subarray Sum)

What: Dynamic programming that keeps the maximum sum ending at each position.

When to use: Finding contiguous subarray with largest sum (can include negative numbers).

```
int maxSubarraySum(vector<int>& arr) {
    int maxSoFar = arr[0], maxEndingHere = arr[0];
    for (int i = 1; i < arr.size(); ++i) {
        maxEndingHere = max(arr[i], maxEndingHere + arr[i]);
        maxSoFar = max(maxSoFar, maxEndingHere);
    }
    return maxSoFar;
}
```

Time: $O(n)$. **Space:** $O(1)$.

Real-life analogy: Finding the most profitable period to hold a stock, where you can buy and sell once.

5.2 Dutch National Flag Problem (Sort 0,1,2)

What: Sort an array containing only 0, 1, and 2 in one pass (three-way partition).

When to use: When we need to partition three distinct values without extra memory.

```

void sortColors(vector<int>& nums) {
    int low = 0, mid = 0, high = nums.size() - 1;
    while (mid <= high) {
        if (nums[mid] == 0) swap(nums[low++], nums[mid++]);
        else if (nums[mid] == 1) mid++;
        else swap(nums[mid], nums[high--]);
    }
}

```

Time: $O(n)$. **Space:** $O(1)$.

5.3 Trapping Rainwater

What: Given elevation map (bar heights), compute how much water can be trapped after rain.

Approach 1 (prefix max): `leftMax[i] = max left height up to i`, `rightMax[i]` similarly, water at `i = min(leftMax[i], rightMax[i]) - height[i]`. **Time & Space:** $O(n)$.

Approach 2 (two pointers): $O(1)$ space.

```

int trap(vector<int>& height) {
    int left = 0, right = height.size() - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            leftMax = max(leftMax, height[left]);
            water += leftMax - height[left];
            left++;
        } else {
            rightMax = max(rightMax, height[right]);
            water += rightMax - height[right];
            right--;
        }
    }
    return water;
}

```

Real-life analogy: Water collects between buildings of varying heights; the amount at a point is limited by the shorter of the tallest building to its left and right.

5.4 Container with Most Water

What: Given vertical lines at positions i with height `height[i]`, find two lines that together with the x-axis form a container holding the most water.

When to use: Two-pointer greedy approach.

```
int maxArea(vector<int>& height) {
    int left = 0, right = height.size() - 1, maxWater = 0;
    while (left < right) {
        int w = right - left;
        int h = min(height[left], height[right]);
        maxWater = max(maxWater, w * h);
        if (height[left] < height[right]) left++;
        else right--;
    }
    return maxWater;
}
```

Time: $O(n)$. **Space:** $O(1)$.

Summary Table

Concept	What (brief)	When to use	Time Complexity
Prefix sum	Pre-computed cumulative sums	Range sum queries	$O(n)$ pre, $O(1)$ per query
Difference array	Range updates via diffs	Batch range increments	$O(1)$ per update, $O(n)$ finalize
Sliding window (while loop)	Maintain subarray range	Subarray/substring optimal	$O(n)$
Two pointers	Two indices moving	Sorted arrays, palindrome	$O(n)$

KMP	Pattern matching with LPS	Large text, repetitive pattern	$O(n+m)$
Rabin-Karp	Hashing for pattern match	Multiple patterns, rolling hash	$O(n+m)$ avg
Kadane	Maximum subarray sum	Single subarray max sum	$O(n)$
Dutch National Flag	Three-way partition	Sort 0,1,2 in-place	$O(n)$
Trapping rainwater	Water volume between bars	Heightmap water trapping	$O(n)$
Container with most water	Max area (2 lines)	Maximum area problem	$O(n)$

The next chapter will cover linked lists (singly, doubly, circular) and their operations, including cycle detection and merging.

Chapter 4: Linked Lists

This chapter covers linked list data structures: singly, doubly, and circular linked lists. It details basic operations, two-pointer techniques, reversal algorithms, merging, intersection problems, and advanced copying.

1. Singly Linked List

What: A linear data structure where each node contains data and a pointer (`next`) to the next node. The last node points to `nullptr`.

When to use:

- Dynamic memory allocation (no fixed size)
- Frequent insertions/deletions at arbitrary positions ($O(1)$ if you have a reference to the node)
- When contiguous memory is not required or wasteful

Node structure (Java):

```
class Node {
    int data;
    Node next;

    Node(int val) {
        this.data = val;
        this.next = null;
    }
}
```

2. Doubly Linked List

What: Each node has pointers to both the next and previous nodes.

When to use:

- Need efficient backward traversal
- Deletion of a given node (no need to find previous)

- Implementing LRU cache, undo/redo functionality

Node structure:

```
class DNode {
    int data;
    DNode prev;
    DNode next;

    DNode(int val) {
        this.data = val;
        this.prev = null;
        this.next = null;
    }
}
```

3. Circular Linked List

What: The last node points back to the first node (singly or doubly circular). No node points to `nullptr`.

When to use:

- Round-robin scheduling
- Games where players take turns cyclically
- Implementing a circular buffer

4. Basic Operations

Operation	Singly	Doubly	Time Complexity
Traversal	$O(n)$	$O(n)$	$O(n)$
Search by value	$O(n)$	$O(n)$	$O(n)$
Insert at head	$O(1)$	$O(1)$	$O(1)$
Insert at tail (without tail ptr)	$O(n)$	$O(1)$ if tail known	$O(1) / O(n)$
Insert after given node	$O(1)$	$O(1)$	$O(1)$

Delete head	O(1)	O(1)	O(1)
Delete given node (without prev)	O(n)	O(1)	O(n)/O(1)

Example: Insert at head of singly linked list:

```
void insertAtHead(Node head, int val) {
    Node newNode = new Node(val);
    newNode.next = head;
    head = newNode;
}
```

5. Two-Pointer Techniques

What: Using two pointers moving at different speeds (fast/slow) or different start positions to solve problems in $O(n)$ time and $O(1)$ space.

When to use: Cycle detection, finding middle, finding nth node from end, detecting intersection.

5.1 Finding Middle of List

Slow pointer moves one step, fast pointer moves two steps. When fast reaches end, slow is at the middle.

```
Node findMiddle(Node head) {
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

5.2 Cycle Detection (Floyd's Cycle Detection)

What: Detect if a linked list has a cycle using slow (1 step) and fast (2 steps). If they meet, a cycle exists.

When to use: Verify infinite loops, corrupted linked structures, detecting loops in reference chains.

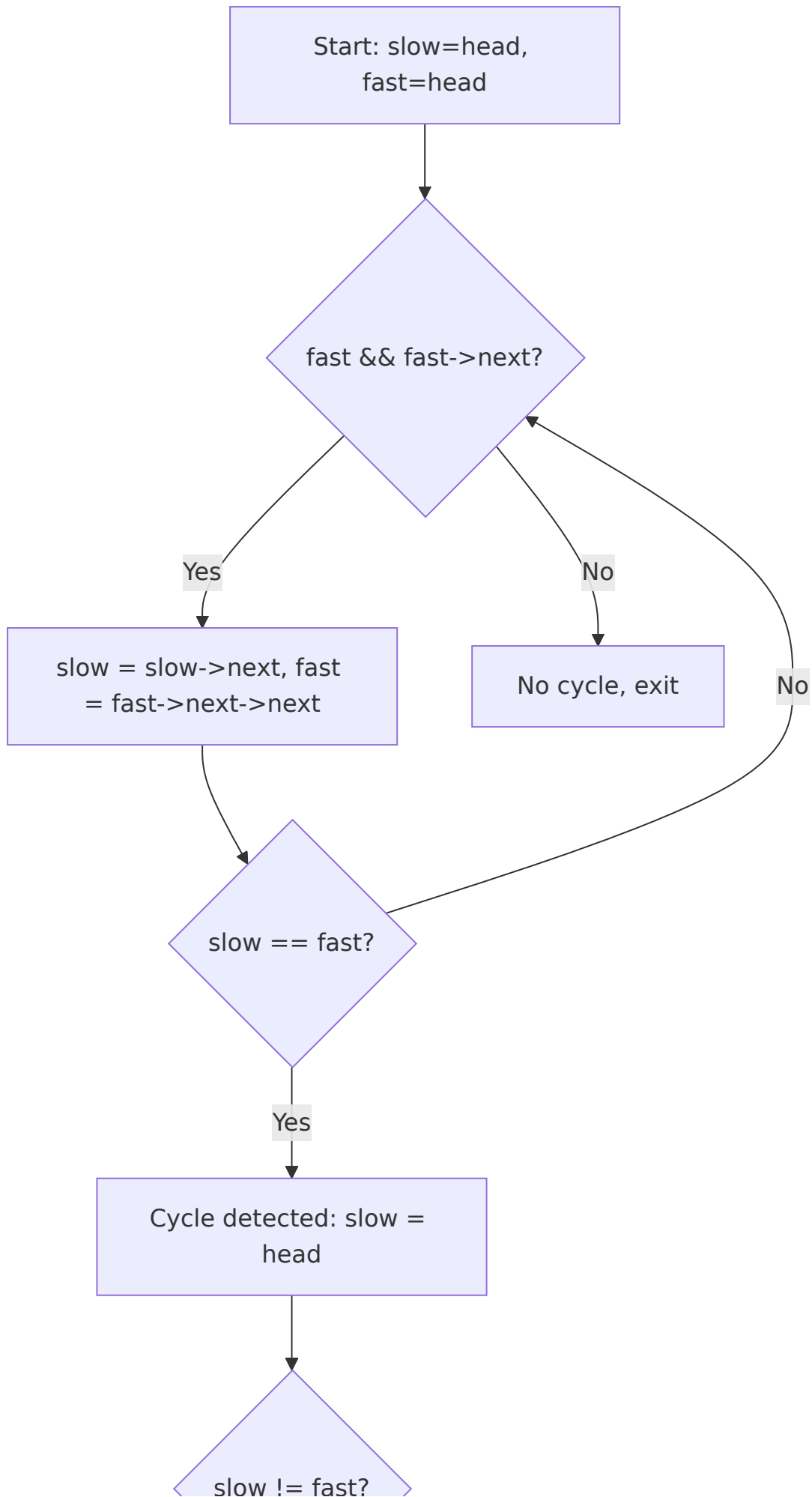
```
boolean hasCycle(Node head) {
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

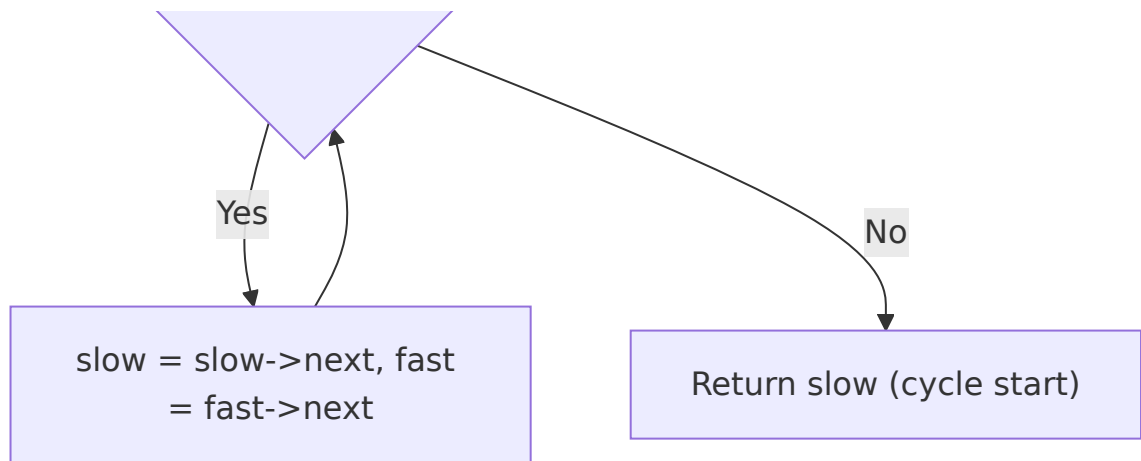
5.3 Finding Start of Cycle

What: After slow and fast meet, reset one pointer to head, move both one step at a time. Their meeting point is the cycle start.

```
Node detectCycleStart(Node head) {
    Node slow = head;
    Node fast = head;
    boolean hasCycle = false;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) { hasCycle = true; break; }
    }
    if (!hasCycle) return null;
    slow = head;
    while (slow != fast) {
        slow = slow.next;
        fast = fast.next;
    }
    return slow;
}
```

Real-life analogy: Two runners on a circular track, one twice as fast. They meet; then starting one from the start point and moving both at same speed brings them together at the cycle entry.





6. Reversal

6.1 Iterative Reversal

What: Reverse the links using three pointers (prev, curr, next).

```
Node reverseIterative(Node head) {  
    Node prev = null;  
    Node curr = head;  
    Node next = null;  
    while (curr != null) {  
        next = curr.next;  
        curr.next = prev;  
        prev = curr;  
        curr = next;  
    }  
    return prev;  
}
```

6.2 Recursive Reversal

What: Reverse the rest of the list, then adjust the current node's next.


```

Node reverseRecursive(Node head) {
    if (head == null || head.next == null) return head;
    Node newHead = reverseRecursive(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}

```

6.3 Reverse in Groups of k

What: Reverse every contiguous block of k nodes. If the last group has fewer than k nodes, leave it as is.

```

Node* reverseKGroup(Node* head, int k) {
    Node* curr = head;
    int count = 0;
    while (curr && count < k) { curr = curr->next; count++; }
    if (count == k) {
        Node* reversedHead = reverseKGroup(curr, k);
        // reverse current group of k
        Node* prev = nullptr;
        Node* curr2 = head;
        for (int i = 0; i < k; ++i) {
            Node* next = curr2->next;
            curr2->next = prev;
            prev = curr2;
            curr2 = next;
        }
        head->next = reversedHead;
        return prev;
    }
    return head;
}

```

7. Merge and Intersection Problems

7.1 Merge Two Sorted Lists

What: Combine two sorted linked lists into one sorted list.

```
Node* mergeTwoSorted(Node* l1, Node* l2) {
    Node dummy(0);
    Node* tail = &dummy;
    while (l1 && l2) {
        if (l1->data < l2->data) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
```

7.2 Find Intersection Point of Two Lists

What: Two singly linked lists merge at a common node. Find that node.

Approach:

1. Compute lengths of both lists.
2. Advance the longer list's pointer by the difference.
3. Move both pointers until they meet.

```

Node* getIntersectionNode(Node* headA, Node* headB) {
    int lenA = 0, lenB = 0;
    Node* temp = headA;
    while (temp) { lenA++; temp = temp->next; }
    temp = headB;
    while (temp) { lenB++; temp = temp->next; }
    Node* ptrA = headA;
    Node* ptrB = headB;
    if (lenA > lenB) {
        for (int i = 0; i < lenA - lenB; ++i) ptrA = ptrA->next;
    } else {
        for (int i = 0; i < lenB - lenA; ++i) ptrB = ptrB->next;
    }
    while (ptrA != ptrB) {
        ptrA = ptrA->next;
        ptrB = ptrB->next;
    }
    return ptrA;
}

```

8. Copy List with Random Pointer

Problem: Each node has an extra `random` pointer that can point to any node in the list or `nullptr`. Create a deep copy.

Approach: Interleave copy nodes, assign random pointers, then separate.

```

struct RNode {
    int val;
    RNode* next;
    RNode* random;
    RNode(int v) : val(v), next(nullptr), random(nullptr) {}
};

RNode* copyRandomList(RNode* head) {
    if (!head) return nullptr;
    // Step 1: create copy nodes interleaved
    RNode* curr = head;
    while (curr) {
        RNode* copy = new RNode(curr->val);
        copy->next = curr->next;
        curr->next = copy;
        curr = copy->next;
    }
    // Step 2: assign random pointers
    curr = head;
    while (curr) {
        if (curr->random)
            curr->next->random = curr->random->next;
        curr = curr->next->next;
    }
    // Step 3: separate lists
    RNode dummy(0);
    RNode* newTail = &dummy;
    curr = head;
    while (curr) {
        newTail->next = curr->next;
        newTail = newTail->next;
        curr->next = curr->next->next;
        curr = curr->next;
    }
    return dummy.next;
}

```

Time: $O(n)$, **Space:** $O(1)$ extra (excluding output).

Real-life analogy: Cloning a reference-heavy document where some pages refer to other pages in the original. The interleaving trick ensures you know the new copy of the target when assigning the random pointer.

Summary Table

Concept	Technique	Time	Space
Find middle	Fast & slow pointers	$O(n)$	$O(1)$
Cycle detection	Floyd's algorithm	$O(n)$	$O(1)$
Find cycle start	Floyd + reset	$O(n)$	$O(1)$
Reverse list	Iterative / recursive	$O(n)$	$O(1)$ / $O(n)$ stack
Reverse in groups of k	Recursive with grouping	$O(n)$	$O(n/k)$ stack
Merge sorted lists	Two-pointer traversal	$O(m+n)$	$O(1)$
Intersection point	Length difference	$O(m+n)$	$O(1)$
Copy with random	Interleaving	$O(n)$	$O(1)$

The next chapter will cover stacks and queues (implementations, applications, and problems like next greater element, valid parentheses, sliding window maximum).

Chapter 5: Stacks and Queues

This chapter covers linear data structures that restrict access order: stacks (LIFO), queues (FIFO), deques, and priority queues (heaps). It includes implementations, applications, and classic problems.

1. Stack

What: A linear data structure that follows Last-In-First-Out (LIFO) principle. Elements are added and removed from the same end (top).

Real-life analogy: A stack of plates – you put a new plate on top and take the top plate first.

When to use:

- Function call management (call stack)
- Undo/Redo operations (e.g., text editors)
- Parsing expressions (balanced parentheses, postfix evaluation)
- Backtracking algorithms (DFS, maze solving)
- Browser history (back button)

1.1 Array-Based Implementation

```
class StackArray {
    private int[] arr;
    private int capacity;
    private int topIndex;

    public StackArray(int size) {
        capacity = size;
        topIndex = -1;
        arr = new int[capacity];
    }

    public void push(int x) {
        if (topIndex == capacity - 1) throw new RuntimeException("Stack
overflow");
        arr[++topIndex] = x;
    }

    public void pop() {
        if (topIndex < 0) throw new RuntimeException("Stack empty");
        topIndex--;
    }

    public int top() {
        if (topIndex < 0) throw new RuntimeException("Stack empty");
        return arr[topIndex];
    }

    public boolean empty() { return topIndex == -1; }
}
```

1.2 Linked-List-Based Implementation

```

class StackLL {
    private static class Node {
        int data;
        Node next;

        Node(int val) {
            this.data = val;
            this.next = null;
        }
    }

    private Node topNode;

    public StackLL() {
        topNode = null;
    }

    public void push(int x) {
        Node newNode = new Node(x);
        newNode.next = topNode;
        topNode = newNode;
    }

    public void pop() {
        if (topNode == null) throw new RuntimeException("Stack empty");
        Node temp = topNode;
        topNode = topNode.next;
    }

    public int top() {
        if (topNode == null) throw new RuntimeException("Stack empty");
        return topNode.data;
    }

    public boolean empty() { return topNode == null; }
}

```

1.3 Applications

Balanced Parentheses

Check if parentheses, brackets, and braces are properly nested.

```
boolean isBalanced(String expr) {
    Stack<Character> st = new Stack<>();
    for (char c : expr.toCharArray()) {
        if (c == '(' || c == '{' || c == '[')
            st.push(c);
        else if (c == ')' || c == '}' || c == ']') {
            if (st.isEmpty()) return false;
            char top = st.peek();
            if ((c == ')' && top != '(') ||
                (c == '}' && top != '{') ||
                (c == ']' && top != '['))
                return false;
            st.pop();
        }
    }
    return st.empty();
}
```

Expression Evaluation (Postfix)

When to use: Compilers, calculators.

Evaluate a postfix expression using a stack:

```

int evaluatePostfix(String postfix) {
    Stack<Integer> st = new Stack<>();
    for (char c : postfix.toCharArray()) {
        if (Character.isDigit(c)) st.push(c - '0');
        else {
            int b = st.pop();
            int a = st.pop();
            switch (c) {
                case '+': st.push(a + b); break;
                case '-': st.push(a - b); break;
                case '*': st.push(a * b); break;
                case '/': st.push(a / b); break;
            }
        }
    }
    return st.pop();
}

```

Infix to Postfix uses operator precedence and a stack (Shunting-yard algorithm).

Next Greater Element

What: For each element in an array, find the next element to the right that is greater.

```

List<Integer> nextGreaterElement(List<Integer> nums) {
    int n = nums.size();
    List<Integer> result = new ArrayList<>(Collections.nCopies(n, -1));
    Deque<Integer> st = new ArrayDeque<>(); // stores indices of
elements waiting for a greater element
    for (int i = 0; i < n; ++i) {
        while (!st.isEmpty() && nums.get(st.peek()) < nums.get(i)) {
            result.set(st.pop(), nums.get(i));
        }
        st.push(i);
    }
    return result;
}

```

Time: $O(n)$. **Real-life analogy:** Find the next taller person in a line looking to the right.

Largest Rectangle in Histogram

Problem: Given bar heights, find the largest rectangle area that can be formed.

Approach: Use a stack to find the nearest smaller bar on left and right.

```
int largestRectangleArea(vector<int>& heights) {
    stack<int> st;
    int maxArea = 0;
    heights.push_back(0); // sentinel
    for (int i = 0; i < heights.size(); ++i) {
        while (!st.empty() && heights[st.top()] > heights[i]) {
            int h = heights[st.top()];
            st.pop();
            int left = st.empty() ? -1 : st.top();
            maxArea = max(maxArea, h * (i - left - 1));
        }
        st.push(i);
    }
    heights.pop_back();
    return maxArea;
}
```

Min Stack (getMin in $O(1)$)

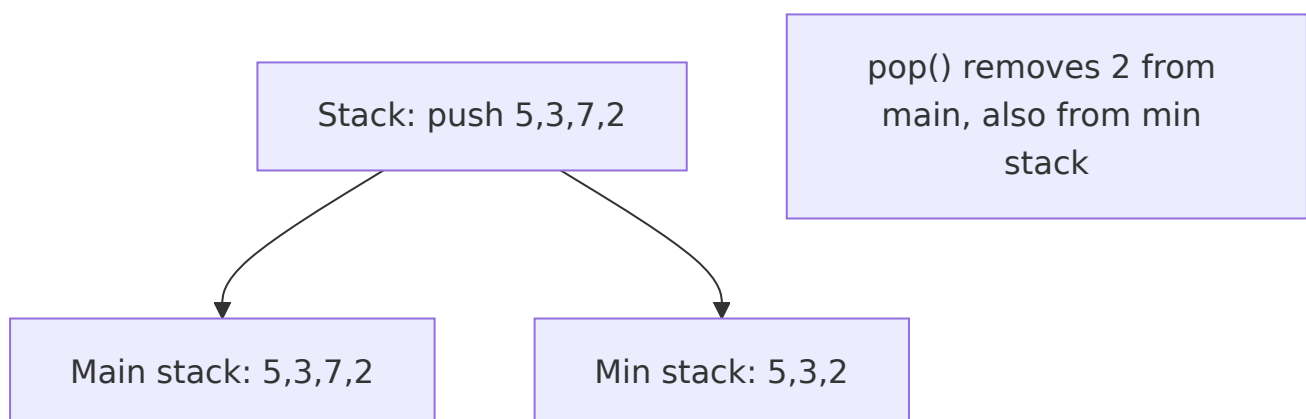
Design: A stack that supports `push`, `pop`, `top`, and `getMin` all in $O(1)$ time.

Approach: Use an auxiliary stack that stores the current minimum at each level.

```

class MinStack {
    stack<int> st;
    stack<int> minSt;
public:
    void push(int x) {
        st.push(x);
        if (minSt.empty() || x <= minSt.top())
            minSt.push(x);
    }
    void pop() {
        if (st.top() == minSt.top()) minSt.pop();
        st.pop();
    }
    int top() { return st.top(); }
    int getMin() { return minSt.top(); }
};

```



2. Queue

What: First-In-First-Out (FIFO) principle. Elements are added at the rear and removed from the front.

Real-life analogy: A queue of people waiting in line – the first person arrives first, gets served first.

When to use:

- Task scheduling (CPU, printer queue)
- Breadth-First Search (BFS) in graphs and trees

- Buffering (IO buffers, message queues)
- Sliding window problems

2.1 Array-Based Implementation (Circular Queue)

Uses a fixed-size array with `front` and `rear` pointers that wrap around.

```
class CircularQueue {
    int* arr;
    int capacity;
    int frontIdx, rearIdx;
    int count; // optional: number of elements
public:
    CircularQueue(int size) : capacity(size), frontIdx(0), rearIdx(-1),
count(0) {
        arr = new int[capacity];
    }
    ~CircularQueue() { delete[] arr; }
    bool enqueue(int x) {
        if (count == capacity) return false;
        rearIdx = (rearIdx + 1) % capacity;
        arr[rearIdx] = x;
        count++;
        return true;
    }
    bool dequeue() {
        if (count == 0) return false;
        frontIdx = (frontIdx + 1) % capacity;
        count--;
        return true;
    }
    int front() {
        if (count == 0) throw underflow_error("Queue empty");
        return arr[frontIdx];
    }
    bool empty() const { return count == 0; }
};
```

2.2 Linked-List-Based Queue

```
class QueueLL {
    struct Node {
        int data;
        Node* next;
        Node(int val) : data(val), next(nullptr) {}
    };
    Node* frontNode;
    Node* rearNode;
public:
    QueueLL() : frontNode(nullptr), rearNode(nullptr) {}
    void enqueue(int x) {
        Node* newNode = new Node(x);
        if (rearNode) rearNode->next = newNode;
        else frontNode = newNode;
        rearNode = newNode;
    }
    void dequeue() {
        if (!frontNode) throw underflow_error("Queue empty");
        Node* temp = frontNode;
        frontNode = frontNode->next;
        if (!frontNode) rearNode = nullptr;
        delete temp;
    }
    int front() {
        if (!frontNode) throw underflow_error("Queue empty");
        return frontNode->data;
    }
    bool empty() const { return frontNode == nullptr; }
};
```

2.3 Double-Ended Queue (Deque)

What: Allows insertion and deletion at both ends. Available as `std::deque` in C++.

When to use:

- Sliding window maximum/minimum
- Palindrome checking
- Job scheduling with priority at both ends

2.4 Applications

Sliding Window Maximum

Problem: Given an array and window size k , find the maximum element in each sliding window.

Approach: Use a deque storing indices of elements in decreasing order.

```
vector<int> slidingWindowMaximum(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> result;
    for (int i = 0; i < nums.size(); ++i) {
        // remove out-of-window indices
        if (!dq.empty() && dq.front() == i - k)
            dq.pop_front();
        // maintain decreasing order
        while (!dq.empty() && nums[dq.back()] <= nums[i])
            dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1)
            result.push_back(nums[dq.front()]);
    }
    return result;
}
```

Time: $O(n)$. **Real-life analogy:** You have a moving magnifying glass over a number line, and you always want the largest visible number.

BFS Traversal (using queue)

Queue is fundamental for level-order traversal of trees and graphs.

```

void bfs(Node* root) {
    if (!root) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* curr = q.front(); q.pop();
        cout << curr->val << " ";
        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}

```

3. Priority Queue (Heap)

What: A data structure that always gives access to the highest (or lowest) priority element. Typically implemented as a binary heap.

- **Max-heap:** Parent $\geq$ children; maximum at root.
- **Min-heap:** Parent $\leq$ children; minimum at root.

When to use:

- Scheduling tasks by priority
- Finding Kth largest/smallest elements
- Merging multiple sorted sequences
- Dijkstra's and Prim's algorithms

C++ Standard Library: `priority_queue<T>` (max-heap by default). Use `priority_queue<T, vector<T>, greater<T>>` for min-heap.

3.1 Heap Operations

Operation	Description	Time Complexity
<code>heapify</code>	Build heap from array	$O(n)$
<code>insert</code>	Add element, bubble up	$O(\log n)$
<code>extract min/max</code>	Remove root, heapify down	$O(\log n)$

get min/max	Peek root	$O(1)$
decrease key	Reduce value, bubble up	$O(\log n)$

3.2 Manual Heap Implementation (Min-Heap)

```

class MinHeap {
    vector<int> heap;
    void heapifyUp(int idx) {
        while (idx > 0) {
            int parent = (idx - 1) / 2;
            if (heap[parent] <= heap[idx]) break;
            swap(heap[parent], heap[idx]);
            idx = parent;
        }
    }
    void heapifyDown(int idx) {
        int size = heap.size();
        while (true) {
            int left = 2 * idx + 1;
            int right = 2 * idx + 2;
            int smallest = idx;
            if (left < size && heap[left] < heap[smallest]) smallest =
left;
            if (right < size && heap[right] < heap[smallest]) smallest =
right;
            if (smallest == idx) break;
            swap(heap[idx], heap[smallest]);
            idx = smallest;
        }
    }
public:
    void insert(int val) {
        heap.push_back(val);
        heapifyUp(heap.size() - 1);
    }
    int extractMin() {
        if (heap.empty()) throw underflow_error("Heap empty");
        int root = heap[0];
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) heapifyDown(0);
        return root;
    }
};

```

```

    }
    int getMin() { return heap[0]; }
    bool empty() { return heap.empty(); }
};

```

3.3 Application: Kth Largest Element

Approach: Use a min-heap of size k that stores the k largest elements seen so far.

```

int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int x : nums) {
        minHeap.push(x);
        if (minHeap.size() > k) minHeap.pop();
    }
    return minHeap.top();
}

```

Time: $O(n \log k)$. **Space:** $O(k)$.

3.4 Application: Merge K Sorted Lists

Approach: Push the first node of each list into a min-heap (ordered by node value). Repeatedly extract the smallest, push its next node.

```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

struct Compare {
    bool operator()(ListNode* a, ListNode* b) {
        return a->val > b->val; // min-heap
    }
};

ListNode* mergeKLists(vector<ListNode*>& lists) {
    priority_queue<ListNode*, vector<ListNode*>, Compare> pq;
    for (auto list : lists)
        if (list) pq.push(list);
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (!pq.empty()) {
        ListNode* smallest = pq.top(); pq.pop();
        tail->next = smallest;
        tail = tail->next;
        if (smallest->next) pq.push(smallest->next);
    }
    return dummy.next;
}

```

Time: $O(N \log k)$, where N is total nodes, k is number of lists.

3.5 Application: Dijkstra's Algorithm Support

Priority queue is used to always extract the node with the smallest tentative distance.

```

#include <queue>
#include <vector>
using namespace std;

void dijkstra(vector<vector<pair<int,int>>>& graph, int src) {
    int n = graph.size();
    vector<int> dist(n, INT_MAX);
    dist[src] = 0;
    using P = pair<int,int>; // (distance, node)
    priority_queue<P, vector<P>, greater<P>> pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : graph[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

```

Summary Table

Data Structure	Access Order	Implementation Options	Typical O(1) Operations	Applications
Stack	LIFO	Array, Linked List	push, pop, top	Parentheses, DFS, undo, expression eval
Queue	FIFO	Circular array, Linked List	enqueue, dequeue, front	BFS, scheduling, sliding window
Deque	Both ends	Doubly linked list or array	push/pop front/back	Sliding window maximum, palindrome

Priority Queue (Heap)	By priority	Binary heap	getMin, insert, extractMin	Kth largest, merge K lists, Dijkstra
-----------------------	-------------	-------------	----------------------------	--------------------------------------

The next chapter will cover advanced linear data structures: hash tables and maps.

Chapter 6: Recursion and Backtracking

This chapter covers recursion fundamentals, compares recursion with iteration, and explores backtracking as a systematic search technique. Classic problems (N-Queens, Sudoku, subsets, permutations, combination sum) illustrate practical applications.

1. Recursion Fundamentals

What: A function that calls itself to solve smaller instances of the same problem.

1.1 Components

- **Base case:** The simplest input that can be solved directly, terminating the recursion.
- **Recurrence relation:** Expresses the problem in terms of smaller instances of itself.
- **Call stack:** Automatically managed stack storing each active function call.

1.2 Recurrence Relation Example (Factorial)

```
factorial(0) = 1           // base case
factorial(n) = n * factorial(n-1)  // recurrence
```

```
int factorial(int n) {
    if (n <= 1) return 1;      // base case
    return n * factorial(n - 1); // recurrence
}
```

1.3 Tail Recursion vs Non-Tail Recursion

Tail recursion: The recursive call is the last operation performed by the function. The result of the recursive call is returned directly, with no additional computation after it.

```
// Tail-recursive factorial using an accumulator
int factorialTail(int n, int acc) {
    if (n <= 1) return acc;
    return factorialTail(n - 1, n * acc); // recursive call is last
}
```

Non-tail recursion: Additional work (e.g., multiplication) happens after the recursive call returns.

```
int factorialNonTail(int n) {
    if (n <= 1) return 1;
    return n * factorialNonTail(n - 1);    // multiplication after call
}
```

Why tail recursion matters: Some compilers can optimise tail recursion into iteration (tail call elimination), saving stack space and preventing stack overflow. Without optimisation, both have the same $O(n)$ space.

1.4 Recursion vs Iteration

Aspect	Recursion	Iteration
Code clarity	Often shorter and more intuitive for divide-and-conquer, trees, graphs	More verbose for naturally recursive problems
Performance	Function call overhead	No call overhead
Space complexity	May use $O(n)$ stack space (for depth n)	Usually $O(1)$ extra space
Risk	Stack overflow for deep recursion	No stack overflow
Typical use	Tree traversal, backtracking, divide-and-conquer	Simple loops, when depth is large

When to use recursion:

- Problem has a natural recursive structure (trees, graphs, divide-and-conquer)
- Depth is moderate (logarithmic or small constant factor)
- Clarity is more important than maximum performance

When to use iteration:

- Deep recursion ($n > 10^5$) is required
- Constant space is mandatory
- Performance is critical

2. Backtracking

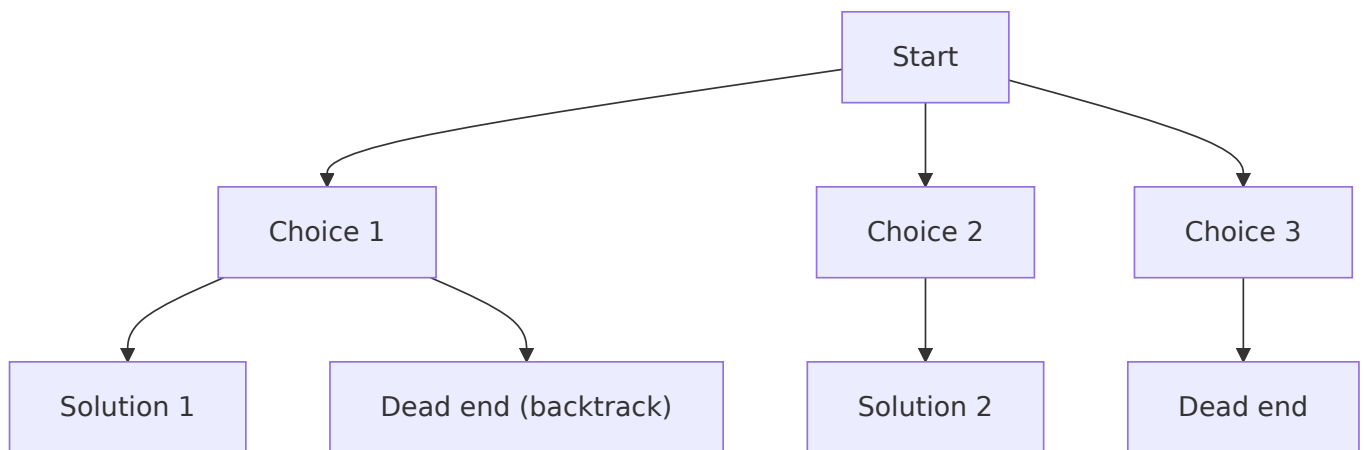
What: A systematic trial-and-error method that incrementally builds candidates and abandons (backtracks) those that cannot lead to valid solutions.

When to use:

- Combinatorial search: subsets, permutations, combinations
- Constraint satisfaction: N-Queens, Sudoku, crossword puzzles
- Pathfinding with constraints (Rat in a maze)

2.1 State Space Tree

Every recursive call represents a node in the state space tree. The root is the initial state. Each branch corresponds to a choice. Leaves are complete solutions or dead ends.



2.2 Pruning Techniques

Pruning means skipping entire branches of the state space tree when they cannot yield a valid solution or cannot beat the best known solution.

Unlike plain brute force, backtracking uses constraints to prune early.

Common pruning strategies:

- **Constraint-based:** If partial assignment violates a constraint, stop exploring this branch.
- **Optimisation-based:** In branch-and-bound, if current cost already exceeds best found, prune.
- **Symmetry / duplicate avoidance:** E.g., generating subsets with increasing indices to avoid repeats.

2.3 General Backtracking Pattern

```

void backtrack(state, choices):
    if is_goal(state):
        record(solution)
        return
    for each choice in choices:
        if is_valid(state + choice):
            make_choice(choice)
            backtrack(state + choice, next_choices)
            undo_choice(choice)

```

3. Classic Backtracking Problems

3.1 Generate All Subsets (Powerset)

Problem: Given a set of distinct integers, return all possible subsets (including empty set).

Approach: At each index, either include or exclude the element.

```

void subsets(List<Integer> nums, int index, List<Integer> current,
List<List<Integer>> result) {
    result.add(new ArrayList<>(current)); //
    include current subset (empty first)
    for (int i = index; i < nums.size(); ++i) {
        current.add(nums.get(i)); // choose
        subsets(nums, i + 1, current, result); // explore
        current.remove(current.size() - 1);
    } // backtrack
}

List<List<Integer>> generateSubsets(List<Integer> nums) {
    List<List<Integer>> result = new ArrayList<>();
    List<Integer> current = new ArrayList<>();
    subsets(nums, 0, current, result);
    return result;
}

```

Time: $O(2^n)$, **Space:** $O(n)$ stack + $O(2^n)$ output.

3.2 Generate Permutations

Problem: Return all possible permutations of a distinct array.

Approach: Swap elements to generate arrangements.

```
void permute(List<Integer> nums, int start, List<List<Integer>> result)
{
    if (start == nums.size()) {
        result.add(new ArrayList<>(nums));
        return;
    }
    for (int i = start; i < nums.size(); ++i) {
        Collections.swap(nums, start, i);          // choose
        permute(nums, start + 1, result);          // explore
        Collections.swap(nums, start, i);          // backtrack
    }
}

List<List<Integer>> generatePermutations(List<Integer> nums) {
    List<List<Integer>> result = new ArrayList<>();
    permute(nums, 0, result);
    return result;
}
```

Time: $O(n * n!)$, **Space:** $O(n)$ stack + $O(n * n!)$ output.

3.3 Combination Sum

Problem: Find all unique combinations of candidates (can be used unlimited times) that sum to target.

Approach: Sort candidates to avoid duplicates. At each step, either include the current candidate (stay at same index) or move to the next.

```

void combinationSum(List<Integer> candidates, int target, int start,
List<Integer> current, List<List<Integer>> result) {
    if (target == 0) {
        result.add(new ArrayList<>(current));
        return;
    }
    for (int i = start; i < candidates.size() && candidates.get(i) <=
target; ++i) {
        current.add(candidates.get(i));
        combinationSum(candidates, target - candidates.get(i), i,
current, result); // i, not i+1 → unlimited use
        current.remove(current.size() - 1);
    }
}

List<List<Integer>> combSum(List<Integer> candidates, int target) {
    Collections.sort(candidates);
    List<List<Integer>> result = new ArrayList<>();
    List<Integer> current = new ArrayList<>();
    combinationSum(candidates, target, 0, current, result);
    return result;
}

```

Time: $O(2^n)$ worst case (but pruned by target). **Space:** $O(n)$ stack.

3.4 N-Queens

Problem: Place N queens on an $N \times N$ chessboard so that no two attack each other (same row, column, diagonal).

Approach: Place queens row by row. Track columns and diagonals using sets or boolean arrays.

```

void solveNQueens(int n, int row, vector<int>& cols, vector<bool>&
diag1, vector<bool>& diag2, vector<vector<string>>& result) {
    if (row == n) {
        vector<string> board(n, string(n, '.'));
        for (int i = 0; i < n; ++i)
            board[i][cols[i]] = 'Q';
        result.push_back(board);
        return;
    }
    for (int col = 0; col < n; ++col) {
        int d1 = row - col + n - 1; // diagonal 1 index
        int d2 = row + col;          // diagonal 2 index
        if (cols[col] || diag1[d1] || diag2[d2]) continue;
        cols[col] = true;
        diag1[d1] = true;
        diag2[d2] = true;
        solveNQueens(n, row + 1, cols, diag1, diag2, result);
        cols[col] = false;
        diag1[d1] = false;
        diag2[d2] = false;
    }
}

vector<vector<string>> nQueens(int n) {
    vector<vector<string>> result;
    vector<int> cols(n, false);
    vector<bool> diag1(2*n - 1, false);
    vector<bool> diag2(2*n - 1, false);
    solveNQueens(n, 0, cols, diag1, diag2, result);
    return result;
}

```

Time: $O(n!)$ with pruning (exact count). **Space:** $O(n)$ stack.

3.5 Sudoku Solver

Problem: Fill a 9×9 Sudoku grid satisfying row, column, and 3×3 subgrid constraints.

Approach: Find empty cell, try digits 1-9, check validity, recurse, backtrack.

```
bool isValid(vector<vector<char>>& board, int row, int col, char ch) {
    for (int i = 0; i < 9; ++i) {
        if (board[row][i] == ch) return false;
        if (board[i][col] == ch) return false;
        if (board[3*(row/3) + i/3][3*(col/3) + i%3] == ch) return false;
    }
    return true;
}

bool solveSudokuHelper(vector<vector<char>>& board) {
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            if (board[i][j] == '.') {
                for (char ch = '1'; ch <= '9'; ++ch) {
                    if (isValid(board, i, j, ch)) {
                        board[i][j] = ch;
                        if (solveSudokuHelper(board)) return true;
                        board[i][j] = '.';
                    }
                }
                return false;
            }
        }
    }
    return true;
}

void solveSudoku(vector<vector<char>>& board) {
    solveSudokuHelper(board);
}
```

Time: Constraint-dependent; worst-case very high but pruned heavily.

3.6 Rat in a Maze

Problem: Find a path from top-left to bottom-right in a grid where some cells are blocked.

Approach: DFS with backtracking and visited marking.

```
bool ratInMaze(vector<vector<int>>& maze, int x, int y,
vector<vector<int>>& solution) {
    int n = maze.size();
    if (x == n-1 && y == n-1 && maze[x][y] == 1) {
        solution[x][y] = 1;
        return true;
    }
    if (x >= 0 && x < n && y >= 0 && y < n && maze[x][y] == 1 &&
solution[x][y] == 0) {
        solution[x][y] = 1;
        if (ratInMaze(maze, x+1, y, solution)) return true;
        if (ratInMaze(maze, x, y+1, solution)) return true;
        if (ratInMaze(maze, x-1, y, solution)) return true;
        if (ratInMaze(maze, x, y-1, solution)) return true;
        solution[x][y] = 0; // backtrack
        return false;
    }
    return false;
}
```

Time: $O(4^{(n^2)})$ worst case, but pruned by obstacles.

4. Summary

Concept	Key Points
Recursion	Base case, recurrence, call stack, tail vs non-tail
Recursion vs iteration	Trade-off: clarity vs stack usage
Backtracking	Search with pruning; state space tree
Subsets	Include/Exclude pattern; $O(2^n)$
Permutations	Swap pattern; $O(n * n!)$
Combination sum	Unlimited usage; sorted input

N-Queens	Row-by-row placement; diagonal tracking
Sudoku	Fill empty cells; constraint checking
Rat in a maze	DFS with visited marking

The next chapter will cover sorting algorithms (bubble, insertion, selection, merge, quick, heap, counting, radix, bucket) with complexity analysis and stability.

Chapter 7: Divide and Conquer

Divide and Conquer is a powerful algorithmic paradigm that solves a problem by breaking it into smaller subproblems, solving each subproblem recursively, and then combining the solutions. This chapter covers the fundamental three-step pattern, recurrence analysis, classic algorithms, when to avoid D&C, important interview problems, and comparisons with dynamic programming and greedy algorithms.

1. The Divide and Conquer Paradigm

1.1 Three Steps

1. **Divide:** Split the problem into smaller subproblems of the same type.
2. **Conquer:** Solve each subproblem recursively. If a subproblem is small enough, solve it directly (base case).
3. **Combine:** Merge the solutions of the subproblems into a solution for the original problem.

Real-life analogy: Organising a large tournament.

- **Divide:** Split all players into two equal halves.
- **Conquer:** Recursively find the champion of each half.
- **Combine:** Have the two champions play the final match.

1.2 Recurrence Relation

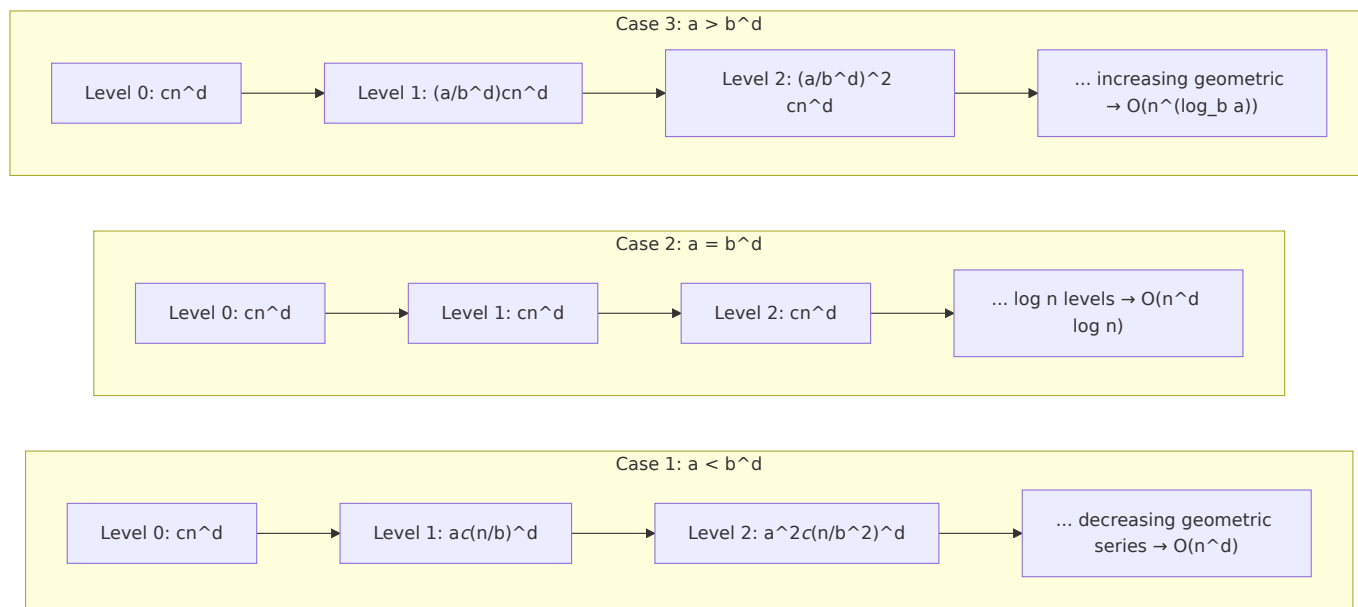
For a D&C algorithm that divides the problem into a subproblems of size n/b and takes $O(n^d)$ time for division and combination, the recurrence is:

$$T(n) = a * T(n/b) + O(n^d)$$

The solution (Master Theorem) depends on the relationship between a , b , and d :

- If $a < b^d$, then $T(n) = O(n^d)$ (work dominated by the combine step).
- If $a = b^d$, then $T(n) = O(n^d \log n)$.
- If $a > b^d$, then $T(n) = O(n^{\log_b a})$ (work dominated by the recursion tree).

Visualising the three cases (work at each level as a geometric series):



1.3 Relation to Recursion

Recursion is the *mechanism* (a function calling itself). Divide and Conquer is a *strategy* that uses recursion to implement the “conquer” step. Not all recursive algorithms are D&C (e.g., DFS on a graph is recursive but not divide-and-conquer because it does not divide the problem into independent subproblems).

2. Classic Divide and Conquer Algorithms

2.1 Binary Search

Problem: Find a target value in a sorted array.

Divide: Compare target with the middle element.

Conquer: Recursively search the left half (if target < mid) or the right half (if target > mid).

Combine: No explicit combine step – just return the result.

Recurrence: $T(n) = T(n/2) + O(1) \rightarrow T(n) = O(\log n)$.

```
int binarySearch(List<Integer> arr, int left, int right, int target) {
    if (left > right) return -1;
    int mid = left + (right - left) / 2;
    if (arr.get(mid) == target) return mid;
    if (arr.get(mid) < target) return binarySearch(arr, mid+1, right,
target);
    return binarySearch(arr, left, mid-1, target);
}
```

Real-life analogy: Looking up a name in a thick phone book – you open it in the middle, then decide whether to search the left or right half, repeating until you find the name.

2.2 Merge Sort

Problem: Sort an array.

Divide: Split the array into two halves (roughly equal size).

Conquer: Recursively sort each half.

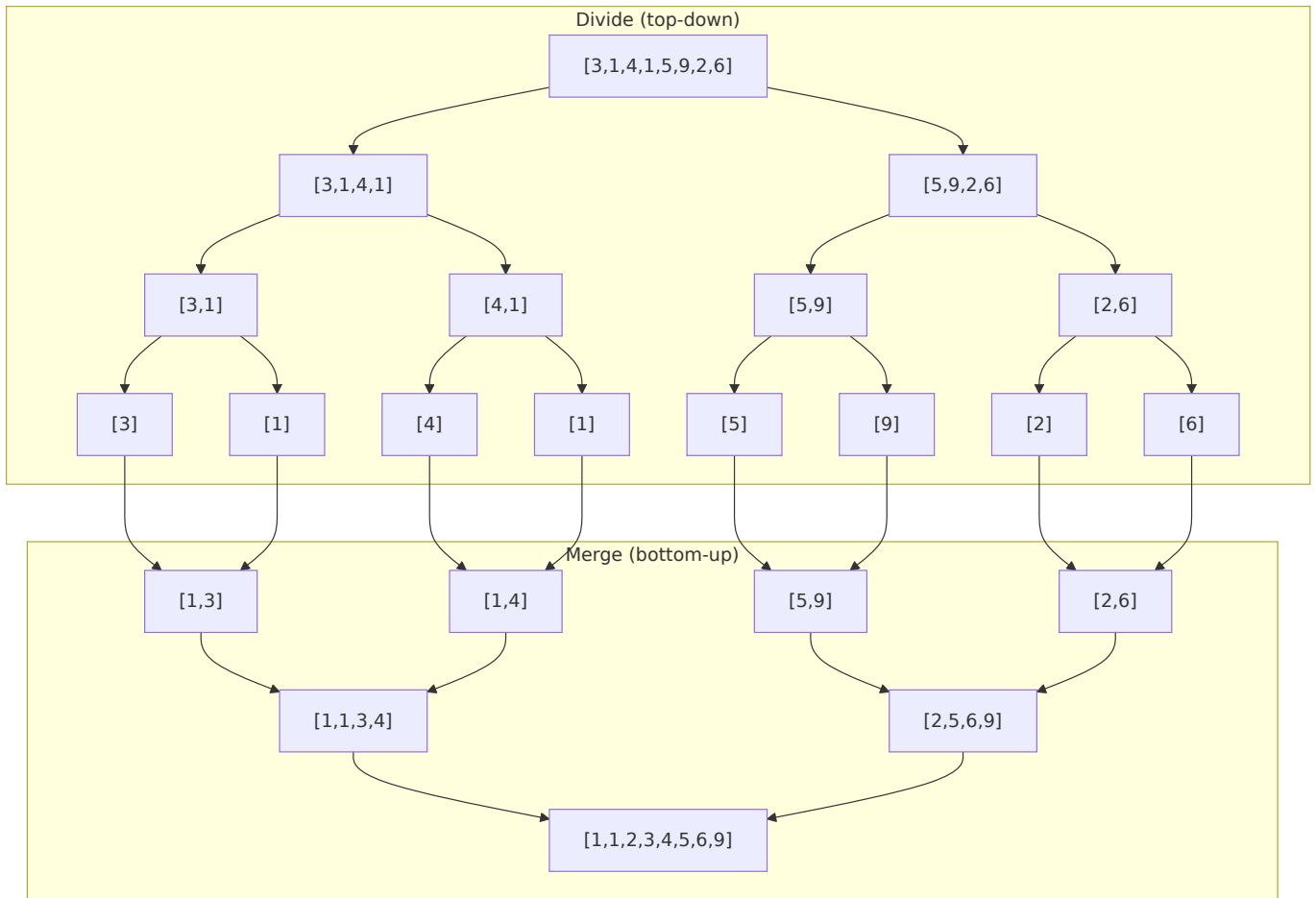
Combine: Merge the two sorted halves into a single sorted array.

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$.

```
void merge(List<Integer> arr, int left, int mid, int right) {
    List<Integer> temp = new ArrayList<>(right - left + 1);
    int i = left, j = mid+1, k = 0;
    while (i <= mid && j <= right)
        temp.add(k++, arr.get(i) <= arr.get(j) ? arr.get(i++) :
arr.get(j++));
    while (i <= mid) temp.add(k++, arr.get(i++));
    while (j <= right) temp.add(k++, arr.get(j++));
    for (int p = 0; p < temp.size(); ++p) arr.set(left + p,
temp.get(p));
}

void mergeSort(List<Integer> arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid+1, right);
    merge(arr, left, mid, right);
}
```

Real-life analogy: Sorting a large pile of exam papers by student ID – split the pile into two halves, sort each half recursively, then merge the sorted piles by repeatedly taking the smallest top paper.



2.3 Quick Sort (Conceptual)

Problem: Sort an array.

Divide: Choose a pivot element, partition the array so that elements less than the pivot come before it and elements greater come after.

Conquer: Recursively sort the left partition and the right partition.

Combine: No explicit combine step – the array is sorted in place.

Recurrence (average case): $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$.

Worst case (pivot always smallest/largest): $T(n) = T(n-1) + O(n) \rightarrow O(n^2)$.

```

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; ++j)
        if (arr[j] < pivot) swap(arr[++i], arr[j]);
    swap(arr[i+1], arr[high]);
    return i+1;
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi-1);
        quickSort(arr, pi+1, high);
    }
}

```

Real-life analogy: Sorting a dictionary by randomly picking a word (pivot) and arranging all words before/after it, then sorting the two groups independently.

2.4 Maximum Subarray Sum (Divide & Conquer Version)

Problem: Find the contiguous subarray with the largest sum.

Divide: Split array into left and right halves.

Conquer: Recursively compute the maximum subarray sum in the left half and in the right half.

Combine: The maximum subarray may cross the middle – compute the maximum sum that starts in the left and ends in the right, then take `max(leftBest, rightBest, crossBest)`.

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (Kadane's algorithm is $O(n)$, but this D&C version illustrates the paradigm).

```

int crossSum(vector<int>& arr, int left, int mid, int right) {
    int leftSum = INT_MIN, sum = 0;
    for (int i = mid; i >= left; --i) {
        sum += arr[i];
        leftSum = max(leftSum, sum);
    }
    int rightSum = INT_MIN;
    sum = 0;
    for (int i = mid+1; i <= right; ++i) {
        sum += arr[i];
        rightSum = max(rightSum, sum);
    }
    return leftSum + rightSum;
}

int maxSubarrayDAC(vector<int>& arr, int left, int right) {
    if (left == right) return arr[left];
    int mid = left + (right - left) / 2;
    int leftMax = maxSubarrayDAC(arr, left, mid);
    int rightMax = maxSubarrayDAC(arr, mid+1, right);
    int crossMax = crossSum(arr, left, mid, right);
    return max({leftMax, rightMax, crossMax});
}

```

2.5 Counting Inversions

Problem: Count the number of pairs (i, j) with $i < j$ and $arr[i] > arr[j]$ (inversions measure how far an array is from sorted).

Approach: Modify merge sort. When merging two sorted halves, if an element from the right half is smaller than an element from the left half, it forms inversions with all remaining elements in the left half.

Recurrence: Same as merge sort: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$.

```

long long mergeAndCount(vector<int>& arr, int left, int mid, int right)
{
    vector<int> temp;
    int i = left, j = mid+1;
    long long invCount = 0;
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) temp.push_back(arr[i++]);
        else {
            temp.push_back(arr[j++]);
            invCount += (mid - i + 1); // all remaining left elements >
arr[j]
        }
    }
    while (i <= mid) temp.push_back(arr[i++]);
    while (j <= right) temp.push_back(arr[j++]);
    for (int k = 0; k < temp.size(); ++k) arr[left + k] = temp[k];
    return invCount;
}

long long countInversions(vector<int>& arr, int left, int right) {
    if (left >= right) return 0;
    int mid = left + (right - left) / 2;
    long long inv = countInversions(arr, left, mid);
    inv += countInversions(arr, mid+1, right);
    inv += mergeAndCount(arr, left, mid, right);
    return inv;
}

```

Real-life analogy: In a crowd of people arranged by height, counting how many pairs are out of order (taller person standing before a shorter person). The divide-and-conquer approach counts inversions in each half and across the halves during merge.

2.6 Karatsuba Algorithm for Fast Multiplication

Problem: Multiply two large integers (or polynomials) faster than the standard $O(n^2)$ algorithm.

Key insight: For two n -digit numbers x and y , write:

$$x = a * 10^{(n/2)} + b$$

$$y = c * 10^{(n/2)} + d$$

Standard multiplication: $x*y = ac*10^n + (ad+bc)*10^{(n/2)} + bd \rightarrow 4$ multiplications
 $(ac, ad, bc, bd) \rightarrow O(n^2)$.

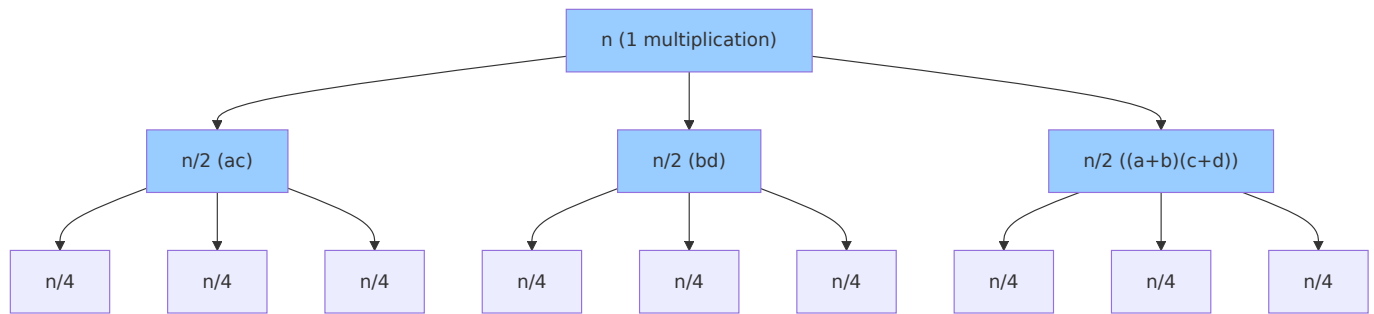
Karatsuba observation: compute:

```
ac = multiply(a, c)
bd = multiply(b, d)
(ad+bc) = multiply(a+b, c+d) - ac - bd
```

This requires only 3 multiplications $\rightarrow$ recurrence $T(n) = 3T(n/2) + O(n) \rightarrow$
 $O(n^{(\log_2 3)}) \approx O(n^{1.585})$.

```
// Simplified for integers (base 10)
long long karatsuba(long long x, long long y) {
    if (x < 10 || y < 10) return x * y;
    int n = max(to_string(x).size(), to_string(y).size());
    int m = n / 2;
    long long power = pow(10, m);
    long long a = x / power, b = x % power;
    long long c = y / power, d = y % power;
    long long ac = karatsuba(a, c);
    long long bd = karatsuba(b, d);
    long long sum = karatsuba(a+b, c+d) - ac - bd;
    return ac * pow(10, 2*m) + sum * pow(10, m) + bd;
}
```

Real-life analogy: Multiplying two large numbers by splitting them into halves – fewer large multiplications, more additions, which are cheaper.



2.7 Closest Pair of Points (2D, Conceptual)

Problem: Given n points in the plane, find the pair with the smallest Euclidean distance.

Divide: Sort points by x-coordinate. Split into left and right halves by the median x .

Conquer: Recursively find the minimum distance δ in the left half and in the right half.

Combine: The closest pair may cross the dividing line – consider only points within δ of the midline; sort these by y and check each against the next few (constant number) points.

Recurrence: $T(n) = 2T(n/2) + O(n \log n)$ (for sorting by y) $\rightarrow$ can be optimised to $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$.

Real-life analogy: Finding the two closest friends in a large party by first separating into left and right halves, finding closest pairs in each half, then checking pairs near the dividing line.

2.8 Fast Exponentiation (Power of a Number)

Problem: Compute a^b efficiently.

Divide: If b is even, $a^b = (a^{b/2})^2$. If odd, $a^b = a * a^{b-1}$.

Conquer: Recursively compute $a^{b/2}$.

Combine: Square the result (or multiply by a for odd case).

Recurrence: $T(n) = T(n/2) + O(1) \rightarrow O(\log b)$.

```

long long fastPow(long long a, long long b) {
    if (b == 0) return 1;
    long long half = fastPow(a, b / 2);
    if (b % 2 == 0) return half * half;
    return half * half * a;
}
  
```

Real-life analogy: Raising a number to a large power by repeatedly squaring – much faster than multiplying a by itself b times.

3. Divide and Conquer vs Other Paradigms

3.1 Divide and Conquer vs Dynamic Programming

Aspect	Divide and Conquer	Dynamic Programming
Subproblem nature	Independent (no overlap)	Overlapping subproblems
Solves	Many distinct subproblems (e.g., merge sort, quick sort)	Problems with repeated subproblems (e.g., Fibonacci, LCS)
Storage	Typically no memoisation	Uses memoisation or tabulation
Classic examples	Binary search, merge sort, closest pair	Knapsack, LCS, shortest paths

Key distinction: In D&C, subproblems are *disjoint* (they do not share smaller subproblems). In DP, subproblems *overlap* (same subproblem appears multiple times).

3.2 Divide and Conquer vs Greedy

Aspect	Divide and Conquer	Greedy
Decision making	Solves all subproblems, then combines	Makes one locally optimal choice per step, never revisits
Correctness	Always correct if recursion works	Requires proof of greedy choice property
Examples	Merge sort, Karatsuba	Activity selection, fractional knapsack

Key distinction: Greedy algorithms do not divide the problem into independent subproblems – they build a solution incrementally.

4. Summary Table

Algorithm	Divide Step	Combine Step	Recurrence	Time Complexity
-----------	-------------	--------------	------------	-----------------

Binary Search	Compare with middle	None	$T(n)=T(n/2)+O(1)$	$O(\log n)$
Merge Sort	Split into halves	Merge two sorted halves	$T(n)=2T(n/2)+O(n)$	$O(n \log n)$
Quick Sort	Partition around pivot	None (in-place)	Avg: $2T(n/2)+O(n)$	Avg $O(n \log n)$, worst $O(n^2)$
Max Subarray D&C	Split into halves	Compute cross sum	$T(n)=2T(n/2)+O(n)$	$O(n \log n)$
Counting Inversions	Split into halves	Count cross inversions during merge	$T(n)=2T(n/2)+O(n)$	$O(n \log n)$
Karatsuba	Split numbers into halves	Combine using 3 multiplications	$T(n)=3T(n/2)+O(n)$	$O(n^{1.585})$
Closest Pair	Split by x-coordinate	Check strip of width 2δ	$T(n)=2T(n/2)+O(n)$	$O(n \log n)$
Fast Exponentiation	Reduce exponent by half	Square/multiply	$T(n)=T(n/2)+O(1)$	$O(\log n)$

5. When NOT to Use Divide and Conquer

Despite its elegance, D&C is not always the best choice. Recognising when to avoid it is as important as knowing when to apply it.

5.1 Overlapping Subproblems → Use Dynamic Programming

If subproblems are not independent but overlap significantly, D&C will recompute the same subproblems many times, leading to exponential time. DP avoids this by storing results.

Example: Fibonacci numbers. D&C recursion `fib(n) = fib(n-1) + fib(n-2)` recomputes the same values exponentially ($O(2^n)$). DP solves it in $O(n)$.

Key question: Does the same subproblem appear repeatedly? If yes, consider DP.

5.2 Too Expensive Combine Step

Even if the divide and conquer steps are efficient, a costly combine operation can dominate the runtime.

Example: Simple recursive matrix multiplication (divide into 4 submatrices, 8 multiplications) has combine step $O(n^2)$ but still yields $O(n^3)$ – the same as naive. Strassen's algorithm reduces multiplications to 7, making the combine step worthwhile only for large n . If the combine step cannot be made sub-linear or linear, D&C may not improve over brute force.

5.3 Problem Size Does Not Reduce Significantly

If the problem cannot be divided into substantially smaller subproblems (e.g., each subproblem is only one or two elements smaller), the recursion depth becomes $O(n)$ and the total complexity may degrade to $O(n^2)$ or worse.

Example: Quick sort with poor pivot choice degrades to $O(n^2)$. In such cases, iterative algorithms or different paradigms (like heap sort) are better.

5.4 Recursion Overhead Is Too High

For very small input sizes, the overhead of function calls and stack management can outweigh the benefits of divide and conquer. In practice, many D&C algorithms switch to a simple iterative sort (e.g., insertion sort) for small subarrays – this is called **hybrid approach**.

5.5 The Problem Is Already Solved More Simply

Sometimes a greedy or direct mathematical formula exists.

Example: Finding the maximum subarray sum – D&C gives $O(n \log n)$, but Kadane's algorithm solves it in $O(n)$ with $O(1)$ space. Similarly, median of two sorted arrays can be solved with binary search (D&C) but can also be approached with two-pointer merging (though less efficient).

Decision Flowchart

Is there overlapping subproblems? → YES → Use DP

NO → Does the combine step take $> O(n \log n)$? → YES → Consider alternative

NO → Does the problem size reduce significantly? → YES → D&C may work

6. Important Interview Problems (Divide & Conquer Edition)

The following problems frequently appear in coding interviews and have elegant D&C solutions.

6.1 Majority Element (D&C Version)

Problem: Find the element that appears more than $\lfloor n/2 \rfloor$ times in an array (assume such an element exists).

D&C approach: Divide the array into two halves. Find the majority candidate in the left half and in the right half. If they are the same, that is the global majority. If different, count their frequencies in the whole array – the one with the higher count is the answer.

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (Moore's voting algorithm is $O(n)$, but D&C version is instructive).

```
int countFreq(vector<int>& nums, int target, int left, int right) {
    int cnt = 0;
    for (int i = left; i <= right; ++i)
        if (nums[i] == target) cnt++;
    return cnt;
}

int majorityElementDC(vector<int>& nums, int left, int right) {
    if (left == right) return nums[left];
    int mid = left + (right - left) / 2;
    int leftMaj = majorityElementDC(nums, left, mid);
    int rightMaj = majorityElementDC(nums, mid+1, right);
    if (leftMaj == rightMaj) return leftMaj;
    int leftCount = countFreq(nums, leftMaj, left, right);
    int rightCount = countFreq(nums, rightMaj, left, right);
    return leftCount > rightCount ? leftMaj : rightMaj;
}
```

6.2 The Skyline Problem

Problem: Given a list of buildings (left, right, height), return the skyline (list of key points where the height changes).

D&C approach:

- Divide the list of buildings into two halves.
- Recursively compute the skyline for each half.
- Merge the two skylines by sweeping from left to right, always keeping the taller height.

Time: $O(n \log n)$. **Space:** $O(n)$.

(Full implementation is lengthy; the key insight is that merging skylines is similar to merging two sequences of horizontal segments.)

6.3 Search in Rotated Sorted Array

Problem: A sorted array is rotated at an unknown pivot. Search for a target in $O(\log n)$.

D&C / Binary Search approach: Find which half is sorted and decide where to search.

```
int searchRotated(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[left] <= nums[mid]) { // left half sorted
            if (target >= nums[left] && target < nums[mid])
                right = mid - 1;
            else
                left = mid + 1;
        } else { // right half sorted
            if (target > nums[mid] && target <= nums[right])
                left = mid + 1;
            else
                right = mid - 1;
        }
    }
    return -1;
}
```

Time: $O(\log n)$. This is essentially binary search with an extra condition – a pure D&C example.

6.4 Median of Two Sorted Arrays

Problem: Find the median of two sorted arrays of sizes m and n in $O(\log(m+n))$ time.

D&C / Binary Search approach: Partition both arrays such that the left part contains $(m+n+1)/2$ elements and the maximum of the left part is less than or equal to the minimum of the right part.

```

double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
    if (nums1.size() > nums2.size())
        return findMedianSortedArrays(nums2, nums1);
    int m = nums1.size(), n = nums2.size();
    int low = 0, high = m;
    while (low <= high) {
        int partition1 = (low + high) / 2;
        int partition2 = (m + n + 1) / 2 - partition1;
        int maxLeft1 = (partition1 == 0) ? INT_MIN : nums1[partition1 - 1];
        int minRight1 = (partition1 == m) ? INT_MAX : nums1[partition1];
        int maxLeft2 = (partition2 == 0) ? INT_MIN : nums2[partition2 - 1];
        int minRight2 = (partition2 == n) ? INT_MAX : nums2[partition2];
        if (maxLeft1 <= minRight2 && maxLeft2 <= minRight1) {
            if ((m + n) % 2 == 0)
                return (max(maxLeft1, maxLeft2) + min(minRight1, minRight2)) / 2.0;
            else
                return max(maxLeft1, maxLeft2);
        } else if (maxLeft1 > minRight2) {
            high = partition1 - 1;
        } else {
            low = partition1 + 1;
        }
    }
    return 0.0;
}

```

Time: $O(\log(\min(m, n)))$. This is a classic divide-and-conquer (binary search) on the smaller array.

7. When to Apply D&C – Quick Reference

Condition	Recommendation
Problem can be split into independent subproblems	✅ Consider D&C

Subproblems overlap significantly	✗ Use DP
Combine step is $O(n)$ or $O(n \log n)$ and problem size halves	✓ D&C likely works
Combine step is $O(n^2)$ or higher	✗ Look for alternative
Problem is naturally recursive (trees, divide-and-conquer patterns)	✓ D&C is natural

The next chapter will cover dynamic programming in depth (overlapping subproblems, tabulation vs memoization, and classic DP problems).

Chapter 8: Sorting and Searching

This chapter covers essential sorting algorithms (comparison-based and non-comparison) and searching techniques, including binary search variations. Each algorithm includes asymptotic analysis, stability, in-place property, and C++ implementation.

1. Sorting Algorithms Overview

Sorting arranges elements in a specific order (ascending/descending). Choice of algorithm depends on input size, data characteristics, and space constraints.

1.1 Comparison Matrix

Algorithm	Best Case	Average Case	Worst Case	Space	Stable	In-place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes	No
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes	No

1.2 Key Concepts

- Comparison-based sort:** Uses element comparisons (e.g., `<`, `>`). Lower bound $\Omega(n \log n)$ for worst case.
- Non-comparison sort:** Uses digit or value properties. Can achieve $O(n)$ under constraints.
- In-place:** Requires $O(1)$ extra memory (excluding input).
- Stable:** Preserves relative order of equal elements.
- Adaptive:** Takes advantage of existing order (e.g., Insertion, Bubble).

2. Comparison-Based Sorts

2.1 Bubble Sort

What: Repeatedly swaps adjacent elements if out of order, pushing the largest element to the end each pass.

When to use: Educational purposes, nearly sorted small arrays.

```
void bubbleSort(List<Integer> arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        boolean swapped = false;
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr.get(j) > arr.get(j + 1)) {
                Collections.swap(arr, j, j + 1);
                swapped = true;
            }
        }
        if (!swapped) break; // adaptive
    }
}
```

Real-life analogy: Bubbles rising in a glass of soda – larger bubbles (elements) move up faster.

2.2 Selection Sort

What: Repeatedly selects the minimum element from the unsorted part and swaps it with the first unsorted element.

```

void selectionSort(List<Integer> arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        int minIdx = i;
        for (int j = i + 1; j < n; ++j)
            if (arr.get(j) < arr.get(minIdx)) minIdx = j;
        Collections.swap(arr, i, minIdx);
    }
}

```

When to use: When memory writes are expensive (minimal swaps: $O(n)$).

2.3 Insertion Sort

What: Builds the sorted array one element at a time by inserting each element into its correct position.

```

void insertionSort(List<Integer> arr) {
    int n = arr.size();
    for (int i = 1; i < n; ++i) {
        int key = arr.get(i);
        int j = i - 1;
        while (j >= 0 && arr.get(j) > key) {
            arr.set(j + 1, arr.get(j));
            j--;
        }
        arr.set(j + 1, key);
    }
}

```

When to use: Small arrays ($n \leq 20$), partially sorted data, online sorting (receiving stream).

Real-life analogy: Sorting a hand of playing cards – take one card and insert it into the correct position.

2.4 Merge Sort

What: Divide-and-conquer: split array in half, sort each half recursively, merge sorted halves.

```

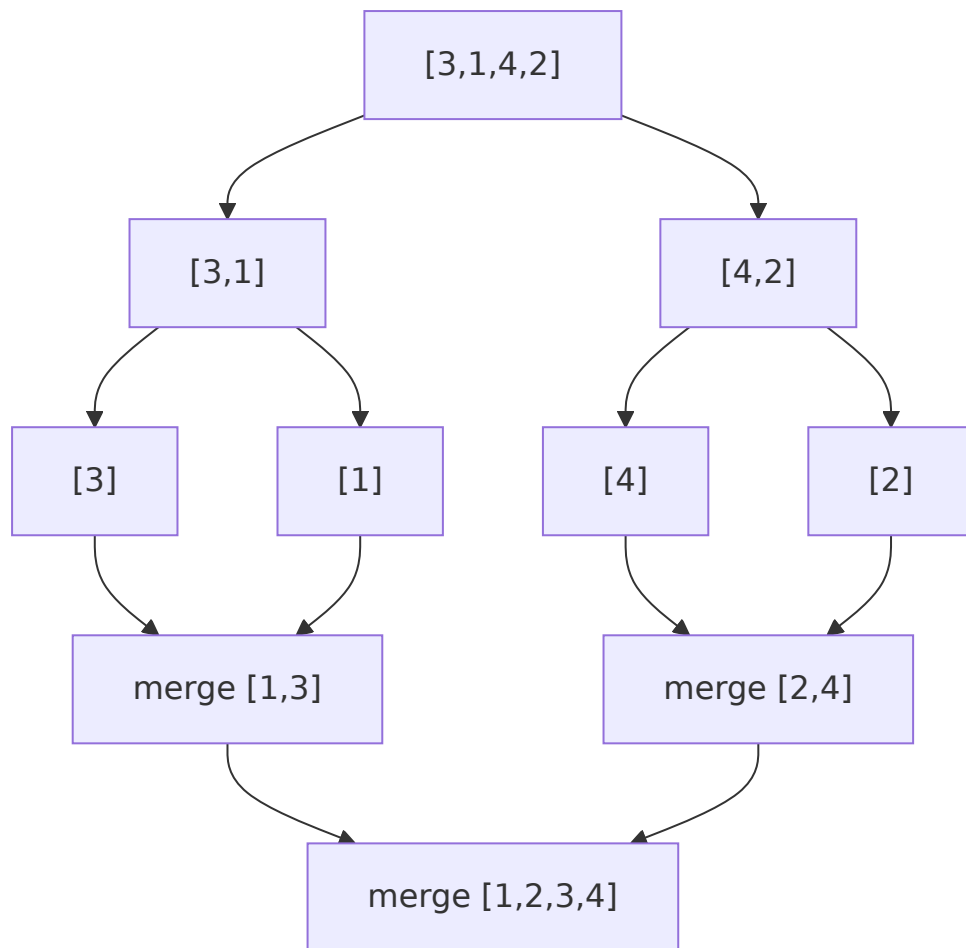
void merge(List<Integer> arr, int left, int mid, int right) {
    List<Integer> temp = new ArrayList<>(right - left + 1);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right)
        temp.add(k++, arr.get(i) <= arr.get(j) ? arr.get(i++) :
arr.get(j++));
    while (i <= mid) temp.add(k++, arr.get(i++));
    while (j <= right) temp.add(k++, arr.get(j++));
    for (int p = 0; p < temp.size(); ++p) arr.set(left + p,
temp.get(p));
}

void mergeSort(List<Integer> arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

```

When to use: Large datasets, stable sorting required, linked lists (no extra array needed for LL).

Time: $O(n \log n)$ always. **Space:** $O(n)$ for array.



2.5 Quick Sort

What: Picks a pivot, partitions array into elements $<$ pivot and $>$ pivot, recursively sorts partitions.

```

int partition(List<Integer> arr, int low, int high) {
    int pivot = arr.get(high);
    int i = low - 1;
    for (int j = low; j < high; ++j) {
        if (arr.get(j) < pivot) {
            i++;
            Collections.swap(arr, i, j);
        }
    }
    Collections.swap(arr, i + 1, high);
    return i + 1;
}

void quickSort(List<Integer> arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

```

When to use: General purpose, fastest on average, in-place. Avoid on already sorted data (use random pivot).

Tail recursion optimisation can reduce stack space.

2.6 Heap Sort

What: Build a max-heap, repeatedly extract root to end, heapify remaining.

```

void heapify(List<Integer> arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && arr.get(left) > arr.get(largest)) largest = left;
    if (right < n && arr.get(right) > arr.get(largest)) largest = right;
    if (largest != i) {
        Collections.swap(arr, i, largest);
        heapify(arr, n, largest);
    }
}

void heapSort(List<Integer> arr) {
    int n = arr.size();
    for (int i = n/2 - 1; i >= 0; --i) heapify(arr, n, i);
    for (int i = n - 1; i > 0; --i) {
        Collections.swap(arr, 0, i);
        heapify(arr, i, 0);
    }
}

```

When to use: Guaranteed $O(n \log n)$, $O(1)$ extra space, but unstable and slower than Quick Sort in practice.

3. Non-Comparison Sorts

3.1 Counting Sort

What: Count occurrences of each value (assuming small range $[0, k-1]$), then compute prefix sums to place elements in output.


```

void countingSort(List<Integer> arr) {
    if (arr.isEmpty()) return;
    int maxVal = Collections.max(arr);
    int minVal = Collections.min(arr);
    int range = maxVal - minVal + 1;
    int[] count = new int[range];
    List<Integer> output = new ArrayList<>(arr.size());
    for (int i = 0; i < arr.size(); i++) count[arr.get(i) - minVal]++;
    for (int i = 1; i < range; ++i) count[i] += count[i-1];
    for (int i = arr.size() - 1; i >= 0; --i) {
        output.set(count[arr.get(i) - minVal] - 1, arr.get(i));
        count[arr.get(i) - minVal]--;
    }
    for (int i = 0; i < arr.size(); i++) arr.set(i, output.get(i));
}

```

When to use: When range k is not too larger than n ($k = O(n)$). Stable version available.

3.2 Radix Sort

What: Sort digits from least significant to most significant using a stable sort (e.g., counting sort per digit).

```

void countingSortByDigit(List<Integer> arr, int exp) {
    List<Integer> output = new ArrayList<>(arr.size());
    int[] count = new int[10];
    for (int x : arr) count[(x / exp) % 10]++;
    for (int i = 1; i < 10; ++i) count[i] += count[i-1];
    for (int i = arr.size() - 1; i >= 0; --i) {
        int digit = (arr.get(i) / exp) % 10;
        output.set(count[digit] - 1, arr.get(i));
        count[digit]--;
    }
    for (int i = 0; i < arr.size(); i++) arr.set(i, output.get(i));
}

void radixSort(List<Integer> arr) {
    int maxVal = Collections.max(arr);
    for (int exp = 1; maxVal / exp > 0; exp *= 10)
        countingSortByDigit(arr, exp);
}

```

When to use: Fixed-length integer keys, when range is large but digit count small.

4. Searching

4.1 Linear Search

What: Traverse the array sequentially until target is found.

```

int linearSearch(List<Integer> arr, int target) {
    for (int i = 0; i < arr.size(); ++i)
        if (arr.get(i) == target) return i;
    return -1;
}

```

When to use: Unsorted data, small arrays, or when data is not random-access (e.g., linked list).

4.2 Binary Search (Iterative & Recursive)

What: Repeatedly divide sorted array in half, discarding the half that cannot contain the target.

```
int binarySearchIterative(List<Integer> arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr.get(mid) == target) return mid;
        else if (arr.get(mid) < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// Recursive version
int binarySearchRecursive(List<Integer> arr, int left, int right, int target) {
    if (left > right) return -1;
    int mid = left + (right - left) / 2;
    if (arr.get(mid) == target) return mid;
    if (arr.get(mid) < target) return binarySearchRecursive(arr, mid + 1, right, target);
    return binarySearchRecursive(arr, left, mid - 1, target);
}
```

Time: $O(\log n)$. **Space:** $O(1)$ iterative, $O(\log n)$ recursive (stack).

4.3 Binary Search Variations

First Occurrence

Return the smallest index where target appears.

```
int firstOccurrence(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1, result = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            result = mid;
            right = mid - 1; // continue left
        } else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return result;
}
```

Last Occurrence

Similarly, move `left` to `mid + 1` on match.

Search in Rotated Sorted Array

Problem: Find target in a sorted array that has been rotated (e.g., [4,5,6,7,0,1,2]).

Approach: Determine which half is sorted and check if target lies in it.

```

int searchRotated(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        // left half sorted
        if (arr[left] <= arr[mid]) {
            if (target >= arr[left] && target < arr[mid]) right = mid -
1;
            else left = mid + 1;
        }
        // right half sorted
        else {
            if (target > arr[mid] && target <= arr[right]) left = mid +
1;
            else right = mid - 1;
        }
    }
    return -1;
}

```

Find Peak Element

Problem: Find any peak element ($\text{arr}[i] > \text{neighbours}$). Assume $\text{arr}[-1] = \text{arr}[n] = -\infty$.

```

int findPeak(vector<int>& arr) {
    int left = 0, right = arr.size() - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] > arr[mid + 1]) right = mid;
        else left = mid + 1;
    }
    return left;
}

```

4.4 Ternary Search

What: Split array into three parts instead of two. Compare target with two midpoints.

```

int ternarySearch(vector<int>& arr, int left, int right, int target) {
    if (left > right) return -1;
    int mid1 = left + (right - left) / 3;
    int mid2 = right - (right - left) / 3;
    if (arr[mid1] == target) return mid1;
    if (arr[mid2] == target) return mid2;
    if (target < arr[mid1]) return ternarySearch(arr, left, mid1 - 1,
target);
    else if (target > arr[mid2]) return ternarySearch(arr, mid2 + 1,
right, target);
    else return ternarySearch(arr, mid1 + 1, mid2 - 1, target);
}

```

Complexity: $O(\log_3 n) \approx O(\log n)$ but with higher constant. Rarely used in practice.

5. Summary

Sort	Best for	Avoid when
Insertion	Small or nearly sorted	Large random data
Merge	Stable sort, large data	Tight memory constraints
Quick	General purpose, in-place	Already sorted (worst case)
Heap	Guaranteed $O(n \log n)$, $O(1)$ space	Stability needed
Counting/Radix	Integer keys with limited range	Wide value range or custom objects

Searching:

- Linear: unsorted or small arrays
- Binary: sorted arrays, $O(\log n)$
- Rotated search: requires adapted binary search

The next chapter will cover tree data structures (binary trees, BST, AVL, etc.).

Chapter 9: Hashing

Hashing provides near-constant-time insertion, deletion, and lookup by mapping keys to array indices using a hash function. This chapter covers hash functions, collision resolution strategies, load factor, rehashing, implementations of hash maps and sets, and classic applications.

1. Hash Functions

What: A function that maps a key (integer, string, or any object) to an integer index within a fixed range `[0, m-1]`.

Desirable properties:

- **Deterministic:** Same key always produces the same hash.
- **Fast to compute:** $O(1)$ or $O(\text{len}(\text{key}))$.
- **Uniform distribution:** Minimises collisions by spreading keys evenly.
- **Minimises collisions:** Different keys should rarely map to the same index.

Simple examples:

- Integer keys: `hash(key) = key % m` (choose `m` as a prime number to improve distribution).
- String keys: polynomial rolling hash `hash = (hash * BASE + char) % m`.

```
// Simple integer hash
int hashInt(int key, int m) { return key % m; }

// Polynomial rolling hash for strings
int hashString(String key, int m) {
    final int BASE = 31;
    long hash = 0;
    for (char c : key.toCharArray()) hash = (hash * BASE + c) % m;
    return (int)hash;
}
```

Real-life analogy: Library classification. A book's title (key) is mapped to a shelf number (hash). Good classification distributes books evenly across shelves.

2. Collision Resolution

A collision occurs when two different keys hash to the same index.

2.1 Chaining (Separate Chaining)

What: Each bucket contains a linked list (or dynamic array) of key-value pairs that hash to that bucket.

Operations:

- Insert: hash to bucket, add to list (check for duplicates if needed).
- Search: hash to bucket, traverse list.
- Delete: hash to bucket, remove from list.

Performance:

- Average: $O(1 + \alpha)$ where $\alpha = n/m$ (load factor).
- Worst case (all keys in one bucket): $O(n)$.

```
class HashMapChaining<K, V> {  
    static class Node<K, V> {  
        K key;  
        V value;  
        Node<K, V> next;  
    }  
}
```



```

    Node(K key, V value) {
        this.key = key;
        this.value = value;
        this.next = null;
    }
}

private Node<K, V>[] buckets;
private int size;
private int capacity;

private int hash(K key) {
    // Handle null keys
    if (key == null) return 0;
    return Math.abs(key.hashCode()) % capacity;
}

@SuppressWarnings("unchecked")
public HashMapChaining(int cap) {
    capacity = cap;
    size = 0;
    buckets = (Node<K, V>[]) new Node[capacity];
}

public void insert(K key, V value) {
    int idx = hash(key);
    Node<K, V> curr = buckets[idx];
    while (curr != null) {
        if (curr.key.equals(key)) {
            curr.value = value;
            return;
        }
        curr = curr.next;
    }
    Node<K, V> newNode = new Node<>(key, value);
    newNode.next = buckets[idx];
    buckets[idx] = newNode;
}

```

```
        size++;
    }

    public boolean find(K key, V[] out) {
        int idx = hash(key);
        Node<K, V> curr = buckets[idx];
        while (curr != null) {
            if (curr.key.equals(key)) {
                out[0] = curr.value;
                return true;
            }
            curr = curr.next;
        }
        return false;
    }
    // delete, destructor omitted for brevity
}
```

****When to use**:** Most general purpose; handles high load factor well.

2.2 Open Addressing

****What**:** All entries are stored in the array itself. When a collision occurs, the table is probed to find an empty slot.

****Probing sequences**:**

- ****Linear probing**:** $\text{index} = (\text{hash} + i) \% m$ for $i = 0, 1, 2, \dots$
 - Advantages: Cache friendly.
 - Disadvantage: Primary clustering $\rightarrow$ long runs of occupied slots.
- ****Quadratic probing**:** $\text{index} = (\text{hash} + c_1*i + c_2*i^2) \% m$.
 - Reduces clustering but may not probe all slots.
- ****Double hashing**:** $\text{index} = (\text{hash}_1 + i * \text{hash}_2(\text{key})) \% m$, where hash_2 is a second hash function.
 - Provides good distribution, avoids clustering.

****Performance**:** Sensitive to load factor; typically keep $\alpha < 0.7$.

```java

```
class HashMapOpenAddressing {
 private static class Entry {
 int key;
 int value;

 Entry(int key, int value) {
 this.key = key;
 this.value = value;
 }
 }

 private Entry[] table;
 private boolean[] occupied;
```

```

private boolean[] deleted; // deleted flag for lazy deletion
private int capacity;
private int size;

private int hash1(int key) { return key % capacity; }
private int hash2(int key) { return 1 + (key % (capacity - 1)); }

public HashMapOpenAddressing(int cap) {
 capacity = cap;
 size = 0;
 table = new Entry[capacity];
 occupied = new boolean[capacity];
 deleted = new boolean[capacity];
}

public void insert(int key, int value) {
 int idx = hash1(key);
 int step = hash2(key);
 for (int i = 0; i < capacity; ++i) {
 int probe = (idx + i * step) % capacity;
 if (!occupied[probe] || deleted[probe]) {
 table[probe] = new Entry(key, value);
 occupied[probe] = true;
 deleted[probe] = false;
 size++;
 return;
 }
 if (occupied[probe] && table[probe].key == key) {
 table[probe].value = value; // update
 return;
 }
 }
 // table full – rehash needed
}

// find and erase similar
};

```

### 3. Load Factor and Rehashing

**Load factor**  $\alpha$  = number of entries / table size.

- For chaining,  $\alpha > 1$  is acceptable (average chain length =  $\alpha$ ).
- For open addressing, keep  $\alpha < 0.7$  to avoid excessive probing.

**Rehashing:** When  $\alpha$  exceeds a threshold (e.g., 0.75), allocate a larger table (usually double size) and re-insert all entries using the new hash function.

```
void rehash() {
 int newCapacity = capacity * 2;
 Entry[] oldTable = table;
 boolean[] oldOccupied = occupied;
 capacity = newCapacity;
 table = new Entry[capacity];
 occupied = new boolean[capacity];
 deleted = new boolean[capacity];
 size = 0;
 for (int i = 0; i < oldTable.length; ++i) {
 if (oldOccupied[i] && !deleted[i])
 insert(oldTable[i].key, oldTable[i].value);
 }
}
```

### 4. Hash Map and Hash Set Implementations

#### 4.1 Hash Map (Dictionary)

Stores key-value pairs. C++ `std::unordered_map` (average  $O(1)$  operations).

#### 4.2 Hash Set

Stores only keys (no values). C++ `std::unordered_set`.

**Simple hash set using chaining:**

```

class HashSet {
 private List<List<Integer>> buckets;

 private int hash(int key) { return Math.abs(key) % buckets.size(); }

 public HashSet(int cap) {
 buckets = new ArrayList<>(cap);
 for (int i = 0; i < cap; i++) {
 buckets.add(new ArrayList<>());
 }
 }

 public void add(int key) {
 int idx = hash(key);
 List<Integer> bucket = buckets.get(idx);
 if (bucket.contains(key)) return;
 bucket.add(key);
 }

 public boolean contains(int key) {
 int idx = hash(key);
 List<Integer> bucket = buckets.get(idx);
 return bucket.contains(key);
 }

 public void remove(int key) {
 int idx = hash(key);
 List<Integer> bucket = buckets.get(idx);
 bucket.remove(Integer.valueOf(key));
 }
};

```

## 5. Applications

### 5.1 Two-Sum Problem

**Problem:** Given an array of integers and a target, return indices of two numbers that add to target.

**Approach:** Use a hash map from value to index. For each element `nums[i]`, check if `target - nums[i]` exists in the map.

```
int[] twoSum(int[] nums, int target) {
 Map<Integer, Integer> m = new HashMap<>(); // value -> index
 for (int i = 0; i < nums.length; ++i) {
 int complement = target - nums[i];
 if (m.containsKey(complement))
 return new int[] {m.get(complement), i};
 m.put(nums[i], i);
 }
 return new int[] {};
}
```

**Time:**  $O(n)$ . **Space:**  $O(n)$ .

## 5.2 Subarray Sum Equals K

**Problem:** Count number of contiguous subarrays whose sum equals k.

**Approach:** Use prefix sum and a hash map storing frequency of each prefix sum. For each prefix sum `curr`, add the count of `curr - k` seen so far.

```
int subarraySum(int[] nums, int k) {
 Map<Integer, Integer> freq = new HashMap<>();
 freq.put(0, 1); // empty prefix sum
 int prefix = 0, count = 0;
 for (int x : nums) {
 prefix += x;
 count += freq.getOrDefault(prefix - k, 0);
 freq.put(prefix, freq.getOrDefault(prefix, 0) + 1);
 }
 return count;
}
```

**Time:**  $O(n)$ . **Space:**  $O(n)$ .

## 5.3 Longest Consecutive Sequence

**Problem:** Find the length of the longest consecutive elements sequence in unsorted array ( $O(n)$  time).

**Approach:** Insert all numbers into a hash set. For each number, if its predecessor (`num - 1`) is not in the set, it is the start of a sequence. Then count consecutive numbers.

```
int longestConsecutive(int[] nums) {
 Set<Integer> s = new HashSet<>();
 for (int num : nums) {
 s.add(num);
 }
 int best = 0;
 for (int x : nums) {
 if (!s.contains(x - 1)) {
 int len = 1;
 while (s.contains(x + len)) len++;
 best = Math.max(best, len);
 }
 }
 return best;
}
```

**Time:**  $O(n)$  (each element visited at most twice). **Space:**  $O(n)$ .

## 5.4 Group Anagrams

**Problem:** Group strings that are anagrams of each other.

**Approach:** For each string, sort its characters to form a key, or use a frequency count as key.



```

List<List<String>> groupAnagrams(String[] strs) {
 Map<String, List<String>> groups = new HashMap<>();
 for (String s : strs) {
 char[] chars = s.toCharArray();
 Arrays.sort(chars);
 String key = new String(chars);
 groups.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
 }
 return new ArrayList<>(groups.values());
}

```

**Time:**  $O(n * k \log k)$  where  $k$  is max string length. **Space:**  $O(nk)$ .

## 5.5 LRU Cache (Hash Map + Doubly Linked List)

**Problem:** Design a cache that evicts the least recently used item when capacity is exceeded.

**Approach:**

- Hash map: key  $\rightarrow$  node pointer (for  $O(1)$  access).
- Doubly linked list: nodes in order of recent use (most recent at head, least at tail).

**Operations:**

- `get(key)` : if key exists, move node to head, return value.
- `put(key, value)` : if key exists, update value and move to head; else insert new node at head; if over capacity, remove tail.

```

class LRUCache {
 struct Node {
 int key, val;
 Node *prev, *next;
 Node(int k, int v) : key(k), val(v), prev(nullptr),
next(nullptr) {}
 };
 unordered_map<int, Node*> m;
 Node *head, *tail;
 int cap, size;

 void addToHead(Node* node) {
 node->next = head->next;
 node->prev = head;
 head->next->prev = node;
 head->next = node;
 }

 void removeNode(Node* node) {
 node->prev->next = node->next;
 node->next->prev = node->prev;
 }

 void moveToHead(Node* node) {
 removeNode(node);
 addToHead(node);
 }

 Node* removeTail() {
 Node* node = tail->prev;
 removeNode(node);
 return node;
 }
public:
 LRUCache(int capacity) : cap(capacity), size(0) {
 head = new Node(-1, -1);
 tail = new Node(-1, -1);
 head->next = tail;
 tail->prev = head;
 }
}

```

```

int get(int key) {
 if (m.find(key) == m.end()) return -1;
 Node* node = m[key];
 moveToHead(node);
 return node->val;
}

void put(int key, int value) {
 if (m.find(key) != m.end()) {
 Node* node = m[key];
 node->val = value;
 moveToHead(node);
 return;
 }
 Node* newNode = new Node(key, value);
 m[key] = newNode;
 addToHead(newNode);
 size++;
 if (size > cap) {
 Node* toRemove = removeTail();
 m.erase(toRemove->key);
 delete toRemove;
 size--;
 }
}
};

```

**Time:**  $O(1)$  per operation. **Space:**  $O(\text{capacity})$ .

**Real-life analogy:** A librarian keeps frequently borrowed books on a shelf near the entrance (most recent). When new books arrive, the least recently borrowed book is moved to storage.

## 6. Summary

| Concept       | Key Points                                                  |
|---------------|-------------------------------------------------------------|
| Hash function | Maps key to index; uniform distribution reduces collisions  |
| Chaining      | Simple, handles $\alpha > 1$ ; uses linked lists per bucket |

|                     |                                                                                           |
|---------------------|-------------------------------------------------------------------------------------------|
| Open addressing     | Stores all entries in array; uses probing sequences                                       |
| Load factor         | $n/m$ ; rehash when exceeding threshold (e.g., 0.75)                                      |
| Hash map / set      | Average $O(1)$ operations; <code>unordered_map</code> / <code>unordered_set</code> in C++ |
| Two-sum             | Hash map stores seen values                                                               |
| Subarray sum = K    | Prefix sum + frequency map                                                                |
| Longest consecutive | Hash set enables $O(n)$ sequence detection                                                |
| Group anagrams      | Sorted string or frequency count as key                                                   |
| LRU Cache           | Hash map + doubly linked list gives $O(1)$ get/put                                        |

The next chapter will cover binary trees, binary search trees (BST), AVL trees, and tree traversals.

# Chapter 10: Trees

Trees are hierarchical data structures where each node has zero or more children. This chapter covers binary trees, binary search trees, balanced BSTs, heaps, and advanced trees (Trie, Segment Tree, Fenwick Tree, Disjoint Set Union). Each section includes definitions, operations, complexity, C++ code, and real-life analogies.

## 1. Binary Trees

A binary tree is a tree where each node has at most two children: left and right.

### 1.1 Terminology

- **Root:** Topmost node (no parent).
- **Parent / Child:** Direct connection upward / downward.
- **Leaf:** Node with no children.
- **Height:** Number of edges on the longest path from node to a leaf (root height = tree height).
- **Depth:** Number of edges from root to the node.

```
 1 (root, depth=0)
 / \
 2 3
 / \ \
 4 5 6 (leaf)
```

**Height of node 1** = 2 (path 1-2-5 or 1-3-6).

**Depth of node 5** = 2.

**Real-life analogy:** A family genealogy tree – root is the earliest ancestor, children are descendants, leaves are people with no known descendants.

### 1.2 Tree Traversals

Traversal = visiting every node exactly once.

#### Depth-First Search (DFS) variants

| Traversal | Order               | Use Case                          |
|-----------|---------------------|-----------------------------------|
| Preorder  | root → left → right | Copying tree, prefix expression   |
| Inorder   | left → root → right | BST gives sorted order            |
| Postorder | left → right → root | Deleting tree, postfix expression |

**Recursive implementations:**

```

class TreeNode {
 int val;
 TreeNode left, right;

 TreeNode(int x) {
 this.val = x;
 this.left = null;
 this.right = null;
 }
}

void preorder(TreeNode root) {
 if (root == null) return;
 System.out.print(root.val + " ");
 preorder(root.left);
 preorder(root.right);
}

void inorder(TreeNode root) {
 if (root == null) return;
 inorder(root.left);
 System.out.print(root.val + " ");
 inorder(root.right);
}

void postorder(TreeNode root) {
 if (root == null) return;
 postorder(root.left);
 postorder(root.right);
 System.out.print(root.val + " ");
}

```

**Iterative inorder (using stack):**

```

void inorderIterative(TreeNode root) {
 Stack<TreeNode> st = new Stack<>();
 TreeNode curr = root;
 while (curr != null || !st.isEmpty()) {
 while (curr != null) {
 st.push(curr);
 curr = curr.left;
 }
 curr = st.pop();
 System.out.print(curr.val + " ");
 curr = curr.right;
 }
}

```

## Breadth-First Search (Level Order)

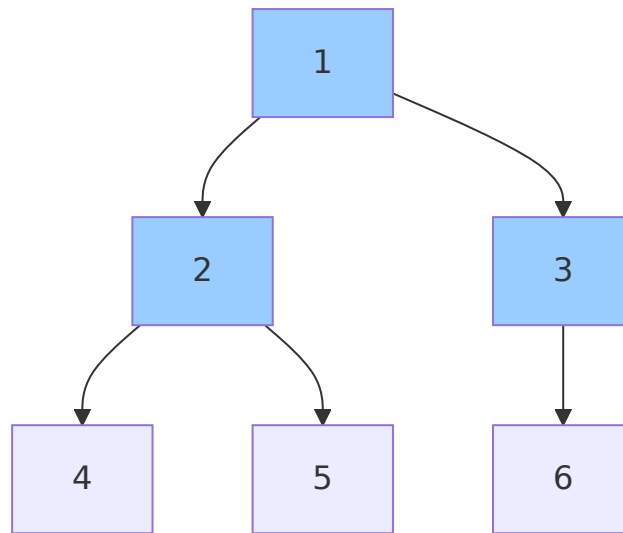
Visits level by level. Uses a queue.

```

void levelOrder(TreeNode root) {
 if (root == null) return;
 Queue<TreeNode> q = new LinkedList<>();
 q.add(root);
 while (!q.isEmpty()) {
 TreeNode node = q.poll();
 System.out.print(node.val + " ");
 if (node.left != null) q.add(node.left);
 if (node.right != null) q.add(node.right);
 }
}

```





- Preorder: 1,2,4,5,3,6
- Inorder: 4,2,5,1,3,6
- Postorder: 4,5,2,6,3,1
- Level order: 1,2,3,4,5,6

## 1.3 Construction of Tree from Traversals

**From inorder and preorder:** First element of preorder is root. Find it in inorder – left part is left subtree, right part is right subtree. Recurse.

```
TreeNode buildTree(List<Integer> preorder, List<Integer> inorder, int
preStart, int inStart, int inEnd, Map<Integer, Integer> inMap) {
 if (preStart >= preorder.size() || inStart > inEnd) return null;
 TreeNode root = new TreeNode(preorder.get(preStart));
 int rootIdx = inMap.get(root.val);
 int leftSize = rootIdx - inStart;
 root.left = buildTree(preorder, inorder, preStart+1, inStart,
rootIdx-1, inMap);
 root.right = buildTree(preorder, inorder, preStart+leftSize+1,
rootIdx+1, inEnd, inMap);
 return root;
}
```

## 1.4 Diameter of Binary Tree

**Definition:** The longest path (in edges) between any two nodes. It may or may not pass through the root.

**Approach:** For each node, compute height of left and right subtrees, update diameter as  $\text{leftHeight} + \text{rightHeight}$ . Return height.

```
int diameter(TreeNode root, int[] res) {
 if (root == null) return 0;
 int left = diameter(root.left, res);
 int right = diameter(root.right, res);
 res[0] = Math.max(res[0], left + right);
 return 1 + Math.max(left, right);
}
int diameterOfBinaryTree(TreeNode root) {
 int[] res = new int[1];
 diameter(root, res);
 return res[0];
}
```

## 1.5 Lowest Common Ancestor (LCA)

**Definition:** The deepest node that is an ancestor of both given nodes p and q.

**Approach:** Recursively search. If root matches p or q, return root. Otherwise, recurse left and right. If both sides return non-null, LCA is root; else return the non-null side.

```
TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
 if (root == null || root == p || root == q) return root;
 TreeNode left = lowestCommonAncestor(root.left, p, q);
 TreeNode right = lowestCommonAncestor(root.right, p, q);
 if (left != null && right != null) return root;
 return (left != null) ? left : right;
}
```

**Real-life analogy:** Finding the common manager of two employees in an organisational hierarchy.

## 1.6 Maximum Path Sum

**Problem:** Find the maximum sum path between any two nodes (values can be negative).

**Approach:** For each node, compute the maximum gain from its left and right children (negative gains are ignored). Update global maximum as `root->val + leftGain + rightGain`. Return `root->val + max(leftGain, rightGain)` to parent.

```
int maxPathSum(TreeNode root, int[] ans) {
 if (root == null) return 0;
 int left = Math.max(0, maxPathSum(root.left, ans));
 int right = Math.max(0, maxPathSum(root.right, ans));
 ans[0] = Math.max(ans[0], root.val + left + right);
 return root.val + Math.max(left, right);
}

int maxPathSum(TreeNode root) {
 int[] ans = new int[1];
 ans[0] = Integer.MIN_VALUE;
 maxPathSum(root, ans);
 return ans[0];
}
```

## 2. Binary Search Trees (BST)

**Property:** For every node, all nodes in left subtree have value  $<$  node value, and all nodes in right subtree have value  $>$  node value. No duplicates.

### 2.1 Search, Insert, Delete

**Search:**  $O(\log n)$  average,  $O(n)$  worst (skewed).

```
TreeNode* searchBST(TreeNode* root, int val) {
 while (root && root->val != val) {
 if (val < root->val) root = root->left;
 else root = root->right;
 }
 return root;
}
```

**Insert:**  $O(\log n)$  average.

```

TreeNode* insertIntoBST(TreeNode* root, int val) {
 if (!root) return new TreeNode(val);
 if (val < root->val) root->left = insertIntoBST(root->left, val);
 else if (val > root->val) root->right = insertIntoBST(root->right,
val);
 return root;
}

```

**Delete:** Three cases: leaf, one child, two children. For two children, replace with inorder successor (smallest in right subtree) or predecessor (largest in left).

```

TreeNode* findMin(TreeNode* root) {
 while (root->left) root = root->left;
 return root;
}
TreeNode* deleteNode(TreeNode* root, int key) {
 if (!root) return nullptr;
 if (key < root->val) root->left = deleteNode(root->left, key);
 else if (key > root->val) root->right = deleteNode(root->right,
key);
 else {
 if (!root->left) return root->right;
 if (!root->right) return root->left;
 TreeNode* succ = findMin(root->right);
 root->val = succ->val;
 root->right = deleteNode(root->right, succ->val);
 }
 return root;
}

```

## 2.2 Validate BST

Check that every node respects the BST property with a valid range.

```
bool isValidBST(TreeNode* root, long long minVal = LLONG_MIN, long long
maxVal = LLONG_MAX) {
 if (!root) return true;
 if (root->val <= minVal || root->val >= maxVal) return false;
 return isValidBST(root->left, minVal, root->val) &&
 isValidBST(root->right, root->val, maxVal);
}
```

## 2.3 Kth Smallest / Largest Element

**Kth smallest:** Inorder traversal yields sorted order. Count while traversing.

```
int kthSmallest(TreeNode* root, int k) {
 stack<TreeNode*> st;
 TreeNode* curr = root;
 while (curr || !st.empty()) {
 while (curr) { st.push(curr); curr = curr->left; }
 curr = st.top(); st.pop();
 if (--k == 0) return curr->val;
 curr = curr->right;
 }
 return -1;
}
```

**Kth largest:** Reverse inorder (right-root-left) or  $n - k + 1$  th smallest.

## 2.4 Range Sum Queries

Sum of all node values between L and R (inclusive). Recursive.

```
int rangeSumBST(TreeNode* root, int L, int R) {
 if (!root) return 0;
 if (root->val < L) return rangeSumBST(root->right, L, R);
 if (root->val > R) return rangeSumBST(root->left, L, R);
 return root->val + rangeSumBST(root->left, L, R) + rangeSumBST(root-
>right, L, R);
}
```

## 3. Balanced BSTs (Conceptual)

Balanced BSTs maintain  $O(\log n)$  height after insertions/deletions.

### 3.1 AVL Trees

**Balance factor** =  $\text{height}(\text{left}) - \text{height}(\text{right})$ . Allowed values: -1, 0, 1.

**Rotations:** Left, Right, Left-Right, Right-Left to restore balance after insertion/deletion.

**Use case:** When worst-case  $O(\log n)$  is required (e.g., database indexes).

### 3.2 Red-Black Trees

**Rules** (simplified):

1. Root is black.
2. Red nodes have black children.
3. Every path from root to leaf has the same number of black nodes.

**Why used:** C++ `std::map` and `std::set` are typically implemented as Red-Black trees.

Fewer rotations than AVL, good performance.

## 4. Heap (Binary Heap)

A complete binary tree (all levels filled except possibly last, which fills left-to-right). Stored as an array: parent at index  $i$ , children at  $2i+1$  and  $2i+2$ .

**Min-heap:**  $\text{parent} \leq \text{children}$  (smallest at root).

**Max-heap:**  $\text{parent} \geq \text{children}$  (largest at root).

### 4.1 Array Representation

|          |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|
| Indices: | 0  | 1  | 2  | 3  | 4  | 5  |
| Values:  | 10 | 15 | 20 | 25 | 30 | 35 |

Parent of  $i = (i-1)/2$ . Left child =  $2i+1$ . Right child =  $2i+2$ .

### 4.2 Heap Operations

| Operation           | Time                    |
|---------------------|-------------------------|
| <code>insert</code> | $O(\log n)$ – bubble up |

|                            |                                                 |
|----------------------------|-------------------------------------------------|
| extractMin (or max)        | $O(\log n)$ – swap root with last, heapify down |
| getMin                     | $O(1)$                                          |
| heapify (build from array) | $O(n)$                                          |

```

class MinHeap {
 vector<int> heap;
 void heapifyUp(int i) {
 while (i > 0 && heap[i] < heap[(i-1)/2]) {
 swap(heap[i], heap[(i-1)/2]);
 i = (i-1)/2;
 }
 }
 void heapifyDown(int i) {
 int n = heap.size();
 while (true) {
 int left = 2*i+1, right = 2*i+2, smallest = i;
 if (left < n && heap[left] < heap[smallest]) smallest =
left;
 if (right < n && heap[right] < heap[smallest]) smallest =
right;
 if (smallest == i) break;
 swap(heap[i], heap[smallest]);
 i = smallest;
 }
 }
public:
 void push(int val) {
 heap.push_back(val);
 heapifyUp(heap.size()-1);
 }
 int pop() {
 if (heap.empty()) throw;
 int root = heap[0];
 heap[0] = heap.back();
 heap.pop_back();
 if (!heap.empty()) heapifyDown(0);
 return root;
 }
 int top() const { return heap[0]; }
};

```



## 4.3 Heap Sort

Build max-heap, repeatedly swap root with last element and heapify reduced heap.

```
void heapSort(vector<int>& arr) {
 int n = arr.size();
 // build heap using heapify
 for (int i = n/2-1; i >= 0; --i) heapify(arr, n, i);
 for (int i = n-1; i > 0; --i) {
 swap(arr[0], arr[i]);
 heapify(arr, i, 0);
 }
}
```

## 4.4 Priority Queue in C++

`priority_queue<int>` (max-heap), `priority_queue<int, vector<int>, greater<int>>` (min-heap).

# 5. Advanced Trees

## 5.1 Trie (Prefix Tree)

**What:** A tree where each node represents a character. Used for fast prefix-based searches (auto-complete, spell checking).

**Node structure:**

```
struct TrieNode {
 bool isEnd;
 TrieNode* children[26];
 TrieNode() : isEnd(false) { fill(begin(children), end(children),
 nullptr); }
};
```

**Operations:**

- `insert(word)` : traverse, create nodes as needed, mark last as end.
- `search(word)` : traverse; last node must have `isEnd=true`.

- `startsWith(prefix)` : returns true if prefix exists.

```

class Trie {
 TrieNode* root;
public:
 Trie() : root(new TrieNode()) {}

 void insert(string word) {
 TrieNode* node = root;
 for (char c : word) {
 int idx = c - 'a';
 if (!node->children[idx]) node->children[idx] = new
TrieNode();
 node = node->children[idx];
 }
 node->isEnd = true;
 }

 bool search(string word) {
 TrieNode* node = root;
 for (char c : word) {
 int idx = c - 'a';
 if (!node->children[idx]) return false;
 node = node->children[idx];
 }
 return node->isEnd;
 }

 bool startsWith(string prefix) {
 TrieNode* node = root;
 for (char c : prefix) {
 int idx = c - 'a';
 if (!node->children[idx]) return false;
 node = node->children[idx];
 }
 return true;
 }
};

```

**Time:**  $O(\text{len})$  per operation. **Space:**  $O(\text{total characters across words})$ .

**Real-life analogy:** A dictionary organised by prefixes – from "A" to "Z", each level refines the search.

## 5.2 Segment Tree

**What:** A binary tree used for answering range queries (e.g., sum, min, max) and point updates in  $O(\log n)$ .

**Construction:** Leaf nodes store array values; internal nodes store aggregated value of children.

**Range query:** Recursively traverse, combining results from relevant segments.

```

class SegmentTree {
 vector<int> tree;
 int n;
 void build(vector<int>& arr, int node, int l, int r) {
 if (l == r) tree[node] = arr[l];
 else {
 int mid = (l + r) / 2;
 build(arr, 2*node+1, l, mid);
 build(arr, 2*node+2, mid+1, r);
 tree[node] = tree[2*node+1] + tree[2*node+2];
 }
 }
 void update(int idx, int val, int node, int l, int r) {
 if (l == r) tree[node] = val;
 else {
 int mid = (l + r) / 2;
 if (idx <= mid) update(idx, val, 2*node+1, l, mid);
 else update(idx, val, 2*node+2, mid+1, r);
 tree[node] = tree[2*node+1] + tree[2*node+2];
 }
 }
 int query(int ql, int qr, int node, int l, int r) {
 if (qr < l || ql > r) return 0;
 if (ql <= l && r <= qr) return tree[node];
 int mid = (l + r) / 2;
 return query(ql, qr, 2*node+1, l, mid) + query(ql, qr, 2*node+2,
mid+1, r);
 }
public:
 SegmentTree(vector<int>& arr) {
 n = arr.size();
 tree.resize(4 * n);
 build(arr, 0, 0, n-1);
 }
 void update(int idx, int val) { update(idx, val, 0, 0, n-1); }

```

```
int query(int l, int r) { return query(l, r, 0, 0, n-1); }
};
```

**Lazy propagation:** Delays updates to range intervals (e.g., adding a constant to a range) – stores pending updates in nodes.

## 5.3 Fenwick Tree (Binary Indexed Tree)

**What:** Simpler than segment tree for prefix sums and point updates. Uses  $O(n)$  space,  $O(\log n)$  operations.

**Idea:** Each index  $i$  stores sum of a range  $(i - \text{LSB}(i) + 1) \dots i$ .

```
class FenwickTree {
 vector<int> bit;
 int n;
public:
 FenwickTree(int size) : n(size), bit(size+1, 0) {}
 void update(int idx, int delta) { // 1-based index
 while (idx <= n) {
 bit[idx] += delta;
 idx += idx & -idx;
 }
 }
 int query(int idx) { // prefix sum [1..idx]
 int sum = 0;
 while (idx > 0) {
 sum += bit[idx];
 idx -= idx & -idx;
 }
 return sum;
 }
 int rangeSum(int l, int r) { return query(r) - query(l-1); }
};
```

**Use cases:** Counting inversions, frequency arrays, prefix sums.

## 5.4 Disjoint Set Union (Union-Find)

**What:** Data structure for tracking a partition of elements into disjoint sets. Supports `find` (which set does element belong to?) and `union` (merge two sets).

**Optimisations:** Path compression + union by rank / size.

```
class DSU {
 vector<int> parent, rank;
public:
 DSU(int n) {
 parent.resize(n);
 rank.resize(n, 0);
 for (int i = 0; i < n; ++i) parent[i] = i;
 }
 int find(int x) {
 if (parent[x] != x) parent[x] = find(parent[x]); // path
compression
 return parent[x];
 }
 void unite(int x, int y) {
 int rx = find(x), ry = find(y);
 if (rx == ry) return;
 if (rank[rx] < rank[ry]) parent[rx] = ry;
 else if (rank[rx] > rank[ry]) parent[ry] = rx;
 else { parent[ry] = rx; rank[rx]++; }
 }
 bool connected(int x, int y) { return find(x) == find(y); }
};
```

**Time:** Almost  $O(1)$  (inverse Ackermann).

### Applications:

- Kruskal's algorithm (minimum spanning tree)
- Cycle detection in undirected graphs
- Dynamic connectivity

**Real-life analogy:** Social network – each person has a representative. When two people become friends, their groups merge.

## 6. Summary Table

| Tree Type          | Key Operation             | Time (avg)       | Use Case                          |
|--------------------|---------------------------|------------------|-----------------------------------|
| Binary Tree        | Traversal                 | $O(n)$           | Expression parsing, DFS           |
| BST                | Search, Insert, Delete    | $O(\log n)$      | Ordered data, dictionaries        |
| AVL /<br>Red-Black | Balanced ops              | $O(\log n)$      | Maps, sets (worst-case guarantee) |
| Heap               | GetMin/ExtractMin         | $O(1)/O(\log n)$ | Priority queue, heapsort          |
| Trie               | Prefix search             | $O(\text{len})$  | Auto-complete, dictionary         |
| Segment Tree       | Range query, point update | $O(\log n)$      | RMQ, range sums                   |
| Fenwick Tree       | Prefix sum, point update  | $O(\log n)$      | Frequency accumulations           |
| Disjoint Set       | Union, Find               | $\alpha(n)$      | Connectivity, cycle detection     |

The next chapter will cover graphs (representations, traversals, shortest paths, and minimum spanning trees).



# Chapter 11: Graphs

Graphs model relationships between objects. This chapter covers graph representations, traversals (DFS, BFS), shortest path algorithms, minimum spanning trees, topological sorting, cycle detection, strongly connected components, network flow basics, and classic graph problems. Each section includes definitions, complexity, C++ implementations, and real-life analogies.

## 1. Graph Representations

A graph  $G = (V, E)$  consists of vertices (nodes) and edges (connections). Edges can be directed or undirected, weighted or unweighted.

### 1.1 Adjacency Matrix

**What:** A  $V \times V$  matrix where `matrix[u][v] = 1` (or weight) if edge exists from  $u$  to  $v$ .

- **Space:**  $O(V^2)$ . **Time for edge lookup:**  $O(1)$ .
- **When to use:** Dense graphs, or when  $O(1)$  edge existence is critical.

```
vector<vector<int>> adjMat(V, vector<int>(V, 0));
adjMat[u][v] = 1; // add edge u→v
if (adjMat[u][v]) {...} // check edge
```

### 1.2 Adjacency List

**What:** Array of  $V$  lists (or vectors). List at index  $u$  stores neighbours (and possibly weights).

- **Space:**  $O(V + E)$ . **Time for edge lookup:**  $O(\text{degree}(u))$ .
- **When to use:** Sparse graphs, most practical applications.

```
vector<vector<int>> adjList(V);
adjList[u].push_back(v); // unweighted
// weighted
vector<vector<pair<int,int>>> adjWeighted(V);
adjWeighted[u].push_back({v, w});
```

### 1.3 Edge List

**What:** A single list of all edges (u, v, weight). Used in algorithms like Kruskal.

```
struct Edge { int u, v, w; };
vector<Edge> edges;
edges.push_back({u, v, w});
```

**Real-life analogy:**

- Adjacency matrix: A city-to-city road table (rows and columns).
- Adjacency list: Each city has a list of direct flight destinations.
- Edge list: A plain log of all flights.

## 2. Graph Traversals

### 2.1 Depth First Search (DFS)

**What:** Explores as far as possible along each branch before backtracking.

**Uses:** Cycle detection, topological sorting, connected components, solving puzzles.

**Recursive:**

```
void dfs(int u, List<Boolean> visited, List<List<Integer>> adj) {
 visited.set(u, true);
 System.out.print(u + " ");
 for (int v : adj.get(u)) {
 if (!visited.get(v)) dfs(v, visited, adj);
 }
}
```

**Iterative (using stack):**

```

void dfsIterative(int start, List<List<Integer>> adj) {
 List<Boolean> visited = new ArrayList<>
(Collections.nCopies(adj.size(), false));
 Deque<Integer> st = new ArrayDeque<>();
 st.push(start);
 while (!st.isEmpty()) {
 int u = st.pop();
 if (visited.get(u)) continue;
 visited.set(u, true);
 System.out.print(u + " ");
 for (int v : adj.get(u)) {
 if (!visited.get(v)) st.push(v);
 }
 }
}

```

## 2.2 Breadth First Search (BFS)

**What:** Explores level by level, using a queue.

**Uses:** Shortest path in unweighted graphs, word ladder, web crawling.

```

void bfs(int start, List<List<Integer>> adj) {
 List<Boolean> visited = new ArrayList<>
(Collections.nCopies(adj.size(), false));
 Queue<Integer> q = new LinkedList<>();
 visited.set(start, true);
 q.add(start);
 while (!q.isEmpty()) {
 int u = q.poll();
 System.out.print(u + " ");
 for (int v : adj.get(u)) {
 if (!visited.get(v)) {
 visited.set(v, true);
 q.add(v);
 }
 }
 }
}

```



**Real-life analogy:** BFS is like ripples in water after throwing a stone; DFS is like exploring a maze by always turning left.

## 3. Shortest Path Algorithms

### 3.1 Unweighted Graph: BFS

BFS finds shortest path (minimum number of edges) from source to all vertices.

```

List<Integer> bfsShortestPath(int start, List<List<Integer>> adj) {
 int V = adj.size();
 List<Integer> dist = new ArrayList<>(Collections.nCopies(V, -1));
 Queue<Integer> q = new LinkedList<>();
 dist.set(start, 0);
 q.add(start);
 while (!q.isEmpty()) {
 int u = q.poll();
 for (int v : adj.get(u)) {
 if (dist.get(v) == -1) {
 dist.set(v, dist.get(u) + 1);
 q.add(v);
 }
 }
 }
 return dist;
}

```

## 3.2 Weighted Graph (Non-negative): Dijkstra's Algorithm

**What:** Finds shortest paths from source using a min-heap (priority queue). Greedy.

**Time:**  $O((V+E) \log V)$  with binary heap.

```

vector<int> dijkstra(int src, vector<vector<pair<int,int>>>& adj) {
 int V = adj.size();
 vector<int> dist(V, INT_MAX);
 dist[src] = 0;
 using P = pair<int,int>; // (distance, node)
 priority_queue<P, vector<P>, greater<P>> pq;
 pq.push({0, src});
 while (!pq.empty()) {
 auto [d, u] = pq.top(); pq.pop();
 if (d > dist[u]) continue;
 for (auto [v, w] : adj[u]) {
 if (dist[v] > dist[u] + w) {
 dist[v] = dist[u] + w;
 pq.push({dist[v], v});
 }
 }
 }
 return dist;
}

```

**Real-life analogy:** GPS navigation with non-negative road distances – always pick the closest unvisited city.

### 3.3 Weighted Graph (Negative Edges): Bellman-Ford

**What:** Relaxes all edges  $V-1$  times. Detects negative cycles.

**Time:**  $O(V \cdot E)$ .

```

vector<int> bellmanFord(int src, int V, vector<tuple<int,int,int>>&
edges) {
 vector<int> dist(V, INT_MAX);
 dist[src] = 0;
 for (int i = 0; i < V-1; ++i) {
 for (auto [u, v, w] : edges) {
 if (dist[u] != INT_MAX && dist[v] > dist[u] + w)
 dist[v] = dist[u] + w;
 }
 }
 // Check negative cycles
 for (auto [u, v, w] : edges) {
 if (dist[u] != INT_MAX && dist[v] > dist[u] + w)
 return {}; // negative cycle
 }
 return dist;
}

```

### 3.4 All-Pairs Shortest Paths: Floyd-Warshall

**What:** Dynamic programming – consider each vertex as intermediate.

**Time:**  $O(V^3)$ . **Space:**  $O(V^2)$ .

```

vector<vector<int>> floydWarshall(vector<vector<int>>& graph) {
 int V = graph.size();
 vector<vector<int>> dist = graph;
 for (int k = 0; k < V; ++k)
 for (int i = 0; i < V; ++i)
 for (int j = 0; j < V; ++j)
 if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX)
 dist[i][j] = min(dist[i][j], dist[i][k] + dist[k]
[j]);
 return dist;
}

```

## 4. Minimum Spanning Tree (MST)

An MST connects all vertices with minimal total edge weight. No cycles.

## 4.1 Prim's Algorithm

Grows a tree from a start vertex, always adding the cheapest edge from the tree to outside.

**Time:**  $O((V+E) \log V)$  with min-heap.

```
int primMST(vector<vector<pair<int,int>>>& adj) {
 int V = adj.size(), total = 0;
 vector<bool> inMST(V, false);
 using P = pair<int,int>; // (weight, vertex)
 priority_queue<P, vector<P>, greater<P>> pq;
 pq.push({0, 0}); // start from vertex 0
 while (!pq.empty()) {
 auto [w, u] = pq.top(); pq.pop();
 if (inMST[u]) continue;
 inMST[u] = true;
 total += w;
 for (auto [v, weight] : adj[u])
 if (!inMST[v]) pq.push({weight, v});
 }
 return total;
}
```

**Real-life analogy:** Laying fibre optic cable connecting all towns with minimum cost – always extend from current network to the cheapest unconnected town.

## 4.2 Kruskal's Algorithm

Sorts edges by weight, adds edges that do not create cycles (using Union-Find).

**Time:**  $O(E \log E)$  (sorting) +  $O(E \alpha(V))$ .



```

struct DSU {
 vector<int> parent, rank;
 DSU(int n) {
 parent.resize(n); rank.resize(n,0);
 for(int i=0;i<n;++i) parent[i]=i;
 }
 int find(int x) { return parent[x]==x ? x :
parent[x]=find(parent[x]); }
 bool unite(int x, int y) {
 int rx=find(x), ry=find(y);
 if(rx==ry) return false;
 if(rank[rx]<rank[ry]) parent[rx]=ry;
 else if(rank[rx]>rank[ry]) parent[ry]=rx;
 else { parent[ry]=rx; rank[rx]++; }
 return true;
 }
};

int kruskalMST(vector<tuple<int,int,int>>& edges, int V) {
 sort(edges.begin(), edges.end()); // by weight
 DSU dsu(V);
 int total = 0, edgesUsed = 0;
 for (auto [w, u, v] : edges) {
 if (dsu.unite(u, v)) {
 total += w;
 edgesUsed++;
 if (edgesUsed == V-1) break;
 }
 }
 return total;
}

```

## 5. Topological Sorting

**Definition:** Linear ordering of vertices in a DAG (directed acyclic graph) such that for every edge ( $u \rightarrow v$ ),  $u$  appears before  $v$ .

**Real-life analogy:** Course prerequisites – you must take prerequisites before advanced courses.

## 5.1 Kahn's Algorithm (BFS)

Compute indegree of each node. Push zero indegree nodes to queue, remove their outgoing edges, repeat.

```
vector<int> topologicalSortKahn(vector<vector<int>>& adj) {
 int V = adj.size();
 vector<int> indeg(V, 0), result;
 for (int u=0; u<V; ++u)
 for (int v : adj[u]) indeg[v]++;
 queue<int> q;
 for (int i=0; i<V; ++i) if (indeg[i]==0) q.push(i);
 while (!q.empty()) {
 int u = q.front(); q.pop();
 result.push_back(u);
 for (int v : adj[u])
 if (--indeg[v] == 0) q.push(v);
 }
 return result.size() == V ? result : vector<int>{}; // cycle if size
< V
}
```

## 5.2 DFS-based

Perform DFS; push node to result after processing all neighbours (postorder). Reverse the result.

```

void dfsTopo(int u, vector<bool>& visited, stack<int>& st,
vector<vector<int>>& adj) {
 visited[u] = true;
 for (int v : adj[u])
 if (!visited[v]) dfsTopo(v, visited, st, adj);
 st.push(u);
}

vector<int> topologicalSortDFS(vector<vector<int>>& adj) {
 int V = adj.size();
 vector<bool> visited(V, false);
 stack<int> st;
 for (int i=0; i<V; ++i)
 if (!visited[i]) dfsTopo(i, visited, st, adj);
 vector<int> result;
 while (!st.empty()) { result.push_back(st.top()); st.pop(); }
 return result;
}

```

## 6. Cycle Detection

### 6.1 Undirected Graph

**Using DFS:** If a neighbour is already visited and is not the parent, cycle exists.

```

bool hasCycleUndirectedDFS(int u, int parent, vector<bool>& visited,
vector<vector<int>>& adj) {
 visited[u] = true;
 for (int v : adj[u]) {
 if (!visited[v]) {
 if (hasCycleUndirectedDFS(v, u, visited, adj)) return true;
 } else if (v != parent) return true;
 }
 return false;
}

```

**Using Union-Find:** If two vertices of an edge already belong to the same set, cycle exists.

## 6.2 Directed Graph

Using DFS with recursion stack: Maintain a separate `inStack` array.

```
bool hasCycleDirectedDFS(int u, vector<bool>& visited, vector<bool>&
inStack, vector<vector<int>>& adj) {
 visited[u] = true;
 inStack[u] = true;
 for (int v : adj[u]) {
 if (!visited[v]) {
 if (hasCycleDirectedDFS(v, visited, inStack, adj)) return
true;
 } else if (inStack[v]) return true;
 }
 inStack[u] = false;
 return false;
}
```

## 7. Strongly Connected Components (SCC)

A strongly connected component is a maximal set of vertices where every pair has a path in both directions.

### 7.1 Kosaraju's Algorithm

1. Run DFS on original graph to get finishing order (postorder).
2. Reverse the graph.
3. Run DFS on reversed graph in order of decreasing finishing times – each DFS tree gives an SCC.

```

void dfsOrder(int u, vector<bool>& visited, stack<int>& st,
vector<vector<int>>& adj) {
 visited[u] = true;
 for (int v : adj[u]) if (!visited[v]) dfsOrder(v, visited, st, adj);
 st.push(u);
}

void dfsSCC(int u, vector<bool>& visited, vector<int>& comp,
vector<vector<int>>& revAdj) {
 visited[u] = true;
 comp.push_back(u);
 for (int v : revAdj[u]) if (!visited[v]) dfsSCC(v, visited, comp,
revAdj);
}

vector<vector<int>> kosaraju(vector<vector<int>>& adj) {
 int V = adj.size();
 vector<bool> visited(V, false);
 stack<int> st;
 for (int i=0; i<V; ++i) if (!visited[i]) dfsOrder(i, visited, st,
adj);
 // build reverse graph
 vector<vector<int>> revAdj(V);
 for (int u=0; u<V; ++u) for (int v : adj[u]) revAdj[v].push_back(u);
 fill(visited.begin(), visited.end(), false);
 vector<vector<int>> sccs;
 while (!st.empty()) {
 int u = st.top(); st.pop();
 if (!visited[u]) {
 vector<int> comp;
 dfsSCC(u, visited, comp, revAdj);
 sccs.push_back(comp);
 }
 }
 return sccs;
}

```

## 7.2 Tarjan' s Algorithm

Single DFS with low-link values. More efficient (one pass).

## 8. Network Flow (Basic)

**Max flow / min cut:** Find maximum flow from source to sink in a weighted directed graph.

### Ford-Fulkerson (Edmonds-Karp)

Use BFS to find augmenting paths (Edmonds-Karp). Time  $O(V * E^2)$ .

```

int bfsFlow(int s, int t, vector<int>& parent, vector<vector<int>>& cap)
{
 fill(parent.begin(), parent.end(), -1);
 parent[s] = s;
 queue<pair<int,int>> q;
 q.push({s, INT_MAX});
 while (!q.empty()) {
 auto [u, flow] = q.front(); q.pop();
 for (int v = 0; v < cap.size(); ++v) {
 if (parent[v] == -1 && cap[u][v] > 0) {
 parent[v] = u;
 int newFlow = min(flow, cap[u][v]);
 if (v == t) return newFlow;
 q.push({v, newFlow});
 }
 }
 }
 return 0;
}

int maxFlow(int s, int t, vector<vector<int>>& cap) {
 int flow = 0, newFlow;
 vector<int> parent(cap.size());
 while ((newFlow = bfsFlow(s, t, parent, cap)) > 0) {
 flow += newFlow;
 int v = t;
 while (v != s) {
 int u = parent[v];
 cap[u][v] -= newFlow;
 cap[v][u] += newFlow;
 v = u;
 }
 }
 return flow;
}

```

**Real-life analogy:** Water flowing through pipes; maximum flow is the total water that can be sent from source to sink.

## 9. Important Graph Problems

### 9.1 Clone a Graph

**Problem:** Deep copy an undirected graph with arbitrary structure.

**Approach:** BFS/DFS using a hash map (original → copy node).

```
Node* cloneGraph(Node* node) {
 if (!node) return nullptr;
 unordered_map<Node*, Node*> m;
 queue<Node*> q;
 m[node] = new Node(node->val);
 q.push(node);
 while (!q.empty()) {
 Node* cur = q.front(); q.pop();
 for (Node* nei : cur->neighbors) {
 if (m.find(nei) == m.end()) {
 m[nei] = new Node(nei->val);
 q.push(nei);
 }
 m[cur]->neighbors.push_back(m[nei]);
 }
 }
 return m[node];
}
```

### 9.2 Word Ladder

**Problem:** Shortest transformation sequence (word → word) changing one letter at a time, using a dictionary.

**Approach:** BFS from startWord.



```

int ladderLength(string beginWord, string endWord, vector<string>&
wordList) {
 unordered_set<string> dict(wordList.begin(), wordList.end());
 if (dict.find(endWord) == dict.end()) return 0;
 queue<pair<string,int>> q;
 q.push({beginWord, 1});
 while (!q.empty()) {
 auto [cur, len] = q.front(); q.pop();
 for (int i=0; i<cur.size(); ++i) {
 string temp = cur;
 for (char c='a'; c<='z'; ++c) {
 temp[i] = c;
 if (temp == cur) continue;
 if (temp == endWord) return len+1;
 if (dict.find(temp) != dict.end()) {
 q.push({temp, len+1});
 dict.erase(temp);
 }
 }
 }
 }
 return 0;
}

```

## 9.3 Course Schedule (Topological Sort)

**Problem:** Check if all courses can be finished given prerequisites.

**Approach:** Detect cycle in directed graph (DFS or Kahn). If DAG, possible.

```

bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
 vector<vector<int>> adj(numCourses);
 vector<int> indeg(numCourses, 0);
 for (auto& p : prerequisites) {
 adj[p[1]].push_back(p[0]);
 indeg[p[0]]++;
 }
 queue<int> q;
 for (int i=0; i<numCourses; ++i) if (indeg[i]==0) q.push(i);
 int processed = 0;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 processed++;
 for (int v : adj[u])
 if (--indeg[v] == 0) q.push(v);
 }
 return processed == numCourses;
}

```

## 9.4 Number of Islands

**Problem:** Count connected components of '1's in a binary grid.

**Approach:** DFS/BFS to mark visited.

```

void dfsIsland(vector<vector<char>>& grid, int i, int j) {
 if (i<0 || i>=grid.size() || j<0 || j>=grid[0].size() || grid[i]
[j]!='1') return;
 grid[i][j] = '0';
 dfsIsland(grid, i+1, j); dfsIsland(grid, i-1, j);
 dfsIsland(grid, i, j+1); dfsIsland(grid, i, j-1);
}

int numIslands(vector<vector<char>>& grid) {
 int count = 0;
 for (int i=0; i<grid.size(); ++i)
 for (int j=0; j<grid[0].size(); ++j)
 if (grid[i][j] == '1') { dfsIsland(grid, i, j); count++; }
 return count;
}

```

## 9.5 Bipartite Graph Checking

**Problem:** Check if graph is bipartite (2-colorable).

**Approach:** BFS / DFS assigning colours.

```

bool isBipartite(vector<vector<int>>& graph) {
 int V = graph.size();
 vector<int> color(V, -1);
 for (int i=0; i<V; ++i) {
 if (color[i] != -1) continue;
 queue<int> q;
 q.push(i); color[i] = 0;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (int v : graph[u]) {
 if (color[v] == -1) {
 color[v] = color[u] ^ 1;
 q.push(v);
 } else if (color[v] == color[u]) return false;
 }
 }
 }
 return true;
}

```

## 10. Summary Table

| Algorithm      | Graph Type                  | Time Complexity   | Key Data Structure |
|----------------|-----------------------------|-------------------|--------------------|
| DFS/BFS        | Any                         | $O(V+E)$          | Stack / Queue      |
| Dijkstra       | Weighted (non-negative)     | $O((V+E) \log V)$ | Min-heap           |
| Bellman-Ford   | Weighted (negative allowed) | $O(V \cdot E)$    | Edge list          |
| Floyd-Warshall | Weighted (all-pairs)        | $O(V^3)$          | 2D array           |
| Prim (MST)     | Weighted undirected         | $O((V+E) \log V)$ | Min-heap           |
| Kruskal (MST)  | Weighted undirected         | $O(E \log E)$     | DSU + sorting      |

|                  |                   |                  |                            |
|------------------|-------------------|------------------|----------------------------|
| Topological Sort | DAG               | $O(V+E)$         | Queue (Kahn) / Stack (DFS) |
| Kosaraju (SCC)   | Directed          | $O(V+E)$         | DFS on reverse graph       |
| Edmonds-Karp     | Weighted directed | $O(V \cdot E^2)$ | BFS + residual graph       |

The next chapter will cover advanced topics: dynamic programming (memoisation, tabulation, classic DP problems).

# Chapter 12: Dynamic Programming (DP)

Dynamic Programming is a method for solving complex problems by breaking them into smaller overlapping subproblems and storing the results of each subproblem to avoid redundant computation. This chapter covers DP fundamentals, common mistakes, pattern recognition, when NOT to use DP, classic DP problems, subsequence and substring problems, knapsack, DP on strings, trees, bitmask DP, and Kadane's algorithm as a DP view.

## 1. DP Fundamentals

### 1.1 Key Concepts

- **Overlapping Subproblems:** The problem can be broken into subproblems that are reused multiple times. Example: Fibonacci:  $F(n) = F(n-1) + F(n-2)$  recomputes the same values.
- **Optimal Substructure:** An optimal solution can be constructed from optimal solutions of its subproblems. Example: Shortest path from A to C passing through B – the path  $A \rightarrow B$  and  $B \rightarrow C$  must be optimal.
- **Memoization (Top-Down):** Recursive approach with caching. Solve the problem recursively and store results before returning.
- **Tabulation (Bottom-Up):** Iterative approach. Build a table from the smallest subproblem upwards.

### 1.2 Real-Life Analogy

- **Memoization:** Taking notes during an exam. When you solve a problem, you write down the answer so that if it appears again in a different part, you just look it up.
- **Tabulation:** Building a table of times tables row by row; each new row uses previously computed rows.

### 1.3 Fibonacci Example

**Memoization (Top-Down):**

```

private static long[] memo = new long[100];

static {
 Arrays.fill(memo, -1);
}

public static long fibMemo(int n) {
 if (n <= 1) return n;
 if (memo[n] != -1) return memo[n];
 return memo[n] = fibMemo(n-1) + fibMemo(n-2);
}

```

### Tabulation (Bottom-Up):

```

public static long fibTab(int n) {
 if (n <= 1) return n;
 long[] dp = new long[n+1];
 dp[0] = 0;
 dp[1] = 1;
 for (int i = 2; i <= n; ++i)
 dp[i] = dp[i-1] + dp[i-2];
 return dp[n];
}

```

### Space-Optimised:

```

public static long fibOpt(int n) {
 if (n <= 1) return n;
 long a = 0, b = 1, c;
 for (int i = 2; i <= n; ++i) {
 c = a + b;
 a = b;
 b = c;
 }
 return b;
}

```

## 2. Common DP Mistakes

Even with a correct recurrence, DP solutions often fail due to implementation pitfalls. Recognising these mistakes improves debugging and interview performance.

| Mistake                                                   | Description                                                                                | Example                                                                                          | Correct Approach                                                                       |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Incorrect initialisation</b>                           | Base cases or DP table initial values are wrong.                                           | In LCS, initialising <code>dp[0][j] = 1</code> instead of 0.                                     | <code>dp[0][j] = 0</code><br>(empty prefix has no common subsequence).                 |
| <b>Out-of-order iteration</b>                             | In tabulation, traversing inner loop in wrong direction for 1D knapsack.                   | <code>for (int w = 0; w &lt;= W; ++w)</code> for 0/1 knapsack – reuses same item multiple times. | Iterate <code>w</code> from <code>W</code> down to <code>weight[i]</code> .            |
| <b>Off-by-one indices</b>                                 | Using <code>dp[i]</code> when <code>i</code> should represent length or index incorrectly. | <code>dp[n]</code> accessed when <code>dp</code> size is <code>n</code> .                        | Use <code>dp[n+1]</code> when state is array length.                                   |
| <b>Missing handling of impossible states</b>              | Using <code>INT_MAX</code> without checking overflow.                                      | <code>dp[i] = min(dp[i-1], dp[i-2]) + cost</code> without guarding <code>INT_MAX</code> .        | Use <code>INT_MAX/2</code> for additions, check if <code>dp[i-1] != INT_MAX/2</code> . |
| <b>Using recursive memoization without clearing cache</b> | Global memo array reused across multiple test cases without resetting.                     | <code>memo</code> retains values from previous call.                                             | Reset or use local memo per call.                                                      |
| <b>Forgetting to store computed result</b>                | Recursive function returns value but does not memoise before returning.                    | <code>return fib(n-1) + fib(n-2)</code> without storing in <code>memo[n]</code> .                | Store before returning.                                                                |



|                                         |                                                                                         |                                                                          |                                |
|-----------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------|--------------------------------|
| <b>Using the wrong DP dimension</b>     | Too few dimensions lose necessary state (e.g., knapsack with two constraints needs 2D). | <code>dp[W]</code> for knapsack with both weight and volume constraints. | Use <code>dp[W][V]</code> .    |
| <b>Premature pruning of subproblems</b> | Skipping some states due to incorrect dominance assumption.                             | In subset sum, assuming larger sum is always better.                     | Evaluate all reachable states. |

## Example: Off-by-One Error in Climbing Stairs

Incorrect:

```
public static int climbStairs(int n) {
 if (n <= 2) return n;
 int[] dp = new int[n+1];
 dp[1] = 1;
 dp[2] = 2;
 for (int i = 3; i <= n; ++i) dp[i] = dp[i-1] + dp[i-2];
 return dp[n];
}
```

Correct:

```
public static int climbStairsCorrect(int n) {
 if (n <= 2) return n;
 int[] dp = new int[n+1];
 dp[1] = 1;
 dp[2] = 2;
 for (int i = 3; i <= n; ++i) dp[i] = dp[i-1] + dp[i-2];
 return dp[n];
}
```

## 3. DP Pattern Recognition

Identifying that a problem can be solved with DP is often the hardest step. The following patterns and questions help recognise DP problems during interviews or coding contests.

## 3.1 Key Questions to Ask

- **Can the problem be broken into smaller subproblems?** Does solving for input size `n` depend on solutions for smaller inputs (e.g., `n-1`, `n/2`)?
- **Are there overlapping subproblems?** Would a naive recursive solution recompute the same inputs many times?
- **Does the problem ask for optimisation?** (maximum, minimum, longest, shortest, number of ways, true/false).
- **Does the problem involve sequences, grids, or subsets?** Common DP domains.
- **Is the problem a variant of classic DP?** (LCS, knapsack, edit distance, coin change, etc.)

## 3.2 Quick Pattern Cheat Sheet

| Problem Characteristics                     | Likely DP Category                      | State Definition Example                                                                      |
|---------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------|
| Sequence of decisions (take/skip, buy/sell) | 1D DP                                   | <code>dp[i]</code> = best answer for first <code>i</code> elements                            |
| Grid path (right/down moves)                | 2D grid DP                              | <code>dp[i][j]</code> = answer for cell <code>(i,j)</code>                                    |
| Two strings comparison                      | String DP (LCS, edit distance)          | <code>dp[i][j]</code> = answer for prefixes <code>s1[0..i-1]</code> , <code>s2[0..j-1]</code> |
| Subset sum / knapsack                       | 0/1 Knapsack                            | <code>dp[w]</code> = max value for capacity <code>w</code>                                    |
| Counting ways (without order)               | Combination DP                          | <code>dp[x]</code> = number of ways to achieve amount <code>x</code>                          |
| Minimising partitions                       | Partition DP (palindrome, matrix chain) | <code>dp[i]</code> = min cost for prefix up to <code>i</code>                                 |
| Tree with parent-child dependencies         | Tree DP                                 | <code>dp[u][0/1]</code> = answer for subtree (node selected or not)                           |
| Small constraints ( $n \leq 20$ )           | Bitmask DP                              | <code>dp[mask][i]</code> = answer for visited set <code>mask</code> ending at <code>i</code>  |

|                           |                  |                                                               |
|---------------------------|------------------|---------------------------------------------------------------|
| Probability / expectation | Probabilistic DP | <code>dp[i]</code> = expected value from state <code>i</code> |
|---------------------------|------------------|---------------------------------------------------------------|

### 3.3 Example Recognition Walkthrough

**Problem:** Given an array of integers, find the length of the longest increasing subsequence.

- **Question:** Can we solve for prefix `i` using smaller prefixes? Yes: LIS ending at `i` depends on all previous `j < i` with `nums[j] < nums[i]`.
- **Overlapping subproblems?** Different `i` may consider the same `j` multiple times – yes.
- **Optimisation?** Yes, longest.
- **Classic pattern?** LIS – known DP with  $O(n^2)$  or patience sorting.

**Result:** Use DP.

### 3.4 Transition from Brute-Force to DP

1. Write a recursive backtracking solution that explores all possibilities.
2. Identify parameters that change during recursion.
3. Use those parameters as state dimensions.
4. Memoise results on those parameters.
5. (Optional) Convert to iterative tabulation.

## 4. When NOT to Use DP

DP is powerful, but it is not always the best or necessary tool. Recognising when other techniques are simpler or more efficient saves time and complexity.

### 4.1 Greedy Works Better

If a problem exhibits the **greedy choice property** (making the locally optimal choice leads to a global optimum), greedy is often simpler and faster ( $O(n)$  or  $O(n \log n)$  vs DP's potential  $O(n^2)$  or higher).

**Examples:**

- **Coin change** (canonical coin systems like US coins): Greedy works. DP only needed for arbitrary denominations.
- **Activity selection** (maximise number of non-overlapping intervals): Greedy by earliest finish time.

- **Minimum number of jumps to end** (when each jump length is limited): Greedy works in  $O(n)$ .
- **Fractional knapsack**: Greedy by value/weight ratio. 0/1 knapsack requires DP.

**How to recognise:** Greedy works when selecting an item does not restrict future choices in a way that requires considering all subsets. Often the problem specifies “choose as many as possible” or “maximise” with no dependency on previously chosen items’ exact sum.

## 4.2 Sliding Window Is Sufficient

If the problem asks for a **contiguous subarray or substring** optimisation (length, sum, maximum) and the constraint involves a monotonic condition (e.g., at most  $K$  distinct characters,  $\text{sum} \leq K$ ), sliding window gives  $O(n)$  time and  $O(1)$  space – much better than DP.

**Examples:**

- **Longest substring with at most  $K$  distinct characters**: Sliding window.
- **Minimum size subarray sum**: Sliding window.
- **Maximum sum subarray of fixed size  $k$** : Sliding window (or prefix sums).
- **Fruit into baskets** (max two types): Sliding window.

**How to recognise:** “Contiguous” , “subarray” , “substring” with a constraint that can be maintained by expanding/shrinking a window. DP would over-complicate (e.g.,  $O(n^2)$  unnecessary).

## 4.3 Binary Search Optimization Is Possible

For problems that ask for **minimum of maximum** or **maximum of minimum** (or similar monotonic predicates), binary search on the answer combined with a feasibility check (often  $O(n)$  or  $O(n \log n)$ ) can be simpler than DP.

**Examples:**

- **Split array largest sum**: Minimise the largest sum of  $m$  subarrays. Binary search + greedy check.
- **Koko eating bananas**: Find minimum speed to eat all bananas within  $H$  hours. Binary search + simulation.
- **Capacity to ship packages within  $D$  days**: Binary search + greedy.
- **Median of two sorted arrays**: Binary search on smaller array.

**How to recognise:** The answer is a single value, and you can test in  $O(n)$  whether a candidate answer is feasible. The function `isFeasible(x)` is monotonic (false for small  $x$ , true for larger  $x$ ). DP would likely be overkill or too slow.

## 4.4 Mathematical or Combinatorial Formula Exists

Some problems have a closed-form solution ( $O(1)$ ) or simple combinatorial formula, making DP unnecessary.

**Examples:**

- **Number of ways to reach the top of  $n$  steps** (with 1 or 2 steps) → Fibonacci (can be DP or Binet's formula).
- **Unique paths in grid** – closed form using factorials:  $C(m+n-2, m-1)$ .
- **$n$ th Catalan number** – formula exists, but DP is still common for understanding.

**How to recognise:** Problem asks for a pure count or specific value that matches a known sequence.

## 4.5 Important Caveat

DP may still be **correct** in all these cases, but it often leads to higher time/space complexity and more code. The table below summarises the decision:

| Technique               | When to Use                                            | Typical Complexity      | DP Alternative Complexity                         |
|-------------------------|--------------------------------------------------------|-------------------------|---------------------------------------------------|
| Greedy                  | Problem has greedy choice property                     | $O(n)$ or $O(n \log n)$ | $O(n^2)$ or higher                                |
| Sliding Window          | Contiguous subarray with monotonic condition           | $O(n)$                  | $O(n^2)$ (e.g., longest palindromic substring DP) |
| Binary Search on Answer | Minimise max / maximise min with monotonic feasibility | $O(n \log M)$           | Often $O(n^2)$ or exponential                     |
| Math Formula            | Direct closed-form exists                              | $O(1)$                  | $O(n)$ or $O(n^2)$                                |

**General rule:** If you suspect a greedy or sliding window solution, test it on small examples. If it holds, skip DP. If you are asked for “all possible” results, DP (or backtracking) is likely

needed.

## 5. Classic DP Problems

### 5.1 Climbing Stairs

**Problem:** Count ways to reach the top of  $n$  steps by taking 1 or 2 steps at a time.

**DP Relation:**  $dp[i] = dp[i-1] + dp[i-2]$  with  $dp[0]=1$ ,  $dp[1]=1$ .

```
int climbStairs(int n) {
 if (n <= 1) return 1;
 int a = 1, b = 1, c;
 for (int i = 2; i <= n; ++i) {
 c = a + b;
 a = b;
 b = c;
 }
 return b;
}
```

### 5.2 House Robber (1D DP)

**Problem:** Maximise sum of non-adjacent elements.

**DP Relation:**  $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$ .

```
int rob(vector<int>& nums) {
 int n = nums.size();
 if (n == 0) return 0;
 if (n == 1) return nums[0];
 int prev2 = nums[0], prev1 = max(nums[0], nums[1]);
 for (int i = 2; i < n; ++i) {
 int curr = max(prev1, prev2 + nums[i]);
 prev2 = prev1;
 prev1 = curr;
 }
 return prev1;
}
```

## 5.3 Minimum Path Sum (Grid)

**Problem:** Find path from top-left to bottom-right minimising sum (only right/down moves).

**DP:**  $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$ .

```
int minPathSum(vector<vector<int>>& grid) {
 int m = grid.size(), n = grid[0].size();
 vector<vector<int>> dp(m, vector<int>(n));
 dp[0][0] = grid[0][0];
 for (int i = 1; i < m; ++i) dp[i][0] = dp[i-1][0] + grid[i][0];
 for (int j = 1; j < n; ++j) dp[0][j] = dp[0][j-1] + grid[0][j];
 for (int i = 1; i < m; ++i)
 for (int j = 1; j < n; ++j)
 dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]);
 return dp[m-1][n-1];
}
```

## 5.4 Unique Paths (Grid)

**Problem:** Number of distinct paths from top-left to bottom-right (right/down moves).

**DP:**  $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ .

```
int uniquePaths(int m, int n) {
 vector<int> dp(n, 1);
 for (int i = 1; i < m; ++i)
 for (int j = 1; j < n; ++j)
 dp[j] += dp[j-1];
 return dp[n-1];
}
```

# 6. Subsequence and Substring Problems

## 6.1 Longest Common Subsequence (LCS)

**Problem:** Maximum length of common subsequence (not necessarily contiguous).

**DP Relation:**

```

dp[i][j] = dp[i-1][j-1] + 1 if s1[i-1] == s2[j-1]
 = max(dp[i-1][j], dp[i][j-1]) otherwise

```

```

int longestCommonSubsequence(string s1, string s2) {
 int m = s1.size(), n = s2.size();
 vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
 for (int i = 1; i <= m; ++i)
 for (int j = 1; j <= n; ++j)
 if (s1[i-1] == s2[j-1])
 dp[i][j] = dp[i-1][j-1] + 1;
 else
 dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
 return dp[m][n];
}

```

## 6.2 Longest Common Substring

**DP:**  $dp[i][j] = dp[i-1][j-1] + 1$  if match, else 0. Track maximum.

## 6.3 Longest Increasing Subsequence (LIS)

**Problem:** Length of longest increasing subsequence.

**DP  $O(n^2)$ :**

```

int lengthOfLIS(vector<int>& nums) {
 int n = nums.size();
 vector<int> dp(n, 1);
 int best = 1;
 for (int i = 1; i < n; ++i) {
 for (int j = 0; j < i; ++j)
 if (nums[j] < nums[i])
 dp[i] = max(dp[i], dp[j] + 1);
 best = max(best, dp[i]);
 }
 return best;
}

```



**$O(n \log n)$  with patience sorting:**

```
int lengthOfLIS(vector<int>& nums) {
 vector<int> tails;
 for (int x : nums) {
 auto it = lower_bound(tails.begin(), tails.end(), x);
 if (it == tails.end()) tails.push_back(x);
 else *it = x;
 }
 return tails.size();
}
```

## 6.4 Edit Distance (Levenshtein Distance)

**Problem:** Minimum number of insertions, deletions, substitutions to convert string A to B.

**DP Relation:**

```
dp[i][j] = dp[i-1][j-1] if A[i-1] == B[j-1]
 = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) otherwise
```

```
int minDistance(string word1, string word2) {
 int m = word1.size(), n = word2.size();
 vector<vector<int>> dp(m+1, vector<int>(n+1));
 for (int i = 0; i <= m; ++i) dp[i][0] = i;
 for (int j = 0; j <= n; ++j) dp[0][j] = j;
 for (int i = 1; i <= m; ++i)
 for (int j = 1; j <= n; ++j)
 if (word1[i-1] == word2[j-1])
 dp[i][j] = dp[i-1][j-1];
 else
 dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
 return dp[m][n];
}
```

## 7. Subset and Knapsack Problems

## 7.1 0/1 Knapsack

**Problem:** Maximise value with capacity  $W$ , each item taken at most once.

**DP:**  $dp[w] = \text{max value achievable with capacity } w$ .

```
int knapsack01(vector<int>& weights, vector<int>& values, int W) {
 int n = weights.size();
 vector<int> dp(W+1, 0);
 for (int i = 0; i < n; ++i)
 for (int w = W; w >= weights[i]; --w)
 dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
 return dp[W];
}
```

## 7.2 Subset Sum

**Problem:** Determine if a subset sums to a given target.

```
bool subsetSum(vector<int>& nums, int target) {
 vector<bool> dp(target+1, false);
 dp[0] = true;
 for (int x : nums)
 for (int s = target; s >= x; --s)
 if (dp[s - x]) dp[s] = true;
 return dp[target];
}
```

## 7.3 Partition Equal Subset Sum

**Problem:** Can array be partitioned into two subsets with equal sum?

**Approach:** Check if subset sum exists for  $\text{total}/2$ .

```
bool canPartition(vector<int>& nums) {
 int sum = accumulate(nums.begin(), nums.end(), 0);
 if (sum % 2) return false;
 return subsetSum(nums, sum/2);
}
```

## 8. DP on Strings

### 8.1 Palindrome Partitioning (Minimum Cuts)

**Problem:** Minimum cuts to partition a string into palindromes.

**DP:** Precompute palindromic substrings. Then `dp[i] = min cuts for prefix s[0..i]`.

```
int minCut(string s) {
 int n = s.size();
 vector<vector<bool>> isPal(n, vector<bool>(n, false));
 for (int len = 1; len <= n; ++len)
 for (int i = 0; i + len - 1 < n; ++i) {
 int j = i + len - 1;
 if (s[i] == s[j] && (len <= 2 || isPal[i+1][j-1]))
 isPal[i][j] = true;
 }
 vector<int> dp(n, INT_MAX);
 for (int i = 0; i < n; ++i) {
 if (isPal[0][i]) dp[i] = 0;
 else {
 for (int j = 0; j < i; ++j)
 if (isPal[j+1][i])
 dp[i] = min(dp[i], dp[j] + 1);
 }
 }
 return dp[n-1];
}
```

### 8.2 Wildcard Matching

**Problem:** Match pattern containing `?` (single char) and `*` (any sequence) with a string.

**DP:** `dp[i][j] = true` if `s[0..i-1]` matches `p[0..j-1]`.

```
bool isMatch(string s, string p) {
 int m = s.size(), n = p.size();
 vector<vector<bool>> dp(m+1, vector<bool>(n+1, false));
 dp[0][0] = true;
 for (int j = 1; j <= n; ++j)
 if (p[j-1] == '*') dp[0][j] = dp[0][j-1];
 for (int i = 1; i <= m; ++i)
 for (int j = 1; j <= n; ++j)
 if (p[j-1] == '*' && (dp[i-1][j] || dp[i][j-1])) dp[i][j] =
true;
 else if (p[j-1] == '?' || s[i-1] == p[j-1])
 dp[i][j] = dp[i-1][j-1];
 return dp[m][n];
}
```

## 8.3 Regular Expression Matching

Similar but with `*` meaning zero or more of the preceding character.

## 9. DP on Trees

### Maximum Independent Set in Tree (no two adjacent selected)

**DP:** For each node, compute `dp[node][0]` = max sum when node NOT selected, `dp[node][1]` = selected.

```
dp[node][0] = sum over children of max(dp[child][0], dp[child][1])
dp[node][1] = node.value + sum over children of dp[child][0]
```

```

void dfs(int u, int parent, vector<vector<int>>& adj, vector<int>& val,
vector<vector<int>>& dp) {
 dp[u][1] = val[u];
 dp[u][0] = 0;
 for (int v : adj[u]) {
 if (v == parent) continue;
 dfs(v, u, adj, val, dp);
 dp[u][0] += max(dp[v][0], dp[v][1]);
 dp[u][1] += dp[v][0];
 }
}

// answer = max(dp[root][0], dp[root][1])

```

## 10. DP with Bitmask – Travelling Salesman Problem (TSP)

**Problem:** Shortest path visiting all cities exactly once and returning to start.

**DP State:** `dp[mask][i]` = minimum distance to have visited set `mask` ending at city `i`.

```

int tsp(vector<vector<int>>& dist) {
 int n = dist.size();
 int FULL = (1 << n) - 1;
 vector<vector<int>> dp(1 << n, vector<int>(n, INT_MAX/2));
 dp[1][0] = 0; // start at city 0
 for (int mask = 1; mask < (1 << n); ++mask) {
 for (int i = 0; i < n; ++i) {
 if (!(mask & (1 << i)) || dp[mask][i] >= INT_MAX/2)
 continue;
 for (int j = 0; j < n; ++j) {
 if (mask & (1 << j)) continue;
 dp[mask | (1 << j)][j] = min(dp[mask | (1 << j)][j],
 dp[mask][i] + dist[i][j]);
 }
 }
 }
 int ans = INT_MAX;
 for (int i = 0; i < n; ++i)
 ans = min(ans, dp[FULL][i] + dist[i][0]);
 return ans;
}

```

**Time:**  $O(2^n * n^2)$ . **Space:**  $O(2^n * n)$ .

**Real-life analogy:** Delivery route planning – which order to visit customers to minimise travel.

## 11. Kadane's Algorithm (DP view)

Kadane's algorithm is a 1D DP for maximum subarray sum.

**DP Relation:**  $dp[i] = \max(nums[i], dp[i-1] + nums[i])$  where  $dp[i]$  is max sum of subarray ending at  $i$ .

```

int maxSubarraySum(vector<int>& nums) {
 int maxEndingHere = nums[0], maxSoFar = nums[0];
 for (int i = 1; i < nums.size(); ++i) {
 maxEndingHere = max(nums[i], maxEndingHere + nums[i]);
 maxSoFar = max(maxSoFar, maxEndingHere);
 }
 return maxSoFar;
}

```

## 12. Summary Table

| Problem Category | Classic Problem     | Recurrence / Approach                                  | Time Complexity           |
|------------------|---------------------|--------------------------------------------------------|---------------------------|
| 1D DP            | Climbing Stairs     | $dp[i] = dp[i-1] + dp[i-2]$                            | $O(n)$                    |
| 2D Grid          | Min Path Sum        | $dp[i][j] = \min(dp[i-1][j], dp[i][j-1]) + grid[i][j]$ | $O(mn)$                   |
| Subsequence      | LCS                 | match $\Rightarrow +1$ else $\max(\text{prev})$        | $O(mn)$                   |
| Subsequence      | LIS                 | $dp[i] = \max(dp[j]) + 1$ over $j < i$                 | $O(n^2)$ or $O(n \log n)$ |
| Edit Distance    | Levenshtein         | $1 + \min(\text{insert, delete, replace})$             | $O(mn)$                   |
| Knapsack         | 0/1 Knapsack        | $dp[w] = \max(dp[w], dp[w-wt] + val)$                  | $O(nW)$                   |
| String           | Wildcard Matching   | $dp[i][j]$ based on star handling                      | $O(mn)$                   |
| Tree DP          | Max Independent Set | node selected/unselected                               | $O(V+E)$                  |
| Bitmask DP       | TSP                 | $dp[\text{mask}][i] = \min$ over $j$                   | $O(2^n n^2)$              |
| 1D DP            | Kadane              | $\text{maxEndingHere} = \max(x, \text{cur} + x)$       | $O(n)$                    |

The next chapter will cover advanced algorithmic techniques (greedy, divide and conquer, two pointers, and union-find) and their applications.



# Chapter 13: Greedy Algorithms

Greedy algorithms build a solution step by step by always choosing the locally optimal choice (the one that looks best at the moment) without revisiting previous decisions. This chapter covers the greedy choice property, pattern recognition, classic problems (activity selection, fractional knapsack, job sequencing, Huffman coding, coin change, minimum platforms, Prim's and Kruskal's MST algorithms), and comparisons with dynamic programming.

## 1. Greedy Choice Property

**What:** A problem exhibits the greedy choice property if a globally optimal solution can be reached by making a sequence of locally optimal (greedy) choices.

**Optimal substructure:** The problem can be broken down into subproblems, and the greedy choice combined with an optimal solution to the remaining subproblem yields an optimal solution for the original problem.

**When to use greedy:**

- The problem can be solved by a series of choices, each selecting the “best” available at that moment.
- The choice does not depend on future choices.
- Greedy choice property and optimal substructure both hold.

**When greedy fails:**

- Problems that require considering trade-offs (e.g., 0/1 knapsack, longest path).
- Problems with overlapping subproblems that benefit from DP (e.g., edit distance, shortest path with negative edges).

**Real-life analogy:** Getting change with the fewest coins using standard denominations (quarters, dimes, nickels, pennies) – always pick the largest coin that fits. This greedy works for US coins but fails for arbitrary denominations (e.g., coins 1, 3, 4 – target 6: greedy picks  $4+1+1=3$  coins, but optimal is  $3+3=2$  coins).

## 2. Pattern Recognition: How to Identify Greedy Problems

Recognising that a problem can be solved greedily is often the most challenging part. Use the following heuristics:

## 2.1 Key Indicators

| Indicator                                        | Description                                                                                                                                                            | Example                                                                    |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Sorting + choosing best local option</b>      | The solution often starts with sorting the input (by start time, finish time, weight, ratio, etc.) and then making a single pass with a simple selection rule.         | Activity selection (sort by finish time), job sequencing (sort by profit). |
| <b>Maximisation or minimisation optimisation</b> | The problem asks for the maximum or minimum value, but unlike DP, subproblems do not overlap heavily.                                                                  | Minimum number of platforms, maximum profit with fractional items.         |
| <b>No need to revisit decisions</b>              | Once you make a choice, you never need to change it later. The greedy choice does not affect the feasibility of remaining choices in a way that requires backtracking. | Huffman coding (merge smallest frequencies, never unmerge).                |
| <b>Interval or resource allocation</b>           | Problems involving scheduling, intervals, or resource allocation often admit greedy solutions.                                                                         | Activity selection, minimum platforms, job sequencing.                     |
| <b>Invariant or exchange argument works</b>      | You can prove that if an optimal solution differs from the greedy choice, you can swap elements to obtain a solution no worse than the greedy one.                     | Fractional knapsack (exchange items with better ratio).                    |

## 2.2 Questions to Ask Yourself

- **Can I sort the input and then make a single pass?** Many greedy algorithms become obvious after sorting.
- **Does the problem ask for “maximum number” or “minimum cost” without requiring extensive state?** If the answer depends only on a few variables (e.g., last finish time, remaining capacity), greedy might work.
- **Does the local choice affect only the immediate remaining subproblem, not the entire future?** Example: In activity selection, picking the earliest-finishing activity leaves

the subproblem of scheduling remaining activities after that finish time – it does not change the structure of later choices.

- **Would a small counter-example break a naive greedy?** Test on a few custom cases. If greedy fails even on a simple case, you likely need DP.

## 2.3 Common Greedy Archetypes

| Archetype                                 | Typical Greedy Rule                                               | Example Problems                                         |
|-------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------|
| <b>Earliest finish time</b>               | Always choose the interval/activity that finishes first.          | Activity selection, minimum rooms for lectures.          |
| <b>Highest value per unit</b>             | Take items with best value/weight ratio first.                    | Fractional knapsack.                                     |
| <b>Largest profit / earliest deadline</b> | Schedule most profitable jobs as late as possible.                | Job sequencing with deadlines.                           |
| <b>Lowest frequency first</b>             | Merge the two smallest frequencies.                               | Huffman coding.                                          |
| <b>Smallest start time + sweep line</b>   | Sort all events, sweep from left to right, maintain active count. | Minimum number of platforms (train arrivals/departures). |
| <b>Largest coin that fits</b>             | Pick largest denomination not exceeding remaining amount.         | Coin change (canonical systems only).                    |
| <b>Smallest edge across cut</b>           | Add the cheapest edge connecting tree to outside.                 | Prim' s algorithm.                                       |
| <b>Smallest edge without cycle</b>        | Sort edges, add if no cycle.                                      | Kruskal' s algorithm.                                    |

## 2.4 When Greedy is Suspect (Use DP Instead)

- **Subproblems overlap heavily** (e.g., shortest path with negative edges, edit distance).
- **The problem involves “all subsets” or “order matters”** (e.g., 0/1 knapsack, TSP).
- **Local choice can affect feasibility of multiple future choices** in a non-trivial way.
- **The greedy choice fails on a small counter-example** – test before implementing.

**Rule of thumb:** If you are not 100% sure, try a DP solution first, then see if you can optimise to a greedy approach by proving the exchange property.

### 3. Activity Selection

**Problem:** Given  $n$  activities with start and finish times, select the maximum number of non-overlapping activities.

**Greedy choice:** Always pick the activity that finishes earliest. This leaves the most remaining time for other activities.

**Algorithm:**

1. Sort activities by finish time.
2. Pick the first activity.
3. For each subsequent activity, if its start time  $\geq$  last chosen finish time, select it.

```

class Activity {
 int start;
 int finish;

 Activity(int start, int finish) {
 this.start = start;
 this.finish = finish;
 }
}

Comparator<Activity> activityComparator = (a, b) -> a.finish - b.finish;

public static int activitySelection(List<Activity> activities) {
 activities.sort(activityComparator);
 int count = 1;
 int lastFinish = activities.get(0).finish;
 for (int i = 1; i < activities.size(); ++i) {
 if (activities.get(i).start >= lastFinish) {
 count++;
 lastFinish = activities.get(i).finish;
 }
 }
 return count;
}

```

**Time:**  $O(n \log n)$  due to sorting. **Space:**  $O(1)$  extra.

**Real-life analogy:** Scheduling the most meetings in a conference room – always choose the meeting that ends earliest, freeing the room for subsequent meetings.

## 4. Fractional Knapsack

**Problem:** Given items with weight and value, and a knapsack capacity, maximise total value by taking fractions of items (you can break items arbitrarily).

**Greedy choice:** Take items with the highest value per unit weight (value/weight ratio) first.

```

struct Item { int value, weight; };
bool compare(Item a, Item b) {
 return (double)a.value/a.weight > (double)b.value/b.weight;
}

double fractionalKnapsack(vector<Item>& items, int capacity) {
 sort(items.begin(), items.end(), compare);
 double totalValue = 0.0;
 for (Item& item : items) {
 if (capacity >= item.weight) {
 totalValue += item.value;
 capacity -= item.weight;
 } else {
 totalValue += item.value * ((double)capacity / item.weight);
 break;
 }
 }
 return totalValue;
}

```

**Time:**  $O(n \log n)$ . **Space:**  $O(1)$ .

**Why greedy works:** Because you can take fractions, filling the remaining capacity with the highest-ratio item is always optimal. This differs from 0/1 knapsack, where greedy fails.

**Real-life analogy:** You have a bag of limited size and want to carry the most valuable mixture of grains (e.g., gold dust, silver dust, copper dust). You take the most expensive dust first, possibly partially.

## 5. Job Sequencing with Deadlines

**Problem:** Each job has a deadline and a profit, takes unit time, and can be scheduled at most one job per time unit. Maximise total profit.

**Greedy choice:** Sort jobs by profit descending. For each job, schedule it at the latest available free slot before its deadline.

```

struct Job { int id, deadline, profit; };
bool compare(Job a, Job b) { return a.profit > b.profit; }

int jobSequencing(vector<Job>& jobs) {
 sort(jobs.begin(), jobs.end(), compare);
 int maxDeadline = 0;
 for (auto& j : jobs) maxDeadline = max(maxDeadline, j.deadline);
 vector<int> slot(maxDeadline + 1, -1);
 int totalProfit = 0;
 for (Job& j : jobs) {
 for (int t = j.deadline; t > 0; --t) {
 if (slot[t] == -1) {
 slot[t] = j.id;
 totalProfit += j.profit;
 break;
 }
 }
 }
 return totalProfit;
}

```

**Time:**  $O(n^2)$  worst case (can be improved to  $O(n \log n)$  using disjoint set union). **Space:**  $O(\text{maxDeadline})$ .

**Real-life analogy:** Freelancer accepting projects with deadlines and payments; you can only do one project per day, so you prioritise high-paying jobs and schedule them as late as possible to leave room for other jobs.

## 6. Huffman Coding

**Problem:** Given character frequencies, build a prefix-free binary code that minimises the total encoded length.

**Greedy choice:** Repeatedly merge the two nodes with smallest frequencies.

**Algorithm:**

1. Create a leaf node for each character with its frequency.
2. Insert all nodes into a min-heap.

3. While heap has more than one node:

- Extract two minimum frequency nodes.
- Create a new internal node with frequency = sum of the two.
- Make the two nodes its children.
- Insert the new node into the heap.

4. The remaining node is the root of the Huffman tree.

```
struct HuffmanNode {
 char ch;
 int freq;
 HuffmanNode *left, *right;
 HuffmanNode(char c, int f) : ch(c), freq(f), left(nullptr),
right(nullptr) {}
 HuffmanNode(int f, HuffmanNode* l, HuffmanNode* r) : ch('\0'),
freq(f), left(l), right(r) {}
};

struct Compare {
 bool operator()(HuffmanNode* a, HuffmanNode* b) { return a->freq >
b->freq; }
};

HuffmanNode* buildHuffmanTree(vector<pair<char, int>>& freqTable) {
 priority_queue<HuffmanNode*, vector<HuffmanNode*>, Compare> pq;
 for (auto& p : freqTable)
 pq.push(new HuffmanNode(p.first, p.second));
 while (pq.size() > 1) {
 HuffmanNode* left = pq.top(); pq.pop();
 HuffmanNode* right = pq.top(); pq.pop();
 HuffmanNode* parent = new HuffmanNode(left->freq + right->freq,
left, right);
 pq.push(parent);
 }
 return pq.top();
}

// Then traverse the tree to generate codes.
```



**Time:**  $O(n \log n)$  ( $n$  = number of unique characters). **Space:**  $O(n)$ .

**Real-life analogy:** Assigning shorter codes to common letters in Morse code (e.g., 'E' is '!') to minimise overall message length.

## 7. Coin Change (Greedy vs DP)

**Problem:** Find the minimum number of coins to make a given amount, given coin denominations.

**Greedy approach:** Always pick the largest coin that does not exceed the remaining amount.

**When greedy works:** When the coin system is *canonical* (e.g., US coins: 1,5,10,25). Example: amount = 30, coins [1,5,10,25] – greedy picks 25+5 = 2 coins.

**When greedy fails:** Non-canonical systems. Example: coins = [1,3,4], amount = 6. Greedy picks 4+1+1 = 3 coins, but optimal is 3+3 = 2 coins. For such cases, use DP.

**DP solution (minimum coins):**

```
int coinChangeDP(vector<int>& coins, int amount) {
 vector<int> dp(amount+1, amount+1);
 dp[0] = 0;
 for (int i = 1; i <= amount; ++i)
 for (int c : coins)
 if (c <= i)
 dp[i] = min(dp[i], dp[i-c] + 1);
 return dp[amount] > amount ? -1 : dp[amount];
}
```

**Takeaway:** Use greedy only if the coin system is proven canonical; otherwise use DP for exact minimum.

## 8. Minimum Number of Platforms (Interval Scheduling)

**Problem:** Given arrival and departure times of trains, find the minimum number of platforms needed so that no train waits.

**Greedy approach:** Sort arrivals and departures separately. Use two pointers to simulate platform usage, counting overlapping intervals.

```

int findPlatform(vector<int>& arr, vector<int>& dep) {
 sort(arr.begin(), arr.end());
 sort(dep.begin(), dep.end());
 int platforms = 0, maxPlatforms = 0;
 int i = 0, j = 0;
 int n = arr.size();
 while (i < n && j < n) {
 if (arr[i] <= dep[j]) {
 platforms++;
 i++;
 maxPlatforms = max(maxPlatforms, platforms);
 } else {
 platforms--;
 j++;
 }
 }
 return maxPlatforms;
}

```

**Time:**  $O(n \log n)$ . **Space:**  $O(1)$ .

**Real-life analogy:** A railway station where you need to determine the peak number of simultaneous trains.

## 9. Prim's Algorithm (Minimum Spanning Tree)

**Problem:** Given a connected weighted undirected graph, find a spanning tree (connects all vertices) with minimum total edge weight.

**Greedy choice:** Start from an arbitrary vertex, repeatedly add the smallest edge that connects a vertex in the current tree to a vertex outside the tree (cut property).

**Approach:** Use a min-heap to always pick the cheapest edge from the frontier.

```

using pii = pair<int, int>; // (weight, vertex)

int primMST(vector<vector<pii>>& adj) {
 int V = adj.size();
 vector<bool> inMST(V, false);
 priority_queue<pii, vector<pii>, greater<pii>> pq;
 pq.push({0, 0}); // start from vertex 0
 int totalWeight = 0;
 while (!pq.empty()) {
 auto [w, u] = pq.top(); pq.pop();
 if (inMST[u]) continue;
 inMST[u] = true;
 totalWeight += w;
 for (auto& [v, weight] : adj[u]) {
 if (!inMST[v])
 pq.push({weight, v});
 }
 }
 return totalWeight;
}

```

**Time:**  $O((V+E) \log V)$  with binary heap. **Space:**  $O(V)$ .

**Real-life analogy:** Laying fibre optic cable to connect all towns with minimum total cable length – always extend the network to the nearest unconnected town.

## 10. Kruskal's Algorithm (Minimum Spanning Tree)

**Problem:** Same as Prim's – find MST of a weighted undirected graph.

**Greedy choice:** Sort all edges by weight. Add the smallest edge that does not create a cycle (uses Union-Find to detect cycles).

```

struct DSU {
 vector<int> parent, rank;
 DSU(int n) {
 parent.resize(n); rank.resize(n, 0);
 for (int i = 0; i < n; ++i) parent[i] = i;
 }
 int find(int x) {
 if (parent[x] != x) parent[x] = find(parent[x]);
 return parent[x];
 }
 bool unite(int x, int y) {
 int rx = find(x), ry = find(y);
 if (rx == ry) return false;
 if (rank[rx] < rank[ry]) parent[rx] = ry;
 else if (rank[rx] > rank[ry]) parent[ry] = rx;
 else { parent[ry] = rx; rank[rx]++; }
 return true;
 }
};

int kruskalMST(vector<tuple<int,int,int>>& edges, int V) {
 // edges: (weight, u, v)
 sort(edges.begin(), edges.end());
 DSU dsu(V);
 int totalWeight = 0, edgesUsed = 0;
 for (auto& [w, u, v] : edges) {
 if (dsu.unite(u, v)) {
 totalWeight += w;
 edgesUsed++;
 if (edgesUsed == V-1) break;
 }
 }
 return totalWeight;
}

```

**Time:**  $O(E \log E)$  due to sorting +  $O(E \alpha(V))$  for Union-Find. **Space:**  $O(V)$ .

**Real-life analogy:** Connecting towns by always choosing the cheapest road that does not create a loop – the towns gradually form a single connected network.

## Prim vs Kruskal

| Feature              | Prim                             | Kruskal                         |
|----------------------|----------------------------------|---------------------------------|
| Graph representation | Adjacency list                   | Edge list                       |
| Data structure       | Min-heap                         | Sorting + DSU                   |
| Best for             | Dense graphs ( $E \approx V^2$ ) | Sparse graphs ( $E \approx V$ ) |
| Start vertex         | Any vertex                       | Not needed                      |
| Cycle detection      | Via visited set                  | Via DSU                         |

## 11. When NOT to Use Greedy

| Problem                                       | Why Greedy Fails                                                                  | Better Approach                     |
|-----------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------|
| 0/1 Knapsack                                  | Taking highest value/weight may leave capacity that cannot be filled fractionally | DP                                  |
| Longest Path in DAG (unweighted)              | Greedy (take edge to next node with many outgoing) does not guarantee longest     | DP on DAG or BFS with modifications |
| Coin change (arbitrary denominations)         | Greedy may pick large coin that prevents optimal combination                      | DP                                  |
| Minimum cost to reach end with variable jumps | Not every locally shortest jump leads to global optimum                           | DP or Dijkstra                      |

## 12. Summary Table

| Problem            | Greedy Choice        | Time Complexity | Notes                              |
|--------------------|----------------------|-----------------|------------------------------------|
| Activity Selection | Earliest finish time | $O(n \log n)$   | Optimal for non-weighted intervals |

|                         |                                              |                                    |                              |
|-------------------------|----------------------------------------------|------------------------------------|------------------------------|
| Fractional Knapsack     | Highest value/weight                         | $O(n \log n)$                      | Allows fractions             |
| Job Sequencing          | Highest profit, schedule as late as possible | $O(n^2)$ or $O(n \log n)$ with DSU | Unit time jobs               |
| Huffman Coding          | Merge smallest frequencies                   | $O(n \log n)$                      | Produces optimal prefix code |
| Coin Change (canonical) | Largest coin $\leq$ remaining                | $O(n \log n)$ if sorted            | Fails for non-canonical      |
| Minimum Platforms       | Sweep line over sorted times                 | $O(n \log n)$                      | Counts maximum overlaps      |
| Prim's MST              | Smallest edge from tree to outside           | $O((V+E) \log V)$                  | Dense graphs                 |
| Kruskal's MST           | Smallest edge not forming cycle              | $O(E \log E)$                      | Sparse graphs + DSU          |

The next chapter will cover advanced topics: divide and conquer (master theorem, closest pair, Strassen's matrix multiplication) and algorithmic paradigms.

# Chapter 14: Important Interview Topics

This chapter synthesises key comparisons between data structures, algorithms, and problem-solving paradigms that frequently appear in technical interviews. Each section highlights trade-offs, use cases, and typical interview questions.

## 1. Array vs Linked List

| Feature                                   | Array                              | Linked List                                        |
|-------------------------------------------|------------------------------------|----------------------------------------------------|
| Memory                                    | Contiguous block                   | Non-contiguous, nodes linked by pointers           |
| Access by index                           | $O(1)$                             | $O(n)$ (traversal required)                        |
| Insertion/Deletion at beginning           | $O(n)$ (shift elements)            | $O(1)$ (update head pointer)                       |
| Insertion/Deletion at end (if tail known) | Amortized $O(1)$ for dynamic array | $O(1)$ with tail pointer                           |
| Insertion/Deletion at arbitrary position  | $O(n)$                             | $O(n)$ to find position, $O(1)$ to update pointers |
| Memory overhead                           | Minimal                            | Extra pointer storage (8 bytes per node on 64-bit) |
| Cache locality                            | Excellent (prefetching)            | Poor (pointer chasing)                             |
| Dynamic sizing                            | Fixed or amortized resizing        | Naturally dynamic                                  |

### Interview tips:

- Use array when you need frequent random access and know approximate size.
- Use linked list when you need frequent insertions/deletions at known positions (e.g., LRU cache).
- **Typical question:** Implement a linked list reversal, detect cycle, find middle.

## 2. Stack vs Queue vs Heap

| Feature            | Stack (LIFO)                  | Queue (FIFO)                | Heap (Priority Queue)                        |
|--------------------|-------------------------------|-----------------------------|----------------------------------------------|
| Access order       | Last-in-first-out             | First-in-first-out          | By priority (min or max)                     |
| Typical $O(1)$ ops | push, pop, top                | enqueue, dequeue, front     | getMin/getMax ( $O(1)$ ), insert $O(\log n)$ |
| Implementation     | Array, linked list            | Circular array, linked list | Binary heap (array)                          |
| Use cases          | Function calls, undo, parsing | BFS, scheduling, buffering  | Dijkstra, Huffman coding, Kth largest        |

#### Interview tips:

- **Stack:** Balanced parentheses, next greater element, evaluate postfix.
- **Queue:** Level order traversal, sliding window maximum (with deque).
- **Heap:** Merge K sorted lists, find median in stream.

## 3. Recursion vs Iteration

| Aspect           | Recursion                                                            | Iteration                                                |
|------------------|----------------------------------------------------------------------|----------------------------------------------------------|
| Code clarity     | Concise for naturally recursive problems (trees, divide-and-conquer) | More verbose but often easier to follow for simple loops |
| Performance      | Function call overhead                                               | No call overhead                                         |
| Space complexity | $O(\text{depth})$ stack space                                        | Usually $O(1)$ extra space                               |
| Risk             | Stack overflow for deep recursion                                    | No stack overflow                                        |
| Tail recursion   | Can be optimised into iteration by compiler (not guaranteed)         | Not applicable                                           |

#### When to use recursion:

- Tree/graph traversals (DFS)
- Divide-and-conquer (merge sort, quick sort)
- Backtracking (N-Queens, permutations)



### When to use iteration:

- Deep recursion ( $n > 10^5$ )
- Performance-critical loops
- When constant space is required

**Interview tip:** Often you are asked to implement both (e.g., Fibonacci, binary search, tree traversal). Know how to convert recursion to iteration using an explicit stack.

## 4. DFS vs BFS

| Feature                    | DFS                                                       | BFS                                              |
|----------------------------|-----------------------------------------------------------|--------------------------------------------------|
| Data structure             | Stack (implicit call stack or explicit)                   | Queue                                            |
| Space complexity           | $O(\text{depth})$ – can be $O(n)$ for skewed tree         | $O(\text{width})$ – often larger for wide graphs |
| Shortest path (unweighted) | Finds a path, not guaranteed shortest                     | Guarantees shortest path in unweighted graphs    |
| Cycle detection            | Easy (recursion stack)                                    | Possible but needs extra tracking                |
| Topological sort           | DFS-based (postorder)                                     | Kahn's algorithm (BFS)                           |
| Best for                   | Connected components, puzzles (maze), solving constraints | Shortest path, web crawling, level order         |

### Interview tips:

- Use BFS for shortest path in grid (word ladder, number of islands distance).
- Use DFS for problems requiring exhaustive search (permutations, subsets) or when graph is deep but narrow.
- Be ready to implement both traversals on graphs and trees.

## 5. Hash Map vs Tree Map

| Feature | Hash Map<br>(unordered_map) | Tree Map (map) |
|---------|-----------------------------|----------------|
|---------|-----------------------------|----------------|

|                                       |                                    |                                                      |
|---------------------------------------|------------------------------------|------------------------------------------------------|
| Underlying structure                  | Hash table                         | Balanced BST (e.g., Red-Black tree)                  |
| Average time for insert/search/delete | $O(1)$                             | $O(\log n)$                                          |
| Worst-case time                       | $O(n)$ (hash collisions)           | $O(\log n)$ guaranteed                               |
| Ordering                              | No                                 | Sorted by key                                        |
| Memory overhead                       | Hash table load factor             | Node pointers (left, right, color)                   |
| Use cases                             | Fast lookups, counting frequencies | Need ordered iteration, range queries, floor/ceiling |

**Interview tip:** In C++, `std::unordered_map` is hash map, `std::map` is tree map. Know when each is preferred. For range queries (e.g., count keys between L and R), tree map is essential.

## 6. Comparison of Sorting Algorithms

| Algorithm | Best          | Average       | Worst         | Space       | Stable | In-place | Adaptive |
|-----------|---------------|---------------|---------------|-------------|--------|----------|----------|
| Bubble    | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes    | Yes      | Yes      |
| Selection | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$      | No     | Yes      | No       |
| Insertion | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes    | Yes      | Yes      |
| Merge     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | Yes    | No       | No       |
| Quick     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | No     | Yes      | No       |
| Heap      | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$      | No     | Yes      | No       |
| Counting  | $O(n+k)$      | $O(n+k)$      | $O(n+k)$      | $O(k)$      | Yes    | No       | No       |
| Radix     | $O(nk)$       | $O(nk)$       | $O(nk)$       | $O(n+k)$    | Yes    | No       | No       |

**Stability** – equal elements preserve relative order. Important when sorting by multiple keys (e.g., sort by name, then by age).

**Interview tip:**

- Quick sort is often fastest in practice for arrays (good cache locality).
- Merge sort is stable and used for linked lists (no extra array overhead) and external sorting.
- Counting/Radix sort when integer range is limited (e.g., sorting 1 million ages 0-120).

## 7. Singly vs Doubly Linked List

| Feature                                 | Singly Linked List                     | Doubly Linked List                    |
|-----------------------------------------|----------------------------------------|---------------------------------------|
| Node structure                          | data + next                            | data + prev + next                    |
| Memory per node                         | Lower                                  | Higher (extra pointer)                |
| Reverse traversal                       | Not possible (without $O(n)$ reversal) | $O(1)$ backward                       |
| Deletion of a given node (without prev) | $O(n)$ to find previous                | $O(1)$ using prev pointer             |
| Insertion before a node                 | $O(n)$                                 | $O(1)$                                |
| Typical use cases                       | Stack, queue, adjacency list           | LRU cache, undo/redo, browser history |

**Interview tip:** Many problems (e.g., palindrome linked list, reverse in groups) are simpler with doubly linked list but must often be solved with singly linked list to test pointer manipulation.

## 8. Binary Search Tree vs Hash Table

| Feature                              | BST (balanced)  | Hash Table                        |
|--------------------------------------|-----------------|-----------------------------------|
| Average search/insert/delete         | $O(\log n)$     | $O(1)$                            |
| Worst-case (unbalanced/ collisions)  | $O(n)$          | $O(n)$                            |
| Ordering                             | Keys are sorted | No order                          |
| Range queries (find between L and R) | $O(\log n + k)$ | Not possible (requires full scan) |
| Predecessor / successor              | $O(\log n)$     | Not supported                     |

|        |               |                      |
|--------|---------------|----------------------|
| Memory | Node pointers | Load factor overhead |
|--------|---------------|----------------------|

**Interview tip:** Use hash table for exact lookups (dictionary). Use BST when ordering matters (e.g., all keys between x and y, find closest value). C++ `std::set` (ordered set) vs `std::unordered_set` (hash set).

## 9. When to Use DP vs Greedy vs Backtracking vs Divide and Conquer

| Paradigm                   | When to Use                                                             | Example                                           |
|----------------------------|-------------------------------------------------------------------------|---------------------------------------------------|
| <b>Divide and Conquer</b>  | Subproblems are independent, no overlap                                 | Merge sort, closest pair, Karatsuba               |
| <b>Dynamic Programming</b> | Overlapping subproblems + optimal substructure                          | Knapsack, LCS, edit distance                      |
| <b>Greedy</b>              | Locally optimal choice leads to global optimum (greedy choice property) | Activity selection, fractional knapsack, Dijkstra |
| <b>Backtracking</b>        | Need to explore all possibilities, constraint satisfaction              | N-Queens, Sudoku, permutations                    |

**Decision flow:**

1. Can you break into smaller independent problems? → D&C.
2. Do subproblems overlap? → DP (memoisation/tabulation).
3. Does a simple local choice always lead to optimum? → Greedy (test with counterexample).
4. Need to search all configurations? → Backtracking.

**Interview tip:** For optimisation problems (max/min), DP is often the answer unless greedy holds. For counting all solutions, backtracking.

## 10. Union-Find (Disjoint Set Union) Applications

**What:** Data structure that tracks elements partitioned into disjoint sets. Supports `find` (which set?) and `union` (merge two sets) in nearly  $O(1)$  with path compression + union by rank.

## Applications:

- **Kruskal's MST algorithm** – detect cycles when adding edges.
- **Dynamic connectivity** – determine if two nodes are connected in an undirected graph after adding edges.
- **Number of connected components** in a graph.
- **Union-find on grid** – connect adjacent cells (e.g., number of islands, regions).
- **Equivalence relation problems** – e.g., friends in a social network, grouping by common factors.

**Example** (union-find on 2D grid to count islands – alternative to DFS):

```
int find(vector<int>& parent, int x) {
 if (parent[x] != x) parent[x] = find(parent, parent[x]);
 return parent[x];
}

void unite(vector<int>& parent, vector<int>& rank, int x, int y) {
 int rx = find(parent, x), ry = find(parent, y);
 if (rx == ry) return;
 if (rank[rx] < rank[ry]) parent[rx] = ry;
 else if (rank[rx] > rank[ry]) parent[ry] = rx;
 else { parent[ry] = rx; rank[rx]++; }
}
```

**Interview tip:** Know the two optimisations – path compression and union by rank/size. Be able to implement DSU from scratch.

## 11. Trie vs Hash Table for String Search

| Feature                       | Trie                          | Hash Table                                 |
|-------------------------------|-------------------------------|--------------------------------------------|
| Search exact string           | $O(L)$ where $L$ = length     | $O(L)$ average (hash computation)          |
| Search prefix / auto-complete | $O(L)$ to traverse            | Not possible (would need to scan all keys) |
| Memory                        | Potentially high (many nodes) | Lower for few strings                      |

|                |                                            |                                   |
|----------------|--------------------------------------------|-----------------------------------|
| Delete         | $O(L)$                                     | $O(1)$ average                    |
| Space overhead | Node pointers per character                | Hash table load factor            |
| Use cases      | Dictionary with prefix queries, IP routing | Exact lookups, frequency counting |

### When to use Trie:

- Auto-complete (prefix search)
- Spell checking (find words with small edit distance)
- Longest common prefix
- Word break problems (DP + trie)

### When to use Hash Table:

- Simple existence check
- Counting word frequencies
- When prefixes are never needed

**Interview tip:** Many problems can be solved with both (e.g., word search, word break), but trie gives  $O(L)$  prefix operations and is often expected for prefix-related tasks.

## 12. Two-Pointer vs Sliding Window

| Feature                    | Two-Pointer                                                                             | Sliding Window                                                                       |
|----------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Typical structure          | Two pointers moving towards each other (opposite) or same direction at different speeds | Left and right pointers, window expands/contracts                                    |
| Use cases                  | Palindromes, pair sum in sorted array, removing duplicates, merge sorted arrays         | Subarray/substring problems with constraints (sum, distinct characters)              |
| Window size                | Fixed or variable (but usually not maintaining a range)                                 | Variable or fixed, always a contiguous segment                                       |
| Condition to move pointers | Usually based on comparison of values at both ends                                      | Usually based on constraint (e.g., $\text{sum} \leq K$ , at most $K$ distinct chars) |

|                 |        |        |
|-----------------|--------|--------|
| Time complexity | $O(n)$ | $O(n)$ |
|-----------------|--------|--------|

### Examples:

- **Two-pointer opposite:** `isPalindrome`, `twoSum` in sorted array.
- **Two-pointer same direction:** `removeDuplicates`, `findMiddle`.
- **Sliding window fixed size:** maximum sum of subarray size  $K$ .
- **Sliding window variable size:** longest substring without repeating characters.

**Interview tip:** Sliding window is a specialised two-pointer technique where pointers define a contiguous window. Recognising which to use is key: if the answer requires a contiguous segment with a constraint → sliding window; if ordering of elements matters (e.g., checking palindrome, pair sum) → two-pointer.

## Summary Table of Comparisons

| Comparison                       | Deciding Factor                                                                       |
|----------------------------------|---------------------------------------------------------------------------------------|
| Array vs Linked List             | Need random access (array) vs frequent insert/delete at known positions (linked list) |
| Stack vs Queue vs Heap           | Access order: LIFO, FIFO, or by priority                                              |
| Recursion vs Iteration           | Depth of recursion, code clarity, stack space                                         |
| DFS vs BFS                       | Shortest path (BFS) vs exhaustive search (DFS)                                        |
| Hash Map vs Tree Map             | Need ordering (tree) vs faster average (hash)                                         |
| Sorting algorithms               | Input size, stability, in-place requirement                                           |
| Singly vs Doubly                 | Need backward traversal / $O(1)$ delete given node                                    |
| BST vs Hash Table                | Range queries & ordering (BST) vs exact lookups (hash)                                |
| DP / Greedy / Backtracking / D&C | Overlap, optimal substructure, greedy choice                                          |
| Union-Find                       | Dynamic connectivity, cycle detection in Kruskal                                      |
| Trie vs Hash Table               | Prefix queries (trie) vs exact matches (hash)                                         |

|                               |                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------|
| Two-pointer vs Sliding Window | Contiguous segment constraint (sliding window) vs pairwise or multi-pointer (two-pointer) |
|-------------------------------|-------------------------------------------------------------------------------------------|

Use this chapter as a quick reference before interviews to refresh key trade-offs and problem-solving patterns.



# Chapter 15: Complexity Analysis & Numerical Problems

This chapter covers advanced complexity analysis techniques (recurrence relations, Master Theorem, amortized analysis, space-time trade-offs) and fundamental numerical algorithms (fast exponentiation, Sieve of Eratosthenes, GCD, modular exponentiation, large factorial handling, Catalan numbers). Each topic includes real-life analogies and visual diagrams.

## 1. Recurrence Relations and the Master Theorem

**What:** A recurrence relation defines a function in terms of its values on smaller inputs. It is used to describe the time complexity of divide-and-conquer algorithms.

**General form:**

$$T(n) = a * T(n/b) + f(n)$$

where:

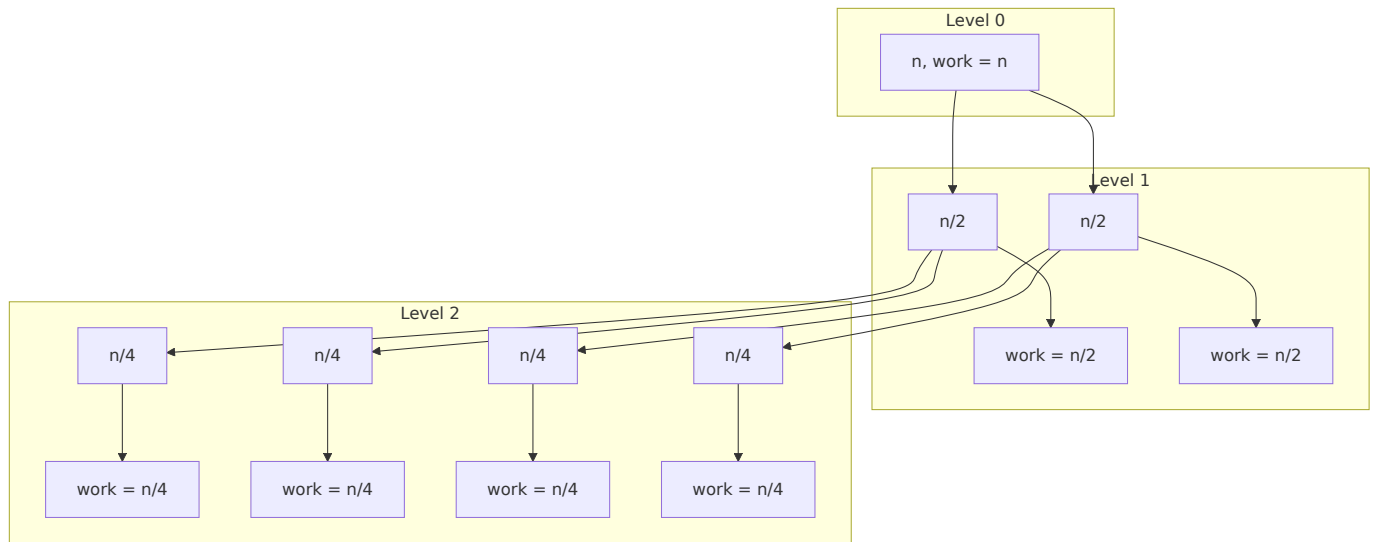
- $a$  = number of subproblems
- $n/b$  = size of each subproblem
- $f(n)$  = cost of dividing and combining (outside the recursion)

### 1.1 Solving Recurrences – The Master Theorem

For recurrences of the form  $T(n) = a T(n/b) + O(n^d)$  (where  $d \geq 0$ ), the solution is:

| Condition | Time Complexity          |
|-----------|--------------------------|
| $a < b^d$ | $T(n) = O(n^d)$          |
| $a = b^d$ | $T(n) = O(n^d \log n)$   |
| $a > b^d$ | $T(n) = O(n^{\log_b a})$ |

**Recursion tree visualisation** (for  $a = 2, b = 2, d = 1$  – merge sort):



Each level does total work  $O(n)$ . There are  $\log_2 n$  levels  $\rightarrow O(n \log n)$ .

**Real-life analogy:** Sorting a large file by repeatedly splitting it into halves, sorting each half, and merging – the number of splits is logarithmic, and each merge step touches every element.

## 1.2 Common Recurrences

| Algorithm            | Recurrence                      | Solution       |
|----------------------|---------------------------------|----------------|
| Binary search        | $T(n) = T(n/2) + O(1)$          | $O(\log n)$    |
| Merge sort           | $T(n) = 2T(n/2) + O(n)$         | $O(n \log n)$  |
| Karatsuba            | $T(n) = 3T(n/2) + O(n)$         | $O(n^{1.585})$ |
| Naive multiplication | $T(n) = 4T(n/2) + O(n)$         | $O(n^2)$       |
| Fibonacci (naive)    | $T(n) = T(n-1) + T(n-2) + O(1)$ | $O(2^n)$       |

## 2. Amortized Analysis

**What:** Amortized analysis averages the cost of operations over a sequence, giving a tighter bound than worst-case per operation. It shows that even if some operations are expensive, the average cost remains low.

### 2.1 Dynamic Array (e.g., `std::vector`)

When a dynamic array runs out of capacity, it doubles its size (reallocates and copies). The cost of reallocation is  $O(n)$  but occurs only every  $2^k$  insertions.

**Amortized cost per insertion:**  $O(1)$ .

**Total cost for  $n$  insertions:**  $O(n)$ .

**Visualisation** (capacity doubling):

### Copy cost

-



1



2



4

### Capacity

1

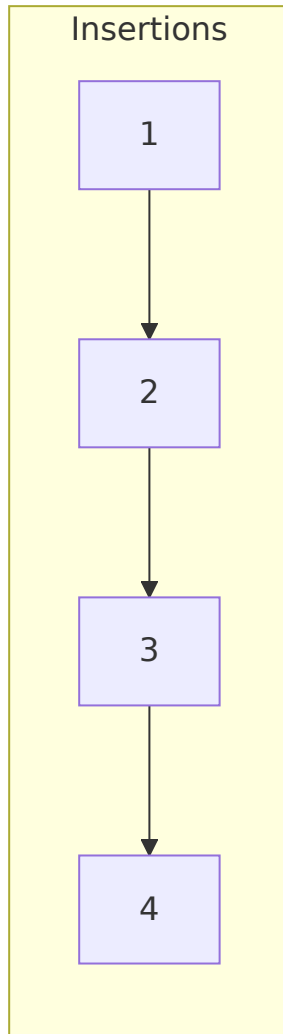
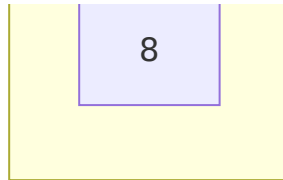


2



4





**Real-life analogy:** Moving to a larger apartment when the current one is full. The move is expensive, but you do it rarely – the average cost per month includes the occasional moving expense.

## 2.2 Binary Counter (Increment)

Incrementing a binary counter flips bits. The total number of bit flips over  $n$  increments is  $O(n)$  (each bit flips every  $2^k$  increments). Amortized cost per increment =  $O(1)$ .

## 2.3 Methods of Amortized Analysis

- **Aggregate method:** Sum total cost over sequence, divide by number of operations.
- **Accounting method:** Assign “credits” to cheap operations to pay for expensive ones.

- **Potential method:** Define a potential function that measures “energy” of data structure.

### 3. Space-Time Trade-offs

**What:** Often you can reduce time by using more memory, or reduce memory by accepting slower time. Choosing the right balance depends on constraints.

| Scenario                               | Trade-off                                                                             |
|----------------------------------------|---------------------------------------------------------------------------------------|
| Precomputation (prefix sums, DP table) | $O(n)$ extra space, $O(1)$ per query vs $O(n)$ per query without precomputation.      |
| Hashing                                | $O(n)$ space, $O(1)$ average lookup vs $O(\log n)$ with tree or $O(n)$ linear search. |
| Memoisation (DP)                       | Store computed results (extra memory) to avoid recomputation.                         |
| In-place algorithms                    | $O(1)$ space but may modify input; slower than out-of-place.                          |

**Real-life analogy:** Keeping a personal cookbook (memory) vs looking up recipes online each time (time). The cookbook takes space but saves time.

### 4. Numerical Problems

## 4.1 Power of a Number – Fast Exponentiation (Binary Exponentiation)

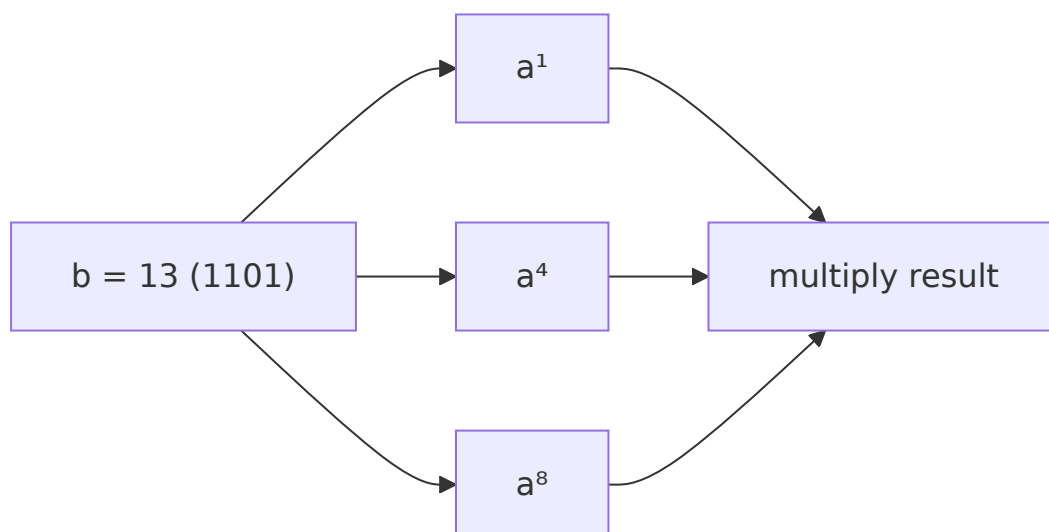
**Problem:** Compute  $a^b$  efficiently.

**Idea:** Use binary representation of exponent.

**Time:**  $O(\log b)$ .

```
long long fastPow(long long a, long long b) {
 long long result = 1;
 while (b > 0) {
 if (b & 1) result *= a;
 a *= a;
 b >>= 1;
 }
 return result;
}
```

**Visual** ( $a^{13} = a^{(1101_2)} = a^8 * a^4 * a^1$ ):



**Real-life analogy:** Instead of adding a number to itself 13 times, you double it repeatedly (1,2,4,8) and add those that match the binary digits of 13.

## 4.2 Sieve of Eratosthenes

**Problem:** Generate all prime numbers up to  $n$ .

**Approach:** Mark multiples of each prime starting from 2.

```
vector<bool> sieve(int n) {
 vector<bool> isPrime(n+1, true);
 isPrime[0] = isPrime[1] = false;
 for (int p = 2; p * p <= n; ++p) {
 if (isPrime[p]) {
 for (int multiple = p * p; multiple <= n; multiple += p)
 isPrime[multiple] = false;
 }
 }
 return isPrime;
}
```

**Time:**  $O(n \log \log n)$ . **Space:**  $O(n)$ .

**Visual** (n=30):

```
Initial: 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30
2 marks: 4 6 8 10 12 14 16 18 20 22 24 26 28
30
3 marks: 6 9 12 15 18 21 24 27
30
5 marks: 10 15 20 25 30
Remaining (primes): 2 3 5 7 11 13 17 19 23 29
```

**Real-life analogy:** In a classroom, you ask all students in even positions to sit down (leave only odd), then those in positions multiple of 3, etc. The ones still standing are at prime positions.

## 4.3 Greatest Common Divisor – Euclidean Algorithm

**What:** Find the largest integer dividing both numbers.

**Approach:**  $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ .



```
int gcd(int a, int b) {
 while (b) {
 int t = b;
 b = a % b;
 a = t;
 }
 return a;
}
```

**Time:**  $O(\log \min(a, b))$ .

**Proof:** Each step reduces the numbers at least by half.

**Real-life analogy:** Finding the largest square tile that can exactly cover a rectangular floor of size  $a \times b$ . You repeatedly cut squares of size  $b \times b$  from the  $a \times b$  rectangle; the remainder becomes the new problem.

## 4.4 Modular Exponentiation

**Problem:** Compute  $(a^b) \% m$  efficiently without overflow.

**Approach:** Combine fast exponentiation with modulo at each multiplication.

```
long long modPow(long long a, long long b, long long m) {
 long long result = 1;
 a %= m;
 while (b > 0) {
 if (b & 1) result = (result * a) % m;
 a = (a * a) % m;
 b >>= 1;
 }
 return result;
}
```

**Time:**  $O(\log b)$ . **Use cases:** Cryptography (RSA), hashing, avoiding overflow in large powers.

## 4.5 Factorial of Large Numbers (Handling Overflow)

For large  $n$ ,  $n!$  exceeds standard integer ranges. Two common approaches:

1. **Compute modulo** (typical in competitive programming):

```
fact[i] = (fact[i-1] * i) % MOD.
```

2. **Compute exact value** using arbitrary-precision (big integers) – C++ `__int128` or external libraries.

```
// Modulo factorial
long long factMod(int n, long long mod) {
 long long res = 1;
 for (int i = 2; i <= n; ++i) res = (res * i) % mod;
 return res;
}
```

**Real-life analogy:** Counting permutations of a large deck of cards. The exact number is astronomically large; often we only need the remainder modulo some number (e.g., for hashing).

## 4.6 Catalan Numbers

**Definition:** Count of many combinatorial structures:

- Number of valid bracket sequences with `n` pairs.
- Number of binary search trees with `n` nodes.
- Number of ways to triangulate a convex `(n+2)`-gon.

**Recurrence:**

$$C_0 = 1$$
$$C_{n+1} = \sum_{i=0}^n C_i * C_{n-i} \quad (\text{for } n \geq 0)$$

Closed form:  $C_n = (1/(n+1)) * \text{binom}(2n, n)$ .

**First few Catalan numbers:** 1, 1, 2, 5, 14, 42, 132, 429, ...

**Implementation** (using DP):

```

long long catalan(int n) {
 vector<long long> C(n+1, 0);
 C[0] = 1;
 for (int i = 1; i <= n; ++i)
 for (int j = 0; j < i; ++j)
 C[i] += C[j] * C[i-1-j];
 return C[n];
}

```

**Time:**  $O(n^2)$ . For large  $n$ , use closed form with modular arithmetic.

**Real-life analogy:** Number of ways to parenthesise a mathematical expression (e.g.,  $a+b+c+d$ ). For 4 numbers, the 5 possible binary tree structures correspond to the 5th Catalan number.

## 5. Summary Table

| Concept                | Key Formula / Method                | Time Complexity     | Space Complexity |
|------------------------|-------------------------------------|---------------------|------------------|
| Master Theorem         | $T(n) = aT(n/b) + O(n^d)$           | Case dependent      | -                |
| Amortized analysis     | Dynamic array doubling              | $O(1)$ amortised    | $O(n)$           |
| Fast exponentiation    | Binary exponentiation               | $O(\log b)$         | $O(1)$           |
| Sieve of Eratosthenes  | Mark multiples                      | $O(n \log \log n)$  | $O(n)$           |
| Euclidean GCD          | <code>while(b) { b, a%b }</code>    | $O(\log \min(a,b))$ | $O(1)$           |
| Modular exponentiation | modPow with exponentiation          | $O(\log b)$         | $O(1)$           |
| Catalan numbers (DP)   | $C_0=1, C_{n+1} = \sum C_i C_{n-i}$ | $O(n^2)$            | $O(n)$           |

The next chapter will cover system design fundamentals for coding interviews (scalability, caching, load balancing, database indexing, etc.).

# Chapter 16: Advanced & Miscellaneous Topics

This chapter covers algorithms and data structures that appear frequently in intermediate to advanced coding interviews but do not fit neatly into other chapters. Topics include monotonic stack, deque for sliding window, ordered sets, rolling hash, majority vote algorithms, reservoir sampling, interval merging, meeting rooms II, and randomized algorithms.

## 1. Monotonic Stack

**What:** A stack that maintains elements in increasing or decreasing order. It is used to find the next greater (or smaller) element for each position in  $O(n)$  time.

### 1.1 Next Greater Element

**Problem:** For each element in an array, find the next element to the right that is greater.

**Algorithm:** Iterate from left to right, maintaining a decreasing stack (from bottom to top). For each new element, pop while stack top is smaller – the current element is the next greater for those popped indices.

```
List<Integer> nextGreaterElement(List<Integer> nums) {
 int n = nums.size();
 List<Integer> result = new ArrayList<>(Collections.nCopies(n, -1));
 Deque<Integer> st = new ArrayDeque<>(); // stores indices
 for (int i = 0; i < n; ++i) {
 while (!st.isEmpty() && nums.get(st.peek()) < nums.get(i)) {
 result.set(st.pop(), nums.get(i));
 }
 st.push(i);
 }
 return result;
}
```

**Time:**  $O(n)$ . **Space:**  $O(n)$ .

**Real-life analogy:** In a line of people, find the next taller person for each.

## 1.2 Largest Rectangle in Histogram

**Problem:** Given bar heights, find the largest rectangular area in the histogram.

**Approach:** Use a monotonic increasing stack (store indices with increasing heights). When a shorter bar is encountered, pop taller bars and calculate area with the popped bar as the smallest height.

```
int largestRectangleArea(List<Integer> heights) {
 Deque<Integer> st = new ArrayDeque<>();
 int maxArea = 0;
 List<Integer> h = new ArrayList<>(heights);
 h.add(0); // sentinel
 for (int i = 0; i < h.size(); ++i) {
 while (!st.isEmpty() && h.get(st.peek()) > h.get(i)) {
 int height = h.get(st.pop());
 int left = st.isEmpty() ? -1 : st.peek();
 maxArea = Math.max(maxArea, height * (i - left - 1));
 }
 st.push(i);
 }
 return maxArea;
}
```

**Time:**  $O(n)$ . **Space:**  $O(n)$ .

## 2. Deque Applications – Sliding Window Maximum

**Problem:** Find the maximum in every contiguous subarray of size  $k$ .

**Approach:** Maintain a deque storing indices of candidate maximums. The deque is kept in decreasing order of values. For each new index  $i$ :

- Remove indices from front that are out of the window ( $i - k$ ).
- Remove indices from back whose values are  $\leq$  current value (they will never be maximum).
- Push current index.

- The front is the maximum for the window ending at `i` (when `i >= k-1`).

```
List<Integer> slidingWindowMaximum(List<Integer> nums, int k) {
 Deque<Integer> dq = new ArrayDeque<>();
 List<Integer> result = new ArrayList<>();
 for (int i = 0; i < nums.size(); ++i) {
 if (!dq.isEmpty() && dq.peekFirst() == i - k) dq.pollFirst();
 while (!dq.isEmpty() && nums.get(dq.peekLast()) <= nums.get(i))
 dq.pollLast();
 dq.offerLast(i);
 if (i >= k - 1) result.add(nums.get(dq.peekFirst()));
 }
 return result;
}
```

**Time:**  $O(n)$ . **Space:**  $O(k)$ .

**Real-life analogy:** A moving magnifying glass over a number line – you always keep the largest visible number at the front of the queue.

### 3. Multiset and Ordered Set (Balanced BST)

**What:** Data structures that maintain elements in sorted order and support insertion, deletion, and lookup in  $O(\log n)$ . In C++: `std::set` (unique keys) and `std::multiset` (allowing duplicates). Implemented as Red-Black trees.

**Use cases:**

- Maintaining a running median (use two multisets or heaps).
- Order statistics (find  $k$ th smallest in dynamic set).
- Range queries (elements between  $L$  and  $R$ ).

**Example: Running Median** using two multisets (max-heap and min-heap):

```

void addNum(int num, multiset<int>& left, multiset<int>& right) {
 if (left.empty() || num <= *left.rbegin()) left.insert(num);
 else right.insert(num);
 if (left.size() > right.size() + 1) {
 right.insert(*left.rbegin());
 left.erase(prev(left.end()));
 } else if (right.size() > left.size()) {
 left.insert(*right.begin());
 right.erase(right.begin());
 }
}

double findMedian(multiset<int>& left, multiset<int>& right) {
 if (left.size() > right.size()) return *left.rbegin();
 return (*left.rbegin() + *right.begin()) / 2.0;
}

```

**Real-life analogy:** A sorted list of exam scores that updates as new scores arrive – you always need the median quickly.

## 4. Rolling Hash (Rabin-Karp)

**What:** A hashing technique that computes the hash of a sliding window in  $O(1)$  after  $O(n)$  preprocessing. Used for pattern matching.

**Polynomial rolling hash:**

$$\text{hash}(s) = \sum s[i] * p^i \bmod M$$

When sliding the window, update:

$$\text{new\_hash} = (\text{old\_hash} - s[\text{left}] * p^{(\text{len}-1)}) * p + s[\text{right}+1]$$

```

#define MOD 1000000007
#define BASE 31

vector<int> rabinKarp(string text, string pattern) {
 int n = text.size(), m = pattern.size();
 if (m > n) return {};
 long long pHash = 0, tHash = 0, pow = 1;
 for (int i = 0; i < m; ++i) {
 pHash = (pHash * BASE + pattern[i]) % MOD;
 tHash = (tHash * BASE + text[i]) % MOD;
 if (i > 0) pow = (pow * BASE) % MOD;
 }
 vector<int> occurrences;
 for (int i = 0; i <= n - m; ++i) {
 if (pHash == tHash) {
 // double-check actual characters to avoid false positives
 if (text.substr(i, m) == pattern) occurrences.push_back(i);
 }
 if (i < n - m) {
 tHash = (tHash - text[i] * pow % MOD + MOD) % MOD;
 tHash = (tHash * BASE + text[i+m]) % MOD;
 }
 }
 return occurrences;
}

```

**Time:**  $O(n+m)$  average. **Space:**  $O(1)$ .

**Real-life analogy:** Quickly checking if a fingerprint (pattern) appears in a long strip of paper – you slide a window and compare a compact signature, only verifying when signatures match.

## 5. Boyer-Moore Majority Vote Algorithm

**Problem:** Find the majority element (element that appears more than  $\lfloor n/2 \rfloor$  times) in  $O(n)$  time and  $O(1)$  space.

**Algorithm:**



1. **Candidate selection:** Pair up different elements and cancel them. The remaining candidate is the only possible majority element.
2. **Verification:** Count occurrences of the candidate to confirm.

```
int majorityElement(vector<int>& nums) {
 int candidate = 0, count = 0;
 for (int x : nums) {
 if (count == 0) candidate = x;
 count += (x == candidate) ? 1 : -1;
 }
 // verification pass (optional if majority guaranteed)
 int freq = 0;
 for (int x : nums) if (x == candidate) freq++;
 return (freq > nums.size()/2) ? candidate : -1;
}
```

**Real-life analogy:** In a battle, each pair of different fighters eliminates each other. The last survivor could be the one with the majority.

## 6. Reservoir Sampling

**Problem:** Randomly select  $k$  elements from a stream of unknown length (or very large) with equal probability.

**Algorithm:** For  $i < k$ , keep all first  $k$  elements. For  $i \geq k$ , replace a random element with probability  $k/(i+1)$ .

```
vector<int> reservoirSample(vector<int>& stream, int k) {
 vector<int> reservoir(k);
 for (int i = 0; i < k; ++i) reservoir[i] = stream[i];
 for (int i = k; i < stream.size(); ++i) {
 int j = rand() % (i + 1);
 if (j < k) reservoir[j] = stream[i];
 }
 return reservoir;
}
```

**Proof of uniform probability:** Each element has probability  $k/n$  of being selected.

**Real-life analogy:** Randomly selecting 10 songs from a streaming service that keeps adding new songs – you maintain a representative sample.

## 7. Interval Merging and Partitioning

**Problem:** Merge overlapping intervals.

**Approach:** Sort intervals by start time, then iterate and merge if current start  $\leq$  last end.

```
vector<pair<int,int>> mergeIntervals(vector<pair<int,int>>& intervals) {
 if (intervals.empty()) return {};
 sort(intervals.begin(), intervals.end());
 vector<pair<int,int>> merged;
 merged.push_back(intervals[0]);
 for (auto& [start, end] : intervals) {
 if (start <= merged.back().second) {
 merged.back().second = max(merged.back().second, end);
 } else {
 merged.push_back({start, end});
 }
 }
 return merged;
}
```

**Time:**  $O(n \log n)$ . **Space:**  $O(n)$ .

**Real-life analogy:** Merging overlapping calendar appointments.

## 8. Meeting Rooms II (Minimum Meeting Rooms Required)

**Problem:** Given intervals (start, end), find the minimum number of rooms needed to schedule all meetings without overlap.

**Approach 1 – Sweep line:** Count simultaneous meetings by sorting all events (start +1, end -1) and using cumulative sum.

```

int minMeetingRooms(vector<pair<int,int>>& intervals) {
 vector<pair<int,int>> events; // (time, type): +1 for start, -1 for
 end
 for (auto& [s, e] : intervals) {
 events.emplace_back(s, 1);
 events.emplace_back(e, -1);
 }
 sort(events.begin(), events.end());
 int curr = 0, maxRooms = 0;
 for (auto& [time, delta] : events) {
 curr += delta;
 maxRooms = max(maxRooms, curr);
 }
 return maxRooms;
}

```

**Approach 2 – Two pointers** (sort starts and ends separately, similar to minimum platforms).

**Real-life analogy:** Determining the peak number of simultaneous lectures in a university.

## 9. Randomized Algorithms

Randomized algorithms use random numbers to simplify or speed up solutions. They often run faster on average and have small probability of error (or deterministic with high probability).

### 9.1 Quickselect (Randomized Selection)

**Problem:** Find the kth smallest element in unsorted array.

**Approach:** Randomly choose a pivot, partition the array, then recurse into one side.

```

int quickselect(vector<int>& nums, int left, int right, int k) {
 if (left == right) return nums[left];
 int pivotIdx = left + rand() % (right - left + 1);
 swap(nums[pivotIdx], nums[right]);
 int i = left - 1;
 for (int j = left; j < right; ++j)
 if (nums[j] <= nums[right]) swap(nums[++i], nums[j]);
 swap(nums[i+1], nums[right]);
 if (k == i+1) return nums[k];
 else if (k < i+1) return quickselect(nums, left, i, k);
 else return quickselect(nums, i+2, right, k);
}

```

**Expected time:**  $O(n)$ . **Worst case:**  $O(n^2)$  if pivots are always bad (rare with randomness).

## 9.2 Randomized Quicksort

Same as quicksort but the pivot is chosen randomly. This eliminates the worst-case  $O(n^2)$  for sorted input. Expected  $O(n \log n)$ .

## 10. Summary Table

| Topic                     | Core Idea                                   | Time               | Space  | Use Case                               |
|---------------------------|---------------------------------------------|--------------------|--------|----------------------------------------|
| Monotonic stack           | Maintain order to find next greater/smaller | $O(n)$             | $O(n)$ | Largest rectangle, stock span          |
| Deque for sliding window  | Maintain indices in decreasing order        | $O(n)$             | $O(k)$ | Sliding window max/min                 |
| Multiset / ordered set    | Balanced BST                                | $O(\log n)$ per op | $O(n)$ | Running median, order stats            |
| Rolling hash (Rabin-Karp) | Polynomial hash sliding                     | $O(n+m)$ avg       | $O(1)$ | Pattern matching, plagiarism detection |
| Boyer-Moore majority      | Pair cancellation                           | $O(n)$             | $O(1)$ | Majority element (freq > $n/2$ )       |

|                      |                          |                        |             |                       |
|----------------------|--------------------------|------------------------|-------------|-----------------------|
| Reservoir sampling   | Random replacement       | $O(n)$                 | $O(k)$      | Sample from stream    |
| Interval merging     | Sort by start, merge     | $O(n \log n)$          | $O(n)$      | Overlap merging       |
| Meeting Rooms II     | Sweep line (event count) | $O(n \log n)$          | $O(n)$      | Min rooms / platforms |
| Quickselect          | Randomized partition     | $O(n)$ expected        | $O(\log n)$ | kth smallest          |
| Randomized quicksort | Random pivot             | $O(n \log n)$ expected | $O(\log n)$ | General sorting       |

These advanced topics are often the difference between a good solution and an optimal one in coding interviews. Practice implementing each from scratch to build fluency.